

About This Phase

Phase 0 laid out the whole course and the rules you're working under: type every example yourself, never copy-paste, and build every lesson project. Phase 1 is where that actually starts. By the end of it, seventeen lessons from now, you'll have written real programs that read files, handle errors, and are organized into a proper installable package. Nothing here is filler; every lesson is something a later phase, or a later lesson in this same phase, genuinely depends on.

Every lesson follows the same structure: Why This Matters, Learning Objectives, the lesson itself, Common Mistakes to Avoid, and a Lesson Project.

Contents

Lesson 1.1: Why Python?	2
Lesson 1.2: Setup & First Program	5
Lesson 1.3: Variables & Data Types	8
Lesson 1.4: Operators	13
Lesson 1.5: Control Flow (if/elif/else)	19
Lesson 1.6: Loops (for and while)	25
Lesson 1.7: String Manipulation	31
Lesson 1.8: Lists & Tuples	39
Lesson 1.9: Dictionaries & Sets	46
Lesson 1.10: List/Dict/Set Comprehensions	54
Lesson 1.11: Functions	59
Lesson 1.12: Docstrings & Type Hints	69
Lesson 1.13: File Handling & JSON	75
Lesson 1.14: Error Handling (Try/Except)	84
Lesson 1.15: Packages & Imports	91
Lesson 1.16: Virtual Environments & Pip	96
Lesson 1.17: Everyday Libraries	101
Phase 1 Capstone Project	107

Lesson 1.1: Why Python?

Why This Matters

You're about to spend real hours of your life on this language, so it's worth five minutes understanding what you're actually getting for that time, and why Python specifically, out of the dozens of languages you could have picked instead.

Learning Objectives

By the end of this lesson, you will be able to:

- Explain, in plain language, what a programming language actually does
- Describe two things that make Python's design different from most other languages
- Name at least three real fields where Python is a dominant, industry-standard tool
- Explain the difference between an interpreted language and a compiled one, at a conceptual level

What a Programming Language Actually Does

A computer, underneath everything, only understands one thing: electrical signals that are either on or off, which we write as 1s and 0s. Nobody writes real software that way. A programming language is a translation layer: you write instructions in something close to English, and a program translates those instructions into the 1s and 0s the machine can actually run. Every language, from Python to C to JavaScript, exists to solve the same basic problem: letting a human describe a process precisely enough that a machine can execute it, without the human needing to think in binary.

Languages differ enormously in how close to “plain English” they let you get, and how much they force you to manage the computer's low-level details yourself, like where in memory a piece of data lives. Python sits deliberately far on the “close to English” end of that spectrum. A line like

```
1 if age >= 18:  
2     print("You can vote.")
```

reads almost like a sentence: “if age is at least 18, print that you can vote.” That's not an accident; it's Python's entire design philosophy. Compare that to how the same idea is often written in other languages, where a version of this same logic might read closer to `if(age>=18){printf("You can vote.");}`, packed with braces, parentheses, and a semicolon, none of which describe the actual idea any more clearly than the plain-English version does. Python strips away as much of that extra ceremony as it reasonably can, leaving mostly the part that actually expresses what you mean.

What Makes Python's Design Different

Two decisions, made early in Python's design, explain most of why it feels different from many other languages you'll hear about.

Indentation is syntax, not decoration. In many languages, the spacing you see is purely for human readability, and the computer only cares about symbols like curly braces `{}` to know where a block of code starts and ends; you could remove every line break and space from those languages, leaving one long unreadable line, and the program would still run identically. In Python, the indentation itself marks the block. That `print` statement above is indented because it belongs to the `if`; if it weren't indented, Python wouldn't consider it part of the `if` at all, and it would run every single time, regardless of age, as an entirely separate, unconditional line. This forces every Python program, written by anyone, to at least be laid out consistently, since inconsistent indentation isn't just ugly in Python, it's a

broken program: mixing how far different lines are indented, when they're meant to belong to the same block, raises an actual error before your program even starts running, rather than merely looking messy.

Python is dynamically typed. In some languages, you have to declare up front exactly what kind of data a variable will hold (a whole number, a piece of text, and so on) before you can use it, and the computer strictly enforces that declaration, refusing to even start the program if a number ever tries to sneak into a spot meant for text. Python doesn't require this: you can write `age = 25` and Python figures out on its own that `age` is holding a whole number, without you announcing it. You'll learn exactly how this works in Lesson 1.3. This makes Python noticeably faster to write and experiment with, since there's no upfront bookkeeping to do before you can try an idea out, at the cost of some errors only showing up when the program actually runs, and specifically only when that exact line finally executes, instead of every such mistake being caught before the program runs at all. Later lessons (Lesson 1.12 on type hints, and tools built on top of them) show you how to get some of that up-front safety back, when you want it, without giving up Python's flexibility.

On top of both of those, Python ships with a huge standard library (tools for handling files, dates, math, and text are already built in, no extra installation needed) and has one of the largest third-party package ecosystems of any language, which is exactly what lets Phase 3 hand you real, industrial-strength tools, like the `requests` library for talking to web APIs, instead of making you build everything from scratch.

Where Python Actually Gets Used

This isn't a language kept alive by nostalgia or tutorials. Python is a working, industry-standard tool in a genuinely wide range of fields, several of which this exact course walks you through building something real in:

- **Web backends and databases:** servers that power real apps and sites, handling requests, talking to databases, and returning data.
- **Automation and scripting:** small programs that do tedious, repetitive tasks (renaming hundreds of files, pulling data from a website, generating reports) automatically. Large parts of Phase 1 and all of Phase 3 are built around exactly this.
- **Data science and machine learning:** Python is the dominant language in both fields, for the same reason it's used here: readable syntax plus an enormous ecosystem of specialized libraries.

You aren't learning a toy language to be later replaced by a "real" one. Everything from here through Phase 3 is the real thing, at a genuinely professional level, just introduced patiently.

Interpreted, Not Compiled

When you eventually run a Python file (starting in the very next lesson), here's conceptually what happens: a program called the Python interpreter reads your code and executes it more or less line by line, translating and running as it goes, one instruction at a time, rather than translating the entire file up front. This is different from a compiled language like C, where an entire separate step, run before your program ever executes at all, first translates your whole program into a standalone binary file full of raw machine instructions, which you then run separately, entirely on its own, with the original source code no longer involved at all.

The practical difference you'll feel constantly: with Python, there's no separate "build" step standing between writing a line and seeing what it does. You write a file, you run it, you see the result, immediately, and if something's wrong, you fix that one line and run it again, immediately. That tight feedback loop is a large part of why Python is such a common choice for beginners and for fast, exploratory work alike, and it's the loop you'll be living in for the rest of this course. The trade-off, worth knowing about even though it won't matter much at this stage, is raw execution speed: a compiled program, already translated fully into machine instructions ahead of time, generally runs faster than an interpreted one translating as it goes, which is part of why some performance-critical software is still written in compiled languages. For the overwhelming majority of programs this course builds, that difference is never something you'll notice.

Try It Yourself 1.1: Before you’ve installed anything, pick one everyday task you do on a computer regularly (organizing photos, tracking expenses, checking a website for updates). Write down, in three to five plain-English steps, exactly what you do, in order, precisely enough that someone with zero context could follow it. That’s the actual skill underneath programming: breaking a task into an exact, ordered sequence of steps. Everything from Lesson 1.2 onward is about learning the vocabulary to hand that sequence to a computer.

Common Mistakes to Avoid

- **Trying to memorize everything before writing any code.** Programming is learned by doing, not by memorizing syntax in the abstract. You will forget the exact syntax for plenty of things in this course and look it up again, and that’s completely normal, not a sign you’re behind.
- **Comparing Python’s speed to compiled languages and concluding it’s “bad.”** Python trades some raw execution speed for a massive gain in how fast you can write and change code. For the vast majority of real-world programs, including everything you’ll build in this course, that trade is exactly the right one.
- **Assuming you need a computer science degree, or advanced math, to start.** You don’t. This course assumes none of that, and the entire roadmap in Phase 0 is built assuming an absolute beginner’s starting point.
- **Trying to learn Python and another language at the same time.** Splitting focus this early roughly doubles the time it takes to become comfortable with either one. Finish this course first.

Lesson 1.1 Project: Your Why

No code yet, on purpose; the setup happens in the very next lesson. Instead, write a short, plain-text note to yourself (in any text editor, or even on paper) answering three questions:

1. What is one specific thing you want to be able to build or automate by the end of this course?
2. Which phase of the course (from the Phase 0 roadmap) do you think that goal is closest to?
3. What’s your plan for actually following the one rule from Phase 0: typing every example yourself, instead of copy-pasting?

Keep this note. It’s worth rereading around the middle of Phase 3, when the material gets noticeably harder and it helps to remember why you started.

Lesson Summary: Quick Reference

- A programming language translates precise, human-written instructions into something a machine can execute
- Python’s indentation is syntax, not just style; it determines what belongs to what
- Python is dynamically typed: you don’t declare a variable’s type up front
- Python is a genuine industry standard in web backends, databases, and automation, not just a teaching language
- Python is interpreted: you run a file directly, with no separate build/compile step

Lesson 1.2: Setup & First Program

Why This Matters

Every lesson from here on assumes Python is installed and you can run a file from a terminal. This lesson is the one time in the whole course where the goal isn't a programming concept; it's getting your machine into a state where every later lesson just works.

Learning Objectives

By the end of this lesson, you will be able to:

- Install Python on your operating system and confirm it's working
- Use a terminal well enough to navigate folders and run a file
- Install and configure VS Code as your code editor
- Write and run your first real Python program

Installing Python

Head to python.org/downloads. The site detects your operating system and offers the right installer.

Windows: run the installer, and on the very first screen, tick the checkbox that says “Add Python to PATH” before clicking Install. This single checkbox is the single most common thing beginners miss, and skipping it means your terminal won't be able to find Python afterward.

macOS: run the installer package and follow the prompts. macOS ships with an old, internal copy of Python 2 for its own use; ignore it completely. This course, and every command in it, uses python3.

Linux: most distributions already include Python 3. Open a terminal and check the version (instructions below); if it's missing, install it through your package manager, for example `sudo apt install python3` on Ubuntu or Debian.

Verifying the Installation

Open your terminal:

- **Windows:** press the Windows key, type `cmd`, press Enter (or use “Windows Terminal”).
- **macOS:** press `Cmd + Space`, type `Terminal`, press Enter.
- **Linux:** press `Ctrl + Alt + T`, or open “Terminal” from your applications menu.

Type this and press Enter:

```
1 python3 --version
```

On Windows, if that doesn't work, try:

```
1 python --version
```

You should see something like `Python 3.13.0`. If you see an error instead, revisit the installation step and make sure “Add to PATH” was checked (Windows), or try restarting your terminal or computer; PATH changes often need a fresh terminal window to take effect.

Try It Yourself 1.2a: Open your terminal. Use `pwd/cd` to see where you currently are, then run `mkdir python_practice` to create a folder, then `cd python_practice` to move into it. Run `pwd` again to confirm you're inside it. You'll work inside this folder for the rest of this phase.

What Is the Terminal?

The terminal is a text-based way to control your computer: instead of clicking icons, you type commands, and the computer responds with text, printed directly below whatever you typed. It looks intimidating at first, mostly because it offers no buttons or menus to explore, only a blinking cursor waiting for input, but this entire course only needs a handful of commands, and every one of them does something you already understand conceptually from using a mouse: moving between folders, listing what's inside one, creating a new one.

Command	What it does	Example
<code>pwd</code> (or <code>cd</code> alone on Windows)	Print the folder you're currently in	<code>pwd</code>
<code>ls</code> (Windows: <code>dir</code>)	List files in the current folder	<code>ls</code>
<code>cd foldername</code>	Move into a folder	<code>cd projects</code>
<code>cd ..</code>	Move up one folder level	<code>cd ..</code>
<code>mkdir name</code>	Create a new folder	<code>mkdir python_practice</code>
<code>python3 file.py</code>	Run a Python file	<code>python3 hello.py</code>

Installing a Code Editor: VS Code

You can write Python in Notepad, but a real code editor makes learning dramatically easier: it colors your code, catches typos, and lets you run programs with one click.

1. Go to code.visualstudio.com and download Visual Studio Code for your operating system.
2. Install it like any normal application.
3. Open VS Code, click the Extensions icon in the left sidebar (it looks like four squares), search for "Python" (the official one, published by Microsoft), and click Install.

Your First File

1. In VS Code, go to File → Open Folder, and select the `python_practice` folder you created earlier.
2. Click File → New File, and name it `hello.py`. The `.py` extension tells every tool involved "this is a Python file."
3. Type the following exactly as shown (remember: type it, don't paste it):

```
1 print("Hello, world!")
```

4. Save the file (Ctrl+S / Cmd+S).
5. Open a terminal inside VS Code: Terminal → New Terminal. This terminal is already pointed at your project folder, which saves you the `cd` step.
6. Type:

```
1 python3 hello.py
```

You should see:

```
1 Hello, world!
```

Congratulations; you just ran your first program. Every programmer's journey starts with exactly this line. Walk through precisely what just happened: typing `python3 hello.py` told your terminal “start the Python interpreter, and hand it this specific file to run”; the interpreter opened `hello.py`, read the single line inside it, and executed `print("Hello, world!")`, which is a function call telling Python to display whatever's inside the parentheses on the screen. Nothing about this is different, in kind, from every program you'll write for the rest of this course; it's simply the shortest possible one. If you got an error instead, check: is the file saved? Are you in the right folder in the terminal? Did you type `print` correctly, all lowercase?

Try It Yourself 1.2b: Change the text inside the quotation marks to a short message of your own, save the file, and run it again. Then add a second `print(...)` line right below the first one with a different message, save, and run again; you should see both lines print, in order, top to bottom.

Common Mistakes to Avoid

- **Forgetting quotation marks.** `print(Hello, world!)` fails, because Python thinks `Hello` is a variable name, not text. Text always needs quotes: `print("Hello, world!")`.
- **Mismatched parentheses.** `print("Hello, world!"` (missing the closing `)`) produces an error pointing at the next line, which confuses beginners. Always check you have as many `)` as `(`.
- **Running the wrong file.** `python3 hello.py` only works if your terminal is currently inside the folder containing `hello.py`. Use `ls` to check what's actually in your current folder.
- **Using `python` vs `python3`.** On some systems, `python` points to an old Python 2 installation, or doesn't exist at all. If `python3` works, stick with it throughout this course.
- **Not saving before running.** VS Code shows a small dot on the file tab when there are unsaved changes; running the file before saving means you're running the old version of your code.

Lesson 1.2 Project: Personal Greeting Script

Create a new file called `greeting.py`. Write a program that prints four lines: your name, your city, your favorite food, and one goal you have for learning Python. Each line should use its own `print()` call. Run it from the terminal.

This project only uses what you've learned so far (`print`), but it proves you can create a file, write instructions in it, and execute them: the complete, repeatable loop you'll use for every program in this course.

Lesson Summary: Quick Reference

- `python3 --version` checks your installation
- `cd`, `ls/dir`, `mkdir` are the basic terminal navigation commands
- `.py` is the Python file extension
- `python3 filename.py` runs a script from the terminal

Lesson 1.3: Variables & Data Types

Why This Matters

Every program you'll ever write is built out of two things: data, and instructions that act on that data. Variables are how you hold onto data so you can use it more than once, name it something meaningful, and change it over time. Nothing in this course works without them.

Learning Objectives

By the end of this lesson, you will be able to:

- Create variables and explain what assignment actually does
- Identify and use Python's four core data types
- Check a value's type with `type()` and convert between types
- Get text input from a user and understand what type it arrives as

What Is a Variable?

A variable is a name that points to a value stored in memory. Creating one looks like this:

```
1 age = 25
2 print(age)
```

Read `age = 25` as “create a name called `age`, and point it at the value 25”, not as a mathematical equation. This matters because it explains why reassignment works the way it does:

```
1 age = 25
2 print(age)
3 age = 26
4 print(age)
```

The name `age` now points to 26 instead. Nothing about the number 25 changed anywhere; `age` simply points somewhere new. A useful mental picture: think of `age` as a sticky note, not a box. A box would hold the number 25 physically inside it, and changing its contents would mean reaching in and swapping what's there. A sticky note, by contrast, just has a name written on it and gets stuck onto whatever value it currently points to; `age = 26` peels the note off of 25 and sticks it onto 26 instead. The number 25 itself was never modified, since numbers can't be modified at all, only pointed to or not.

Variable names in Python must start with a letter or underscore, can contain letters, digits, and underscores, and are case-sensitive (`age` and `Age` are different names). By convention, Python variable names use `snake_case`: lowercase words separated by underscores, like `first_name` rather than `firstName` or `FirstName`. You'll see this convention followed consistently for the rest of the course.

The Core Data Types

Every value in Python has a type, which determines what you can do with it. Four types cover the vast majority of everyday code, and each is worth understanding on its own.

Integers (`int`). An `int` is a whole number, positive or negative, with no decimal point:

```
1 age = 25
```

```
2 temperature = -10
3 total_students = 0
```

Use an `int` whenever a quantity is naturally a whole count: an age, a number of items, a year. Python's integers have no fixed size limit the way some other languages' do; you can work with arbitrarily large whole numbers without special handling.

Floating-Point Numbers (`float`). A `float` is a number with a decimal point, used for anything that isn't naturally a whole quantity:

```
1 price = 19.99
2 temperature = 98.6
3 gpa = 3.75
```

Use a `float` for measurements, prices, averages, and anything else where fractional values make sense. A number written with a decimal point, even `5.0`, is a `float`; the same number without one, `5`, is an `int`. Python treats these as genuinely different types, even though they represent a mathematically equal value.

Strings (`str`). A `str` ("string") holds text, always wrapped in either single or double quotes:

```
1 name = "Ada"
2 city = 'London'
3 message = "Hello, world!"
```

Python treats `"Ada"` and `'Ada'` identically; the choice between double and single quotes is purely a style preference, though staying consistent within a single project is good practice. Use double quotes throughout this course unless a specific example says otherwise.

Booleans (`bool`). A `bool` holds exactly one of two values, `True` or `False`, capitalized exactly like that:

```
1 is_student = True
2 has_paid = False
3 is_empty = True
```

Use a `bool` for any yes/no, on/off style fact: whether a user is logged in, whether a form was submitted, whether a list is empty. You'll use booleans constantly starting in Lesson 1.5, once `if` statements let you actually act on them.

Confirming a Value's Type

Use the built-in `type()` function any time you want to confirm what type a value actually is:

```
1 age = 25
2 print(type(age))           # <class 'int'>
3
4 price = 19.99
5 print(type(price))         # <class 'float'>
6
7 name = "Ada"
8 print(type(name))          # <class 'str'>
9
10 is_student = True
11 print(type(is_student))    # <class 'bool'>
```

`type()` takes any single value and hands back its type, wrapped in that `<class '... '>` notation, which you'll see constantly and can simply read as “the type is ...”; `type()` itself is a function, called exactly the way `print()` is, just handing back a value instead of displaying one. This is genuinely useful, not just a classroom exercise: once your programs start combining values from different places, a variable typed in by a user, a number pulled from a file, the result of a calculation, it's easy to lose track of exactly what type something actually is, and `type()` is how you check, directly, rather than guessing.

Try It Yourself 1.3a: Create four variables of your own, one of each type, with names that describe something about yourself (your age, a price of something you bought recently, your name, and a true/false fact about yourself). Print each one's value, then print each one's type using `type()`.

Checking and Converting Types

Python won't stop you from trying to combine types that don't mix. This fails:

```
1 age = 25
2 message = "I am " + age + " years old"
3 print(message)
```

That raises a `TypeError`, because `+` between a string and an `int` is ambiguous to Python; it doesn't guess that you meant to convert `age` to text first. You have to do that conversion explicitly, using `str()`:

```
1 age = 25
2 message = "I am " + str(age) + " years old"
3 print(message)
4 # I am 25 years old
```

The reverse direction works the same way. `int()` converts text (or a float) into a whole number, and `float()` converts text (or an `int`) into a decimal number:

```
1 age_text = "25"
2 age_number = int(age_text)
3 print(age_number + 5) # 30
4
5 price_text = "19.99"
6 price_number = float(price_text)
7 print(price_number * 2) # 39.98
```

Conversion isn't guaranteed to succeed; `int("hello")` raises a `ValueError`, because there's no sensible whole number hiding inside that text. You'll learn how to handle that safely with error handling in Lesson 1.14; for now, just be aware that conversion can fail if the text doesn't actually look like a number.

`int()` also works directly on a float, not just on text, but it's worth knowing precisely what it does: it truncates, chopping off everything after the decimal point, rather than rounding to the nearest whole number.

```
1 print(int(3.9)) # 3 - NOT 4; the decimal part is simply dropped
2 print(int(3.1)) # 3
3 print(int(-3.9)) # -3 - truncates toward zero, not downward
```

If you actually want the mathematically nearest whole number, use the built-in `round()` function instead, which follows ordinary rounding rules rather than just discarding the decimal part:

```
1 print(round(3.9)) # 4
2 print(round(3.1)) # 3
3 print(round(3.5)) # 4
```

`int()` and `round()` are easy to reach for interchangeably by mistake; they only agree when the decimal part is exactly `.0`.

Every one of the four core types can also be converted to `bool` with `bool()`, which previews the truthy/falsy rules you'll meet in full in Lesson 1.5: `0`, `0.0`, and `""` all convert to `False`, and every other value of that type converts to `True`.

```
1 print(bool(0))      # False
2 print(bool(5))      # True
3 print(bool(""))     # False
4 print(bool("no"))   # True - any non-empty string, even the word "no", is True
```

That last line is worth pausing on, since it trips people up: `bool("no")` is `True`, not `False`, because `bool()` on a string only cares whether the string is empty or not, not what the text actually says.

Finally, `str()` isn't limited to numbers; it works on essentially any value, including types you haven't met yet, and is the standard way to turn any value into its text representation for printing or concatenation:

```
1 print(str(True))    # "True"
2 print(str(None))    # "None"
```

Getting Input From a User

The built-in `input()` function pauses your program, waits for the user to type something and press Enter, and hands back whatever they typed:

```
1 name = input("What is your name? ")
2 print("Hello, " + name + "!")
```

Here is the single most important fact about `input()` in this entire lesson: it always returns a string, no matter what the user types, even if they type digits.

```
1 age_text = input("How old are you? ")
2 print(type(age_text)) # <class 'str'> -- always, even if they typed "25"
3
4 age_number = int(age_text)
5 print(age_number + 1) # works, now that it's actually an int
```

If you plan to do arithmetic on something the user typed, you must convert it first with `int()` or `float()`. Forgetting this is one of the single most common beginner errors in all of Python, precisely because the mistake doesn't announce itself until you try to do math with it.

Try It Yourself 1.3b: Write a program that asks the user for their birth year using `input()`, converts it to an `int`, and prints their approximate age by subtracting it from 2026.

Constants

Python has no true constant (a variable that's actually locked and can't change), but by convention, variables meant to never change are named in ALL_CAPS:

```
1 PI = 3.14159
2 MAX_USERS = 100
3 COMPANY_NAME = "Acme Corp"
```

Nothing technically stops you from reassigning `PI` later, but writing it in all caps is a signal, to yourself and to anyone reading your code, that this value is meant to stay fixed. You'll see this convention throughout the rest of the course.

Common Mistakes to Avoid

- **Forgetting `input()` always returns a string.** `age = input("Age: ")` followed by `age + 1` raises a `TypeError`, even if the user typed a number, because `age` is still text until you convert it with `int()`.
- **Confusing `=` with `==`.** A single `=` assigns a value; a double `==` compares two values (covered fully in Lesson 1.4). `if age = 18:` is a syntax error in Python, which is actually a helpful safety net compared to some other languages.
- **Using invalid variable names.** `2nd_place = "silver"` fails because names can't start with a digit. `my-var = 5` also fails, because hyphens aren't allowed in names; Python would try to read that as subtraction.
- **Trying to convert text that isn't actually a number.** `int("twenty-five")` raises `ValueError`, because Python has no way to know that means 25. Only text that looks like a plain number, such as `"25"`, converts cleanly.
- **Mixing up `str()` concatenation with commas in `print()`.** `print("Age:", age)` works fine with mixed types, since `print` handles the conversion for you when you separate arguments with commas; it's specifically the `+` operator between a string and a non-string that fails.

Lesson 1.3 Project: Simple Profile Card

Write a program called `profile.py` that:

1. Asks the user, using `input()`, for their name, age (as a whole number), and height in meters (as a decimal).
2. Converts age and height to the correct types (`int` and `float`).
3. Prints a neatly formatted "profile card" showing all three pieces of information along with their types, using `type()` to prove the conversions actually worked.

This project forces you to combine `input()`, all four core types, and type conversion, everything from this lesson, into one small program.

Lesson Summary: Quick Reference

- `x = 5` creates a variable; the four core types are `int`, `float`, `str`, `bool`
- `type(x)` checks a value's type
- `int()`, `float()`, `str()` convert between types
- `input()` always returns a `str`, even when the user types a number

Lesson 1.4: Operators

Why This Matters

Variables give you somewhere to put data. Operators are how you actually do something with it: add two prices, check if someone is old enough, decide whether two conditions are both true. Every meaningful line of logic you'll write in this course leans on the operators in this lesson.

Learning Objectives

By the end of this lesson, you will be able to:

- Use all seven arithmetic operators correctly, including the two beginners consistently mix up
- Use comparison operators to produce a `bool`
- Combine conditions with `and`, `or`, and `not`
- Explain the difference between `==` and `is`, and know which one to use

Arithmetic Operators

Python supports the arithmetic you'd expect, plus two operators most other languages don't make this convenient:

```
1 a = 17
2 b = 5
3
4 print(a + b)      # 22 - addition
5 print(a - b)      # 12 - subtraction
6 print(a * b)      # 85 - multiplication
7 print(a / b)      # 3.4 - division, ALWAYS returns a float
8 print(a // b)     # 3 - floor division, rounds down to a whole number
9 print(a % b)      # 2 - modulo, the remainder after division
10 print(a ** b)    # 1419857 - exponent, a to the power of b
```

Two of these deserve extra attention, because they're where most beginners get surprised.

`/` is regular division, and in Python 3 it always returns a float, even when the division is exact: `10 / 2` gives `5.0`, not `5`. `//` is floor division: it divides, then rounds down to the nearest whole number, discarding the fractional part entirely. `17 // 5` is `3`, not `3.4` and not `4`; `17` divided by `5` is `3.4`, and flooring, always rounding toward negative infinity rather than merely chopping off the decimal part, brings it down to `3`. That distinction, floor versus chop, only produces different results once negative numbers are involved: `-7 // 2` is `-4`, not `-3`, since `-7 / 2` is `-3.5`, and the next whole number below `-3.5`, further from zero, is `-4`; simply chopping off the decimal part would have given `-3` instead, which is why it's worth being precise that floor division rounds down in the strict sense, toward negative infinity, not toward zero.

`%` (modulo) gives you the remainder left over after floor division. It's one of the most quietly useful operators in the language: `number % 2` tells you whether `number` is even (`0`) or odd (`1`); `minutes % 60` tells you the leftover minutes after converting to hours. You'll reach for it constantly once you notice how often "is this a multiple of" or "what's left over" comes up.

Try It Yourself 1.4a: Predict, then verify in your editor: what does `7 // 2` evaluate to? What about `7 % 2`? What about `-7 // 2`? Run all three and compare to your predictions.

Augmented Assignment

A very common pattern is updating a variable based on its own current value:

```
1 score = 10
2 score = score + 5
3 print(score) # 15
```

Python offers a shorthand for this, for every arithmetic operator:

```
1 score = 10
2 score += 5    # same as score = score + 5
3 print(score) # 15
4
5 score -= 3    # same as score = score - 3
6 score *= 2    # same as score = score * 2
7 score /= 4    # same as score = score // 4
8 print(score)
```

`+=`, `-=`, `*=`, `/=`, `//=`, `%=`, and `**=` all follow the same pattern: “apply this operator to the variable’s current value, using the value on the right, and store the result back.” You’ll see `+=` constantly from here on, especially inside loops starting in Lesson 1.6.

Comparison Operators

Comparison operators compare two values and always produce a `bool`:

```
1 a = 10
2 b = 20
3
4 print(a == b)    # False - equal to
5 print(a != b)    # True - not equal to
6 print(a < b)     # True - less than
7 print(a > b)     # False - greater than
8 print(a <= 10)   # True - less than or equal to
9 print(a >= b)    # False - greater than or equal to
```

The single most common beginner mistake in this entire course happens right here: `=` assigns, `==` compares. `score = 10` sets `score` to 10. `score == 10` asks “is `score` equal to 10 right now?” and changes nothing. `if score = 10:` is invalid Python and won’t even run; `if score == 10:` is a valid question you can act on, starting in the very next lesson.

Strings compare alphabetically, character by character, using each character’s underlying code, which is why it’s case-sensitive (uppercase letters sort before lowercase ones):

```
1 print("apple" == "Apple")    # False - different case
2 print("apple" < "banana")    # True - "a" comes before "b"
3 print("app" < "apple")       # True - shorter prefix wins
```

For a case-insensitive comparison (common with things like emails or usernames), lowercase both sides first, using the `.lower()` string method you’ll meet properly in Lesson 1.7:

```
1 user_input = "Ada@example.com"
2 stored_email = "ada@example.com"
3 print(user_input.lower() == stored_email.lower()) # True
```

Chained Comparisons

Python lets you chain comparisons into a single, readable range check:

```
1 age = 25
2 print(18 <= age <= 65)  # True - "is age between 18 and 65?"
```

This is shorthand for `18 <= age` and `age <= 65`, and it reads almost exactly like the English sentence it represents.

== VS is

`==` compares values: “do these look the same?” `is` compares identity: “are these literally the same object in memory?” This distinction becomes concrete once two separate lists can hold equal values (`==` is `True`) without being the same list (`is` is `False`), which you’ll see directly in Lesson 1.8. For everyday comparisons, use `==`. The one common exception: always check for `None` with `is None`, not `== None`:

```
1 middle_name = None
2
3 if middle_name is None:  # the standard, idiomatic check
4     print("No middle name on file.")
```

Logical Operators

Comparison operators answer one yes-or-no question at a time: is age at least 18? Does a password match? Real decisions are almost never that simple; they usually depend on several yes-or-no answers combined into one final decision, the way “let this user in if they’re 18 or older and they have an ID” combines two separate facts into a single rule. and, or, and not are how you combine them.

and: both sides must be true

```
1 age = 25
2 has_id = True
3
4 print(age >= 18 and has_id)  # True - both sides are True
```

and looks at the value on its left and the value on its right, and produces `True` only when both are `True`. If either side, or both sides, is `False`, the whole expression is `False`. Every possible combination looks like this:

```
1 print(True and True)      # True
2 print(True and False)    # False
3 print(False and True)     # False
4 print(False and False)   # False
```

Only the very first combination, both sides `True`, produces `True`. This is worth internalizing as a simple mental rule: with and, a single `False` anywhere ruins the whole thing.

or: at least one side must be true

```
1 age = 16
2 has_guardian_permission = True
3
4 print(age >= 18 or has_guardian_permission)  # True - at least one side is True
```

or produces True the moment either side is True, and only produces False when both sides are False:

```
1 print(True or True)      # True
2 print(True or False)    # True
3 print(False or True)    # True
4 print(False or False)   # False
```

Notice the difference from everyday English here, where “or” sometimes means “one or the other, but not both” (as in “you can have soup or salad”). Python’s or is inclusive: if both sides happen to be True, the result is still True, not an error or a special case.

not: flips a single value

```
1 is_logged_in = False
2 print(not is_logged_in) # True
3
4 is_valid = True
5 print(not is_valid)    # False
```

not takes exactly one value, not two, and simply flips it: True becomes False, and False becomes True. It’s frequently used to turn an awkward double-negative condition into something readable: `if not is_empty:` reads naturally as “if it isn’t empty,” which is usually clearer than restructuring the whole condition around a positive check.

Combining more than two conditions. and, or, and not combine freely, and with more than two conditions, parentheses control exactly how they group, the same way they do in arithmetic:

```
1 age = 25
2 has_id = True
3 has_passport = False
4
5 print(age >= 18 and (has_id or has_passport))
6 # True - old enough, AND has at least one form of ID
7
8 print((age >= 18 and has_id) or has_passport)
9 # True - a differently grouped question, though it happens to agree here
```

Both lines above happen to produce True with these particular values, but they ask genuinely different questions, and with different values they can disagree. The first reads as “you must be old enough, and on top of that, you need some form of ID.” The second reads as “either you’re old enough and have an ID, or you have a passport regardless of age,” which is a much looser rule. Whenever you combine three or more conditions, add parentheses to make the grouping explicit, even in the cases where Python’s default precedence would have gotten it right anyway; it costs nothing and removes any doubt for the next reader.

Short-circuit evaluation. Python evaluates and and or left to right, and it stops the instant the final answer is already certain, without bothering to evaluate the right-hand side at all. This is called short-circuit evaluation, and it’s not just a performance detail; it’s a pattern you’ll rely on deliberately:

```
1 username = ""
2
3 if username and username[0].isupper():
4     print("Starts with a capital letter.")
5 else:
```

```
6 print("Empty or lowercase.")
```

With `username` equal to `""` (falsy, as you'll see fully in Lesson 1.5), and already knows the overall result must be `False` the moment it sees a falsy left side, so it never even evaluates `username[0]`, the part that would otherwise raise an `IndexError` on an empty string. It's worth seeing what would happen without this protection: if you instead wrote the check backward, as `username[0].isupper()` and `username`, Python would evaluate `username[0]` first, immediately hitting the same `IndexError`, since there's no character at position 0 of an empty string to check in the first place; the order genuinely matters, and putting the safety check first is exactly what makes the pattern work. This "check that something is safe, and then use it" pattern, relying on and to stop before the dangerous part runs, is extremely common in real Python code. or short-circuits the same way, in the opposite direction: the moment it finds a truthy left side, it skips evaluating the right side entirely, since the answer is already known to be `True`.

Try It Yourself 1.4b: Create two boolean variables, `is_raining` and `has_umbrella`. Write and print an expression using `and`, `or`, and `not` together that answers: "I will stay dry" (true if it's not raining, or if it is raining but I have an umbrella). Then write a second expression, with parentheses, for three conditions: a user may check out if they have items in their cart and (they're logged in or they chose guest checkout).

Operator Precedence Across Categories

When operators from different categories appear in the same expression, Python evaluates arithmetic first, then comparisons, then logical operators last:

```
1 result = 5 + 3 > 2 * 3 and 10 % 3 == 1
2 # Step 1 (arithmetic): 5+3=8, 2*3=6, 10%3=1
3 # Step 2 (comparisons): 8 > 6 is True, 1 == 1 is True
4 # Step 3 (logical): True and True
5 print(result) # True
```

When an expression gets this dense, parentheses are almost always worth adding even when they aren't strictly required, purely so the next reader (often you, in six months) doesn't have to reconstruct this evaluation order from memory.

Common Mistakes to Avoid

- **Confusing `=` and `==`.** This is worth repeating from earlier in the lesson because it's genuinely the single most common early mistake in Python. `=` assigns; `==` compares.
- **Expecting / to return an int.** `/` always returns a float in Python 3, even `10 / 2`. Use `//` if you specifically want a whole-number result.
- **Forgetting `//` rounds down, not toward zero.** `-7 // 2` is `-4`, which surprises people expecting `-3`. Floor division always rounds toward negative infinity.
- **Using `==` to compare with `None`.** It usually works, but `is None` is the correct, idiomatic check, and is required in some edge cases involving custom objects you'll meet in Phase 2.
- **Skipping parentheses in dense mixed expressions.** `a + b > c and d` technically has a well-defined evaluation order, but if you have to think about it, so will everyone reading your code. Add parentheses for clarity, not just correctness.

Lesson 1.4 Project: Restaurant Bill Splitter

Write a program called `bill_splitter.py` that:

1. Asks the user, using `input()`, for the total bill amount and the number of people splitting it, converting both to the correct numeric types.
2. Calculates each person's share using `/`, and separately calculates it using `//` and `%` to show the whole-dollar share plus the leftover cents.
3. Asks whether the user wants to add an 18% tip (yes/no), and uses a logical expression to decide whether to include it in the total.
4. Prints a clear summary: the total (with or without tip), the per-person share, and whether everyone's share is at least \$5 (using a comparison operator).

This project forces you to combine arithmetic, comparison, and logical operators in one small, genuinely useful program.

Lesson Summary: Quick Reference

- `+` `-` `*` `/` `//` `%` `**` are the seven arithmetic operators; `/` always returns a float, `//` floors, `%` gives the remainder
- `+=` `-=` `*=` `/=` etc. update a variable based on its own value
- `==` `!=` `<` `>` `<=` `>=` compare values and return a `bool`; `=` assigns, `==` compares
- `and`, `or`, `not` combine boolean values; use `is None`, `not == None`
- Arithmetic evaluates before comparisons, which evaluate before logical operators

Lesson 1.5: Control Flow (if/elif/else)

Why This Matters

Every program you’ve written so far runs top to bottom, every line, every time. Real programs need to make decisions: send this email only if the form was valid, apply a discount only if the cart total is high enough. Control flow is how you teach a program to choose.

Learning Objectives

By the end of this lesson, you will be able to:

- Write `if`, `if/else`, and `if/elif/else` statements
- Explain which Python values count as “truthy” and “falsy”
- Nest conditionals correctly, and know when nesting is the wrong tool
- Write a compact conditional expression using the ternary form

The `if` Statement

An `if` statement runs a block of code only when a condition is `True`, and skips that block entirely when the condition is `False`:

```
1 age = 20
2
3 if age >= 18:
4     print("You can vote.")
5
6 print("This line always runs.")
```

Walk through exactly what Python does here. It evaluates `age >= 18`, which produces a `bool` (this is exactly the comparison operator from Lesson 1.4, doing exactly what it always does). Since `age` is 20, that expression evaluates to `True`, so the indented line underneath, `print("You can vote.")`, runs. Then execution continues to `print("This line always runs.")`, which is not indented under the `if`, so it’s not part of the decision at all; it runs every single time, regardless of what `age` was.

Now compare what happens with a different value:

```
1 age = 15
2
3 if age >= 18:
4     print("You can vote.")
5
6 print("This line always runs.")
```

This time `age >= 18` evaluates to `False`. The indented block is skipped entirely, as if it weren’t there; Python doesn’t run it, doesn’t check it, doesn’t do anything with it. Execution jumps straight to `print("This line always runs.")`, which still runs, because it was never part of the `if` block to begin with. Only “This line always runs.” prints; nothing about voting appears at all.

Two things about the syntax here are non-negotiable, not style choices. The `if` line ends with a colon, `:`. And the code that belongs to the `if`, the code that should only run when the condition is true, is indented, conventionally by four spaces. Recall from Lesson 1.1: indentation isn’t decoration in Python, it’s how the language knows what

belongs to the `if` and what doesn't. Every line indented at that level, directly under the `if`, is part of its block; the moment a line returns to the previous indentation level, Python considers the block over.

An `if` block can also contain several lines, not just one, as long as they're all indented consistently at the same level:

```
1 age = 20
2
3 if age >= 18:
4     print("Access granted.")
5     print("Welcome in.")
6     remaining_years = 100 - age
7     print(f"You have {remaining_years} years until you turn 100.")
8
9 print("Checked access.")
```

All three indented lines belong to the `if`; if `age >= 18` is `True`, all three run, in order, top to bottom, exactly like any other sequence of Python code. If it's `False`, none of the three run, not just the first one.

The condition itself doesn't have to be a comparison. It can be any expression that evaluates to something Python can treat as `True` or `False`, including the truthy/falsy values you'll meet in full later in this lesson, and the `and/or/not` combinations from Lesson 1.4:

```
1 is_member = True
2
3 if is_member:
4     print("Member discount applied.")
```

`is_member` is already a `bool`, so it can be used directly as the condition, with no comparison needed at all; `if is_member:` reads as “if `is_member` is true.”

if / else

Often you don't just want to know when a condition is true; you want to do one thing if it's true, and a different thing if it's false. That's what `else` is for: it provides a fallback block that runs precisely when the `if`'s condition is `False`, with no condition of its own to write:

```
1 age = 15
2
3 if age >= 18:
4     print("You can vote.")
5 else:
6     print("You cannot vote yet.")
```

Trace through both possible outcomes. If `age` is 18 or higher, `age >= 18` is `True`: the `if` block runs (“You can vote.” prints) and the `else` block is skipped completely. If `age` is under 18, `age >= 18` is `False`: the `if` block is skipped, and the `else` block runs instead (“You cannot vote yet.” prints). Exactly one of these two blocks runs, every single time the code executes; never both, and never neither. This is the core guarantee an `if/else` pair gives you that a plain `if` alone doesn't: total coverage, with no gap where nothing happens.

Notice that `else` has no condition and no colon-separated expression of its own; it's simply `else:`, meaning “in every other case.” Trying to write `else age < 18:` is a `SyntaxError`; the whole point of `else` is that it already knows its condition is “whatever made the `if` false,” so repeating that logic would be redundant, and Python doesn't allow it.

else blocks can also hold multiple lines, following the same indentation rule as if:

```
1 temperature = 45
2
3 if temperature >= 60:
4     print("It's warm out.")
5     print("A light jacket should be fine.")
6 else:
7     print("It's cold out.")
8     print("Bring a heavy coat.")
9     print("Maybe a hat too.")
```

Here, since temperature is 45, the condition is False, so all three lines in the else block run, and both lines in the if block are skipped entirely.

Try It Yourself 1.5a-i: Write a program that stores a boolean variable `is_raining`. Using if/else, print “Bring an umbrella.” if it’s raining, and “Enjoy the sunshine.” if it isn’t. Run it once with each value to prove both branches actually work.

if / elif / else

An if/else pair only ever gives you two possible paths. Real decisions often have more than two: a letter grade has five tiers, a shipping cost might depend on three different weight ranges. `elif` (short for “else if”) lets you check additional conditions, one after another, stopping at the very first one that’s True:

```
1 score = 75
2
3 if score >= 90:
4     print("Grade: A")
5 elif score >= 80:
6     print("Grade: B")
7 elif score >= 70:
8     print("Grade: C")
9 else:
10    print("Grade: F")
```

Python checks each condition top to bottom, in the exact order they’re written, and runs only the first block whose condition is True, then skips every remaining condition entirely, without even evaluating them, even if a later one would also have been True. Trace it precisely with `score = 75`: `score >= 90` is False, so Python moves to the next check; `score >= 80` is also False, so Python moves on again; `score >= 70` is True, so “Grade: C” prints, and Python stops there completely, never even looking at the else. The else at the end is the final fallback: it only runs if every single condition above it, the if and every elif, was False.

Order genuinely matters here, and getting it backward is a real, common bug. Watch what happens if the conditions are written from lowest to highest instead:

```
1 score = 95
2
3 if score >= 70:
4     print("Grade: C")
5 elif score >= 80:
6     print("Grade: B")
7 elif score >= 90:
8     print("Grade: A")
```

With `score = 95`, you'd expect "Grade: A". But `score >= 70` is checked first, and 95 certainly satisfies it, so "Grade: C" prints, and the two conditions that would have caught the A and B cases are never even reached. Whenever your `elif` conditions overlap the way range checks like this do, order them from most specific (or most restrictive) to least, exactly as in the first, correct version.

You can chain as many `elif` blocks as you need, and the `else` at the end is optional; an `if/elif/elif` chain with no `else` is completely valid, it simply means "if none of these match, do nothing at all":

```
1 day = "Saturday"
2
3 if day == "Saturday":
4     print("Weekend!")
5 elif day == "Sunday":
6     print("Weekend!")
```

If `day` were "Tuesday" here, neither condition would match, there's no `else` to fall back to, and the program would simply print nothing and move on to whatever code comes after, without error.

Try It Yourself 1.5a-ii: Write a program that asks the user for a temperature in Fahrenheit using `input()` (remember to convert it), then prints "Freezing" if it's at or below 32, "Cold" if it's between 33 and 60, "Mild" if it's between 61 and 80, and "Hot" above that, using `if/elif/else`. Then deliberately reorder your conditions incorrectly, run it again with a hot temperature, and confirm for yourself that you get the wrong answer, the same way the grade example above did.

Nested Conditionals

An `if` can contain another `if` inside it, indented one level further, for decisions that only make sense after an earlier condition is already satisfied:

```
1 age = 20
2 has_ticket = True
3
4 if age >= 18:
5     if has_ticket:
6         print("Welcome to the show.")
7     else:
8         print("You need a ticket.")
9 else:
10    print("This show is 18+.")
```

Nesting is the right tool when the inner decision genuinely only applies inside the outer one, exactly as shown here: whether you need a ticket is irrelevant if you're not old enough to be there at all. But nesting more than two or three levels deep quickly becomes hard to read; often, combining conditions with `and` (from Lesson 1.4) expresses the same logic more clearly:

```
1 if age >= 18 and has_ticket:
2     print("Welcome to the show.")
3 elif age >= 18 and not has_ticket:
4     print("You need a ticket.")
5 else:
6     print("This show is 18+.")
```

Neither version is objectively wrong; prefer whichever reads more clearly for the specific logic you're expressing, and default to flattening with and/or once nesting passes two levels.

Truthy and Falsy Values

Any value can be used where a `bool` is expected, not just actual `True/False`; Python converts it using rules that are worth memorizing, because they come up constantly:

```
1 name = ""
2 if name:
3     print("Name provided")
4 else:
5     print("Name is empty") # this runs - "" is falsy
```

The falsy values, the complete list, are: `False`, `None`, `0`, `0.0`, and any empty collection, empty string `""`, empty list `[]`, empty dictionary `{}`. Every other value, including any non-empty string (even `"False"` as text) and any non-zero number, is truthy. Notice the pattern underneath that list, rather than trying to memorize it as an arbitrary set of exceptions: every falsy value represents “nothing” or “zero amount” in some natural sense, an empty collection has nothing in it, `0` is zero of something, `None` represents the deliberate absence of a value entirely, while everything else, having some content or some nonzero value, counts as truthy. This is why `if name:` is common, idiomatic Python for “if name is not empty,” instead of the more verbose `if name != ""`; both ask the exact same question, but the first is how experienced Python developers actually write it, and you'll see it constantly in real code from here on.

```
1 items = []
2 if items:
3     print(f"You have {len(items)} items.")
4 else:
5     print("Your cart is empty.") # this runs - [] is falsy
```

The Ternary (Conditional) Expression

For a simple choice between two values, Python offers a compact, single-line form:

```
1 age = 20
2 status = "adult" if age >= 18 else "minor"
3 print(status) # adult
```

Read this as: “status is “adult” if age >= 18, otherwise it's “minor”.” It's exactly equivalent to a full `if/else` that assigns to the same variable in both branches:

```
1 if age >= 18:
2     status = "adult"
3 else:
4     status = "minor"
```

Use the ternary form when both branches are short and you're assigning a single value; reach for a full `if/else` the moment either branch needs more than one line, since forcing a ternary to do too much makes it harder to read, not easier.

Try It Yourself 1.5b: Using a ternary expression, create a variable message that holds "Weekend!" if a variable day equals "Saturday" or "Sunday", and "Weekday" otherwise. (Hint: you'll need `or` inside the condition.)

Common Mistakes to Avoid

- **Forgetting the colon.** `if age >= 18` without the trailing `:` is a `SyntaxError`. Every `if`, `elif`, and `else` line ends with one.
- **Inconsistent indentation.** Mixing tabs and spaces, or indenting two lines in the same block by different amounts, raises an `IndentationError`. Configure your editor to insert spaces when you press Tab, and stay consistent.
- **Using separate `if` statements instead of `elif`, then being surprised multiple blocks run.** Three separate `if` statements each get checked independently; an `if/elif` chain stops at the first match. These are not interchangeable.
- **Testing truthiness with `== True` or `== False`.** `if is_valid == True:` works, but `if is_valid:` is the idiomatic form; the comparison is redundant since `is_valid` is already a `bool`.
- **Over-nesting instead of combining conditions.** Four levels of nested `if` statements are almost always more clearly expressed with `and/or`, as shown earlier in this lesson.

Lesson 1.5 Project: Movie Ticket Pricing

Write a program called `ticket_pricing.py` that:

1. Asks the user for their age and whether it's a weekday showing (yes/no), using `input()`.
2. Determines the ticket price using `if/elif/else`: \$8 for ages under 13, \$15 for ages 13 to 64, and \$10 for ages 65 and up.
3. Applies a further \$2 discount if it's a weekday showing, using a nested conditional or a combined `and` expression, your choice.
4. Uses a ternary expression to build a one-line message: "Discount applied!" if a discount was given, otherwise "Full price."
5. Prints the final price and the message.

Lesson Summary: Quick Reference

- `if condition:` runs a block only when the condition is `True`; every `if/elif/else` line ends with `:`
- An `elif` chain stops at the first `True` condition and skips the rest
- Falsy values: `False`, `None`, `0`, `0.0`, and empty collections; everything else is `truthy`
- `value_if_true if condition else value_if_false` is the ternary expression
- Prefer `and/or` over deep nesting once you're past two levels

Lesson 1.6: Loops (for and while)

Why This Matters

Every program you've written so far does each thing exactly once. Real programs repeat: process every row in a spreadsheet, keep asking for input until it's valid, check every item in a cart. Loops are how you tell Python to repeat something, and they are one of the small handful of tools this entire course, and nearly everything after it, is built on top of.

Learning Objectives

By the end of this lesson, you will be able to:

- Write a for loop to process every item in a collection, including a string's characters
- Use range() to repeat something a specific number of times
- Write a while loop that repeats until a condition changes, and avoid writing one that never stops
- Use break and continue to control a loop from the inside
- Write a nested loop, and explain exactly how many times its inner block runs

The for Loop

A for loop processes every item in a collection, one at a time, running the same block of code once per item, without you writing a separate line for each one:

```
1 fruits = ["apple", "banana", "cherry"]
2
3 for fruit in fruits:
4     print(fruit)
```

Read this exactly as it sounds: “for each fruit in fruits, do the following.” Python takes the list one item at a time, in order, temporarily names each one fruit (a name you chose; it could be anything), runs the indented block once per item, using that item's value, and then moves to the next one. With three fruits in the list, print(fruit) runs three times, printing "apple", then "banana", then "cherry", each on its own line. Exactly like the if blocks from Lesson 1.5, indentation is what tells Python which lines belong to the loop; the moment a line returns to the previous indentation level, the loop body is over.

A for loop can hold multiple lines, and can use the current item however you need to:

```
1 scores = [85, 92, 78]
2 total = 0
3
4 for score in scores:
5     total = total + score
6     print(f"Running total: {total}")
7
8 print(f"Final total: {total}")
```

This pattern, starting a variable at 0 before the loop and adding to it on every pass, is called accumulation, and it's one of the most common shapes in all of programming: totals, counts, and combined results are almost always built this way, one item at a time. Trace through exactly what total holds at each step, since watching the accumulator change is the key to trusting this pattern: before the loop starts, total is 0. On the first pass, score is 85, and total

= total + score sets total to 0 + 85, or 85. On the second pass, score is 92, and total becomes 85 + 92, or 177. On the third, total becomes 177 + 78, or 255, the final total. total has to exist before the loop starts, initialized to 0; there's nothing to add to on the very first pass otherwise, and Python would raise a `NameError` trying to read a total that was never created.

Try It Yourself 1.6a: Given words = ["the", "quick", "brown", "fox"], write a for loop that builds a single running string, separated by spaces, by accumulating into a variable that starts as "". Print the result, and compare it (just by reading, no need to prove it yet) to what " ".join(words) would give you.

range(): Looping a Specific Number of Times

Often you don't have an existing collection to loop over; you just want to repeat something a set number of times, or loop using a position rather than an actual value. range() generates a sequence of numbers on demand, built exactly for this:

```
1 for i in range(5):
2     print(i)
3 # 0, 1, 2, 3, 4 - starts at 0, stops BEFORE 5, five numbers total
```

range(5) produces five numbers, 0 through 4, stopping before 5; the count of numbers you get always equals the number you passed in. range() also accepts a start and a step, following the same “stop is excluded” rule either way:

```
1 for i in range(2, 10, 3):
2     print(i)
3 # 2, 5, 8 - start at 2, stop before 10, step of 3
4
5 for i in range(10, 0, -2):
6     print(i)
7 # 10, 8, 6, 4, 2 - counting downward with a negative step
```

With one argument, range(stop) counts from 0. With two, range(start, stop) counts from start instead. With three, range(start, stop, step) controls the size, and direction, of each jump; a negative step counts downward, exactly the way a negative step did in the slicing you'll meet properly in the next lesson.

The while Loop

A while loop repeats based on a condition, rather than a fixed collection, continuing for as long as that condition stays True:

```
1 count = 0
2 while count < 5:
3     print(count)
4     count += 1 # without this line, the loop would never end
```

Python checks the condition, count < 5, before every single pass through the loop; the moment it's False, the loop stops, and execution continues with whatever comes after it. The count += 1 line is doing genuinely essential work here, not just incrementing a number for its own sake: without something inside the loop that eventually makes the condition False, you get an infinite loop, one that never stops on its own and has to be killed manually (in most editors and terminals, Ctrl+C does this). This is the single most common bug beginners write with while, and it's worth developing a habit of asking, every time you write one: “what, specifically, inside this loop, is going to eventually make the condition false?”

while loops are also the natural choice when you don't know in advance how many times you'll need to repeat, which a for loop over a fixed collection can't express at all:

```
1 password = ""
2 while password != "secret":
3     password = input("Enter the password: ")
4
5 print("Access granted.")
```

There's no way to know ahead of time how many attempts a user will need; the loop simply keeps asking until the condition (a correct password) is finally met. Use a for loop whenever you're processing a known collection or a fixed number of repetitions, which covers most cases; reach for while specifically when you're repeating until some condition changes and you genuinely can't know in advance how many passes that will take.

Try It Yourself 1.6b: Write a while loop that starts a variable at 100 and keeps dividing it by 2 (printing each result) for as long as it's greater than 1. Count how many passes it takes, using a second variable that starts at 0 and increments once per loop.

Controlling a Loop From Inside: break and continue

Two keywords let you change a loop's behavior from inside its own body, rather than only through the loop's condition. break exits the loop immediately, completely, skipping any remaining items or passes entirely:

```
1 numbers = [3, 7, 2, 9, 4, 1]
2
3 for n in numbers:
4     if n > 8:
5         print(f"Found one over 8: {n}")
6         break
7     print(f"Checking {n}...")
```

The moment `n` is 9, break stops the loop entirely; 4 and 1 are never even looked at, because there was no more reason to keep going once the goal was found. This is a genuinely common, idiomatic pattern: searching through a collection and stopping the instant you find what you needed, instead of wastefully continuing to check everything else.

continue skips just the rest of the current pass and moves straight to the next item, without exiting the loop entirely:

```
1 numbers = [3, -7, 2, -9, 4, -1]
2
3 for n in numbers:
4     if n < 0:
5         continue # skip negative numbers entirely
6     print(f"Processing {n}")
```

Every negative number is skipped, silently, and the loop moves on to the next item; only 3, 2, and 4 ever reach the print() line. continue is the right tool when you want to skip specific items without abandoning the rest of the loop, which break would do instead.

Try It Yourself 1.6c: Given `numbers = [4, 15, 8, 23, 42, 16, 4]`, write a loop that uses continue to skip any number less than 10, and break the moment it finds a number greater than 40, printing every number it actually processes along the way.

Nested Loops

A loop can contain another loop inside it, indented one level further, exactly the way an `if` could contain another `if` in Lesson 1.5. This is how you process something two-dimensional: a grid, a table, every pair of items from two different lists.

```
1 for row in range(3):
2     for col in range(3):
3         print(f"({row}, {col})", end=" ")
4     print() # move to a new line after each row finishes
```

Trace through this carefully, because nested loops are where beginners most often lose track of what's happening. For every single pass of the outer loop (row equal to 0), the entire inner loop runs to completion, all three passes of `col`, before the outer loop even advances to `row = 1`. The output is nine coordinate pairs total, three rows of three, because the inner loop's three passes happen once for each of the outer loop's three passes: $3 \times 3 = 9$. This multiplying-out effect is the defining feature of nested loops, and it's exactly what you want when the problem is genuinely two-dimensional, like comparing every item in one list against every item in another:

```
1 team_a = ["Ada", "Alan"]
2 team_b = ["Grace", "Linus"]
3
4 for player_a in team_a:
5     for player_b in team_b:
6         print(f"{player_a} vs {player_b}")
7 # Ada vs Grace
8 # Ada vs Linus
9 # Alan vs Grace
10 # Alan vs Linus
```

Every player on `team_a` gets paired with every player on `team_b`, four matchups total from two teams of two, again because the inner loop fully completes for each single pass of the outer one. `break` and `continue` inside a nested loop only affect the innermost loop they're written in, not any loop surrounding it; breaking out of the inner `col` loop above would still leave the outer `row` loop running normally, moving on to its next value exactly as before.

Looping Over a String

Recall from Lesson 1.5 (and you'll see it formalized in the very next lesson) that a string is a sequence, just like a list. That means a `for` loop works on a string directly, character by character, with no extra step required:

```
1 word = "Python"
2
3 for letter in word:
4     print(letter)
5 # P
6 # y
7 # t
8 # h
9 # o
10 # n
```

This is exactly the same loop shape you've used all lesson, just walking through individual characters instead of list items. It's a genuinely common pattern: counting how many times a specific character appears, building a new string one character at a time, or checking every character against some rule:

```
1 text = "Hello, World! 123"
2 digit_count = 0
3
4 for char in text:
5     if char.isdigit():
6         digit_count += 1
7
8 print(f"Found {digit_count} digits.") # Found 3 digits.
```

`.isdigit()` is a string method that returns `True` if a single character is a digit; combined with accumulation, this counts every digit in the text in a few lines, one character at a time.

Common Mistakes to Avoid

- **Writing a `while` loop with nothing that changes the condition.** This produces an infinite loop. Always be able to point at the exact line that will eventually make the condition `False`.
- **Off-by-one errors with `range()`.** `range(5)` gives you 0 through 4, five numbers, not 0 through 5. If you need to count to a number inclusively, you generally want `range(1, n + 1)`.
- **Modifying a list while looping over it directly.** Adding or removing items from a list during a `for` loop over that same list produces confusing, unreliable behavior, since the loop's position and the list's contents are shifting under each other at the same time. Loop over a copy (`for item in my_list.copy():`) if you need to modify the original inside the loop.
- **Confusing `break` and `continue`.** `break` exits the whole loop; `continue` only skips the current pass and moves to the next item. Mixing them up either stops too early or doesn't skip when you meant it to.
- **Using a `for` loop with a manually managed counter, out of habit from other languages.** `for i in range(len(my_list)): print(my_list[i])` works, but `for item in my_list:` is simpler and more idiomatic whenever you don't actually need the index itself.
- **Expecting `break` to exit both loops in a nested pair.** `break` only exits the innermost loop it's written in; the outer loop keeps running normally. Exiting both deliberately usually needs a flag variable, checked in the outer loop too.

Lesson 1.6 Project: Number Guessing Game

Write a program called `guessing_game.py` that:

1. Stores a secret number of your choosing.
2. Uses a `while` loop that keeps asking the user to guess, using `input()`, until they get it right.
3. Inside the loop, prints "Too high" or "Too low" for each incorrect guess, using `if/elif`.
4. Counts the number of guesses using an accumulator variable, and uses `break` once the correct guess is made, printing the final guess count.
5. Separately, using a `for` loop and `range()`, prints a simple countdown from 5 to 1, followed by "Go!".
6. Finally, using a nested loop, prints a small multiplication table for numbers 1 through 5 (a grid of `row * col` for every combination), with each row on its own line.

Lesson Summary: Quick Reference

- `for item in collection`: processes every item in a known collection, one at a time, including a string's characters
- `range(start, stop, step)` generates numbers, stopping before `stop`, for a fixed number of repetitions
- `while condition`: repeats until the condition is false; make sure something inside it actually changes the condition
- `break` exits a loop entirely; `continue` skips just the current pass
- A nested loop's inner block runs completely for every single pass of the outer loop

Lesson 1.7: String Manipulation

Why This Matters

You've been printing text since Lesson 1.2, but real programs need to build, inspect, and transform text: pulling a username out of an email, formatting a price, checking whether input matches an expected pattern. This lesson turns strings from something you print into something you actually work with.

Learning Objectives

By the end of this lesson, you will be able to:

- Explain why strings are sequences, and why they're immutable
- Build formatted text using f-strings
- Use the string methods you'll reach for constantly: case, whitespace, search, and split/join
- Extract parts of a string using slicing

Strings Are Sequences

A string is a sequence of individual characters, and like any sequence in Python, each character has a position, or index, starting at 0:

```
1 name = "Python"
2 print(name[0])    # P - first character
3 print(name[1])    # y
4 print(name[-1])   # n - last character, counting from the end
5 print(len(name))  # 6 - the number of characters
```

Negative indices count backward from the end, which is genuinely useful: `name[-1]` is always the last character, regardless of the string's length, without you needing to calculate `len(name) - 1` yourself.

Strings Are Immutable

Once created, a string can never be changed in place. Every operation that looks like it's "modifying" a string is actually building and returning a brand new one:

```
1 name = "Python"
2 name[0] = "J"    # TypeError! strings don't support item assignment
```

This surprises people coming from languages where text is mutable. The fix isn't to modify the string; it's to build a new one and reassign the variable to point at it:

```
1 name = "Python"
2 name = "J" + name[1:]  # slicing, covered later in this lesson
3 print(name)           # Jython
```

`name` now points to an entirely new string; the original "Python" still exists unchanged in memory for as long as anything else needs it, then gets cleaned up automatically once nothing references it anymore.

Concatenation and Repetition

`+` joins strings together; `*` repeats one:

```
1 first = "Py"
2 second = "thon"
3 print(first + second)  # Python
4
5 separator = "_" * 20
6 print(separator)  # -----
```

Recall from Lesson 1.3 that `+` between a string and a non-string raises a `TypeError`; you must convert non-string values with `str()` first, or use an f-string instead, which is almost always the better choice.

f-strings: The Modern Way to Build Text

An f-string lets you embed variables and expressions directly inside a string, prefixed with `f`:

```
1 name = "Ada"
2 age = 28
3 print(f"{name} is {age} years old.")
4 # Ada is 28 years old.
5
6 price = 19.999
7 print(f"Total: ${price:.2f}")
8 # Total: $19.99
```

Anything inside `{}` is evaluated as real Python and inserted into the string, including expressions, not just plain variables:

```
1 a = 5
2 b = 3
3 print(f"{a} + {b} = {a + b}")
4 # 5 + 3 = 8
```

`:.2f` after a value, as shown with `price` above, is a format specifier; it means “show this as a fixed-point decimal with exactly 2 digits.” You’ll use format specifiers constantly once real numbers (prices, percentages, averages) start appearing in your output, since raw floats often print with far more decimal places than anyone wants to read.

Try It Yourself 1.7a: Create variables for a product’s name, price, and quantity. Using one f-string, print a line reading exactly like “3x Widget: \$29.97”, computing the total (`price * quantity`) directly inside the f-string’s braces, formatted to 2 decimal places.

Essential String Methods

A method is a function that belongs to a value, called with a dot: `value.method()`. You already met one in Lesson 1.3 (`.lower()`) without a full explanation; this section covers the string methods you’ll reach for constantly, one at a time, in real depth. It’s worth being precise about how a method differs from the plain functions you’ve used so far, like `print()` or `len()`: a plain function stands on its own, called by its name alone with the value passed inside the parentheses, `len(name)`, while a method is reached through a specific value using a dot, `name.upper()`, and only exists because that particular value’s type, here `str`, defines it. Different types have entirely different sets of methods available; a string’s methods won’t exist on an integer, and you’ll meet an entirely different set of methods on lists starting in Lesson 1.8. Strings have dozens of methods in total; a much smaller handful covers the vast majority of real, everyday use.

Case methods: `.upper()`, `.lower()`, `.title()`

```

1 name = "ada lovelace"
2
3 print(name.upper())    # ADA LOVELACE
4 print(name.lower())    # ada lovelace (already lowercase, unchanged)
5 print(name.title())    # Ada Lovelace - capitalizes the first letter of each word

```

`.upper()` and `.lower()` convert every letter in the string; non-letter characters (digits, punctuation, spaces) are left alone. `.title()` capitalizes the first letter of every word and lowercases the rest, which is useful for display purposes (turning a name typed all lowercase into something presentable), but it's not smart about exceptions; "mcdonald's farm".`title()` gives "Mcdonald'S Farm", capitalizing the S after the apostrophe too, which is rarely what you actually want. Case methods are also the standard way to make a comparison case-insensitive, exactly as you saw in Lesson 1.4: `user_input.lower() == stored_value.lower()`.

Whitespace: `.strip()`, `.lstrip()`, `.rstrip()`

```

1 text = "    Hello, World!    "
2
3 print(text.strip())      # "Hello, World!" - removes leading AND trailing whitespace
4 print(text.lstrip())    # "Hello, World!    " - removes only leading (left)
                           whitespace
5 print(text.rstrip())    # "    Hello, World!" - removes only trailing (right) whitespace

```

“Whitespace” means spaces, tabs, and newlines together, not just the space character. `.strip()` is one of the most quietly essential methods in the language: text typed by a real user, read from a file, or pasted from somewhere else almost always carries stray leading or trailing whitespace that you don't want, and `.strip()` is the standard way to clean it up before doing anything else with the value. None of these three methods touch whitespace that appears inside the string; "a b".`strip()` is still "a b", unchanged, since the spaces there aren't at either end.

Chaining methods together

```

1 raw_input = "  ADA@EXAMPLE.COM  "
2 clean = raw_input.strip().lower()
3 print(clean) # "ada@example.com"

```

Methods can be chained, one after another, because each one returns a brand-new string, which the next method in the chain then acts on, strictly left to right. `raw_input.strip().lower()` first strips the whitespace, producing an intermediate string, then lowercases that result. This is extremely common in real code: cleaning up user input is almost always a short chain of methods like this one, rather than several separate lines.

Replacing text: `.replace()`

```

1 sentence = "the quick brown fox jumps over the lazy fox"
2
3 print(sentence.replace("fox", "cat"))
4 # the quick brown cat jumps over the lazy cat
5
6 print(sentence.replace("fox", "cat", 1))
7 # the quick brown cat jumps over the lazy fox -- only the first match

```

`.replace(old, new)` replaces every occurrence of `old` with `new` by default, not just the first one, which surprises people expecting it to behave like a find-and-replace-once operation. An optional third argument limits how many replacements happen, counting from the left.

Splitting and joining: `.split()` and `.join()`

```
1 sentence = "the quick brown fox"
2 words = sentence.split()
3 print(words)
4 # ['the', 'quick', 'brown', 'fox'] -- splits on whitespace by default
5
6 csv_row = "Ada,28,London"
7 fields = csv_row.split(",")
8 print(fields)
9 # ['Ada', '28', 'London'] -- splits on a specific character instead
10
11 rejoined = " ".join(words)
12 print(rejoined)
13 # the quick brown fox -- the reverse of split
```

`.split()` breaks a string into a list of pieces (you’ll cover lists properly in Lesson 1.8, but you can already read the output here: square brackets holding several string values). With no argument, it splits on any run of whitespace, which conveniently collapses multiple spaces into a single split point. Pass a specific character, like `,`, and it splits on exactly that character instead, which is exactly how you’ll pull apart one line of data in Lesson 1.13.

`.join()` does the reverse: it glues a list of strings back together, inserting whatever string you called it on between each piece. The syntax trips up almost everyone the first time, because it reads backward from how you’d expect: `.join()` is called on the separator, not on the list. `" ".join(words)` means “join words together, using a single space between each piece,” not `words.join(" ")`, which is simply invalid; lists don’t have a `.join()` method at all.

Checking shape: `.startswith()`, `.endswith()`, `.find()`, and `in`

```
1 filename = "report.pdf"
2
3 print(filename.startswith("report")) # True
4 print(filename.endswith(".pdf"))    # True
5 print(filename.endswith(".docx"))   # False
6
7 print(filename.find("."))           # 6 - the index where "." FIRST appears
8 print(filename.find("xyz"))        # -1 - not found, no exception raised
9
10 print("port" in filename)          # True - the "in" keyword checks membership
11 print("xyz" in filename)           # False
```

`.startswith()` and `.endswith()` are the right, idiomatic tool for checking a string’s shape, and are almost always clearer and safer than manually slicing the first or last few characters yourself and comparing. `.find()` returns the index of the first match, or `-1` if nothing was found, deliberately not raising an exception on failure; this echoes the same “signal failure quietly instead of crashing” pattern you’ll see again with dictionaries’ `.get()` in Lesson 1.9. The `in` keyword is usually the cleanest choice when you only need a yes-or-no answer about whether one string appears inside another, without caring exactly where.

Try It Yourself 1.7a-ii: Given sentence = " The Rain In Spain ", write one chained expression that produces exactly "the rain in spain": stripped of surrounding whitespace and fully lowercased. Then use `.split()` on your cleaned result and print how many words it contains, using `len()`.

Slicing

Indexing, from earlier in this lesson, gets you exactly one character: `text[0]`. Slicing gets you a whole piece of a string at once, using the pattern `text[start:stop]`. This is one of the most useful tools in the entire language, and it's worth taking slowly, since the exact rule for which characters end up included is a common source of early confusion.

The basic rule: start is included, stop is not

```
1 text = "Hello, World!"
2 # 0123456789...
3
4 print(text[0:5])    # "Hello" - characters at index 0, 1, 2, 3, 4
5 print(text[7:12])   # "World" - characters at index 7, 8, 9, 10, 11
```

`text[0:5]` does not give you the characters at index 0 through index 5; it gives you the characters starting at index 0 and stopping before index 5, which means indices 0, 1, 2, 3, 4, five characters total. The number of characters you get back is always `stop - start`. This “stop is excluded” rule feels unnatural at first, but it has a genuinely useful property: the length of the slice is always simply `stop - start`, with no mental off-by-one adjustment needed, and two adjacent slices, like `text[0:5]` and `text[5:10]`, fit together perfectly with no overlap and no gap.

Leaving start or stop empty

```
1 text = "Hello, World!"
2
3 print(text[7:])      # "World!" - everything from index 7 to the very end
4 print(text[:5])      # "Hello" - everything from the start up to (not including) index 5
5 print(text[:])       # "Hello, World!" - the entire string, unchanged, as a full copy
```

An empty start means “begin at the very beginning of the string.” An empty stop means “continue all the way to the very end.” `text[:]` with both empty is a common, deliberate way to make a full copy of a string (more useful once you meet mutable sequences like lists in the next lesson, where copying actually matters).

Negative indices in slices

```
1 text = "Hello, World!"
2
3 print(text[-6:])     # "World!" - the last 6 characters
4 print(text[:-1])     # "Hello, World" - everything except the very last character
5 print(text[-6:-1])   # "World" - from 6-from-the-end, up to (not including) 1-from-the-end
```

Negative indices count backward from the end of the string, exactly as they did with plain indexing earlier in this lesson: `-1` is the last character, `-2` is second-to-last, and so on. They work inside a slice exactly the same way positive indices do; `text[-6:]` means “start 6 characters from the end, and go to the end,” and `text[:-1]` means “start at the beginning, and stop 1 character before the end,” which is a common, idiomatic way to chop off a single trailing character, like a newline.

Out-of-range slices don't raise an error

```
1 text = "Hi"
2 print(text[0:100])    # "Hi" - no error, just stops at the actual end
3 print(text[50:100])  # "" - no error, just an empty string
```

This is a genuinely useful difference from plain indexing: `text[100]` on a 2-character string raises an `IndexError` immediately, but a slice with an out-of-range start or stop never raises an error; it simply gives you as much as actually exists, even if that's nothing at all. This makes slicing a safe choice when you're not sure exactly how long a string will be.

The third number: step

```
1 text = "Hello, World!"
2
3 print(text[::2])      # "Hlo ol!" - every 2nd character, start to end
4 print(text[1::2])    # "el,Wrd" - every 2nd character, starting from index 1
5 print(text[::-1])    # "!dlroW ,olleH" - the whole string, reversed
6 print(text[::-2])    # "!lo,olH" - every 2nd character, walking backward
```

A full slice is actually `text[start:stop:step]`, and `step` defaults to 1 (“take every character”) when you leave it out, which is why every earlier example in this lesson worked without ever writing it. Setting `step` to 2 means “take every second character”; setting it to a negative number reverses the direction entirely, walking backward through the string instead of forward. `text[::-1]`, empty start, empty stop, step of `-1`, is the standard, idiomatic way to reverse a string in Python: “walk backward through the whole thing, from true end to true beginning.”

A realistic slicing example

```
1 filename = "quarterly_report_2026.pdf"
2
3 extension = filename[-3:]    # ".pdf" - last 3 characters
4 name_only = filename[:-4]    # "quarterly_report_2026" - drop ".pdf"
5 year = filename[-8:-4]      # "2026" - 4 characters, before the extension
6
7 print(f"File '{name_only}' is a .{extension} from {year}.")
8 # File 'quarterly_report_2026' is a .pdf from 2026.
```

This is exactly the kind of task slicing is genuinely built for: pulling structured pieces out of text whose shape you already know, without writing a character-by-character loop to do it by hand.

Try It Yourself 1.7b: Given `email = "ada@example.com"`, use `.find()` to locate the `@` symbol's index, then use slicing (not a hardcoded number) to extract just the username part, everything before the `@`, into its own variable. Then, separately, use slicing with a negative index to extract just the domain extension ("`com`") without hardcoding how long the whole email is.

Multi-line Strings

Triple quotes, either `"""` or `'''`, create a string that can span multiple lines exactly as typed:

```
1 message = """Dear Ada,
2
3 Thank you for your order.
4 It will ship within 3 days.
5 """
```

```
6 print(message)
```

This is the standard way to write a block of multi-line text (email templates, docstrings, which you'll meet properly in Lesson 1.12) without manually inserting line-break characters.

Escape Sequences

Some characters can't be typed directly inside a string, either because they'd end the string early or because they don't have a visible symbol. A backslash `\` followed by a letter represents them instead:

```
1 print("Line one\nLine two")      # \n - newline
2 print("Column1\tColumn2")        # \t - tab
3 print("She said \"hello\"")      # \" - a literal quote inside a double-quoted string
4 print("C:\\Users\\Ada")           # \\ - a literal backslash
```

`\n` is by far the most common of these; you'll use it constantly when building multi-line output without a triple-quoted string.

Common Mistakes to Avoid

- **Trying to modify a string in place.** `name[0] = "J"` raises `TypeError`, because strings are immutable. Build a new string and reassign the variable instead.
- **Forgetting slicing's stop index is excluded.** `text[0:5]` gives 5 characters, at indices 0 through 4, not through 5. This off-by-one confusion is extremely common at first.
- **Calling `.join()` on the wrong thing.** It belongs on the separator: `" ".join(words)`, not `words.join(" ")`. Reading it as “glue the list together using this separator” helps it stick.
- **Forgetting string methods return a new string rather than modifying in place.** `text.strip()` on its own, with no assignment, does nothing useful; you need `text = text.strip()` to actually keep the result.
- **Using single backslashes in Windows file paths without escaping them.** `"C:\Users"` is broken, because `\U` isn't a recognized escape sequence and Python raises a warning or error depending on context; use `"C:\\Users"` or, better, a raw string like `r"C:\Users"`.

Lesson 1.7 Project: Username Generator

Write a program called `username_generator.py` that:

1. Asks the user for their first name and last name using `input()`.
2. Generates a username by combining the first letter of their first name, their full last name, all lowercase, using slicing and `.lower()` (for example, "Ada" and "LoveLace" becomes "alovelace").
3. Checks whether the generated username is longer than 10 characters using `len()`; if so, truncate it to exactly 10 characters using slicing.
4. Prints the final username, along with the reversed version of it (using `[::-1]`) just to prove you understand slicing.

Lesson Summary: Quick Reference

- Strings are sequences (indexable, sliceable) but immutable (never modified in place)
- f-strings: `f"{name} is {age}"`; format specifiers like `:.2f` control decimal places

- `.upper()` `.lower()` `.strip()` `.replace()` `.split()` `.join()` cover most everyday text work
- `.startswith()` `.endswith()` `.find()` and the `in` keyword check shape and membership
- `text[start:stop:step]` slices; `text[::-1]` reverses a string

Lesson 1.8: Lists & Tuples

Why This Matters

Every variable you've created so far has held exactly one value. Real problems rarely come in quantities of one: a shopping list, a set of exam scores, the rows of a spreadsheet. Lists are how Python holds many values in a single, ordered variable, and they are, without exaggeration, the single most-used data structure in this entire course.

Learning Objectives

By the end of this lesson, you will be able to:

- Create, index, slice, and modify a list
- Use the list methods you'll reach for constantly: adding, removing, sorting, and searching
- Explain why copying a list with `=` is dangerous, and copy one correctly
- Use `enumerate()` and `zip()` to loop with a position or over several collections together
- Explain what a tuple is, and decide when to use one instead of a list

What Is a List?

A list is an ordered, changeable collection of values, written between square brackets, separated by commas:

```
1 fruits = ["apple", "banana", "cherry"]
2 scores = [85, 92, 78, 90]
3 mixed = ["Ada", 28, True, 3.9] # a list can hold different types at once
4
5 print(fruits)
6 print(len(fruits)) # 3 - the number of items
```

“Ordered” means the items stay in the exact sequence you put them in; the first item is always the first item, until you change it yourself. “Changeable” (or mutable, the formal term you’ll see used constantly from here on) means a list can be modified after creation: items added, removed, or replaced, all without creating a brand new list, unlike the strings from Lesson 1.7, which can never be changed in place.

Indexing and Slicing

Lists use the exact same indexing and slicing syntax as strings, because both are sequences in Python, and sequences all share this same interface:

```
1 fruits = ["apple", "banana", "cherry", "date"]
2
3 print(fruits[0])      # "apple" - first item
4 print(fruits[-1])     # "date" - last item
5 print(fruits[1:3])    # ["banana", "cherry"] - a slice, stop excluded, same rule as
                        # strings
6 print(fruits[::-1])   # ["date", "cherry", "banana", "apple"] - reversed
```

Everything you learned about slicing in Lesson 1.7, the excluded stop, negative indices, the optional step, applies here without any changes at all. This consistency, the same rules working across strings, lists, and (later) other sequence types, is deliberate language design, not a coincidence, and it's part of why Python feels predictable once these rules click.

Unlike strings, though, you can assign directly to an index to change a single item in place:

```
1 fruits = ["apple", "banana", "cherry"]
2 fruits[1] = "blueberry"
3 print(fruits) # ["apple", "blueberry", "cherry"]
```

This line would have raised a `TypeError` on a string, exactly as you saw in Lesson 1.7. On a list, it works precisely because lists are mutable.

Adding Items

Three methods cover almost every case of adding to a list, and they are not interchangeable:

```
1 fruits = ["apple", "banana"]
2
3 fruits.append("cherry")
4 print(fruits) # ["apple", "banana", "cherry"] - adds ONE item, at the end
5
6 fruits.insert(1, "apricot")
7 print(fruits) # ["apple", "apricot", "banana", "cherry"] - adds at a specific position
8
9 fruits.extend(["date", "elderberry"])
10 print(fruits) # [..., "date", "elderberry"] - adds every item from another list
```

`.append(item)` always adds exactly one item, at the very end. `.insert(index, item)` adds one item at a specific position, shifting everything from that position onward one slot to the right. `.extend(other_list)` adds every item from another list individually, which is different from `.append(other_list)` in a way that surprises people the first time:

```
1 a = [1, 2, 3]
2 b = [1, 2, 3]
3
4 a.append([4, 5])
5 print(a) # [1, 2, 3, [4, 5]] - the whole list got added as ONE nested item
6
7 b.extend([4, 5])
8 print(b) # [1, 2, 3, 4, 5] - each item got added individually
```

If you want the individual numbers 4 and 5 added to the list, you want `.extend()`; `.append()` with a list argument gives you a list inside your list, which is rarely what beginners actually intend.

Removing Items

Three more methods cover removal, each answering a slightly different question:

```
1 fruits = ["apple", "banana", "cherry", "banana"]
2
3 fruits.remove("banana")
4 print(fruits) # ["apple", "cherry", "banana"] - removes the FIRST matching value
5
6 popped = fruits.pop()
7 print(popped) # "banana" - the item that was removed
8 print(fruits) # ["apple", "cherry"] - .pop() with no argument removes the LAST item
```

```

9
10 popped_first = fruits.pop(0)
11 print(popped_first) # "apple"
12 print(fruits) # ["cherry"]
13
14 del fruits[0]

```

`.remove(value)` searches for a value and removes its first occurrence; if the value appears more than once, only the first match goes, and the rest stay. `.pop()` is different in a genuinely useful way: it removes an item and hands it back to you, which matters when you need to know what you just removed, not just that something was removed. With no argument, `.pop()` removes the last item; with an index, like `.pop(0)`, it removes and returns the item at that specific position. `del fruits[0]` is a third option, using Python's general-purpose `del` statement rather than a list method; it removes an item by index without handing it back to you at all.

Try It Yourself 1.8a: Create a list of five of your favorite movies. Use `.append()` to add a sixth. Use `.insert()` to add a movie at the very front of the list. Use `.pop()` to remove and print the last movie in the list. Print the final list.

Searching, Counting, and Membership

```

1 scores = [85, 92, 78, 92, 90]
2
3 print(92 in scores) # True - the "in" keyword, same as with strings
4 print(scores.count(92)) # 2 - how many times 92 appears
5 print(scores.index(92)) # 1 - the index of the FIRST 92

```

`in` checks membership and gives you a plain `bool`, exactly like it did for strings in Lesson 1.7. `.count(value)` tells you how many times a value appears, total. `.index(value)` tells you where the first occurrence is; unlike a string's `.find()`, calling `.index()` with a value that isn't in the list raises a `ValueError` instead of quietly returning `-1`, so it's worth checking `in` first if you're not sure the value is there.

Sorting and Reversing

```

1 scores = [85, 92, 78, 90]
2
3 scores.sort()
4 print(scores) # [78, 85, 90, 92] - sorts the list IN PLACE, permanently
5
6 scores.sort(reverse=True)
7 print(scores) # [92, 90, 85, 78]
8
9 names = ["Charlie", "ada", "Bob"]
10 names.sort(key=str.lower)
11 print(names) # ['ada', 'Bob', 'Charlie'] - sorted ignoring case

```

`.sort()` rearranges the list itself, permanently, and returns `None`; a very common early bug is writing `scores = scores.sort()`, which throws away the actual list and leaves `scores` holding `None` instead. If you need a sorted copy while leaving the original list untouched, use the built-in function `sorted()` instead, which returns a brand new list:

```

1 original = [3, 1, 2]
2 new_list = sorted(original)

```

```
3 print(original) # [3, 1, 2] - completely untouched
4 print(new_list) # [1, 2, 3] - a new, sorted list
```

`.sort()` (the method, modifies in place, returns `None`) versus `sorted()` (the function, leaves the original alone, returns a new list) is a distinction worth remembering precisely, since mixing them up is an easy, very common mistake. `.reverse()` works the same way as `.sort()`: it reverses the list in place, permanently, as opposed to the `[::-1]` slice from earlier, which produces a brand new, reversed list without touching the original.

The Aliasing Trap: Why `list2 = list1` Is Dangerous

This is the single most important, and most commonly misunderstood, idea in this entire lesson. Recall `==` versus `is` from Lesson 1.4: `==` asks “do these look the same,” `is` asks “are these literally the same object.” With lists, that distinction stops being a minor technicality and starts being something that can genuinely break your program:

```
1 list1 = [1, 2, 3]
2 list2 = list1 # NOT a copy! list2 points to the exact same list
3
4 list2.append(4)
5
6 print(list1) # [1, 2, 3, 4] - list1 changed too!
7 print(list2) # [1, 2, 3, 4]
8 print(list1 is list2) # True - they are literally the same object in memory
```

`list2 = list1` does not create a second, independent list. It creates a second name that points at the exact same list in memory as `list1` already does, the same way two nicknames can both refer to the same person. Modifying the list through either name modifies the one and only list that actually exists; there was never a second one to begin with. This is called aliasing, and it’s a direct, practical consequence of lists being mutable, unlike the numbers and strings you worked with in earlier lessons, where this problem literally cannot arise, since those types can never be changed in place at all.

To actually get an independent copy, one that you can modify without affecting the original, use `.copy()`, or equivalently a full slice, `[ : ]`:

```
1 list1 = [1, 2, 3]
2 list2 = list1.copy() # an independent copy this time
3 # list2 = list1[:]   # this does exactly the same thing
4
5 list2.append(4)
6
7 print(list1) # [1, 2, 3] - untouched
8 print(list2) # [1, 2, 3, 4]
9 print(list1 is list2) # False - two separate lists now
```

Try It Yourself 1.8b: Create a list called `original` with three numbers. Create `broken_copy = original` and `real_copy = original.copy()`. Append a fourth number to `broken_copy`, then append a different fifth number to `real_copy`. Print all three variables and explain to yourself, in a comment, exactly why they end up with the values they do.

Tuples: Ordered, Unchangeable Collections

A tuple is written with parentheses instead of square brackets, and behaves almost exactly like a list, with one deliberate difference: once created, it can never be changed:

```
1 point = (3, 7)
2 colors = ("red", "green", "blue")
3
4 print(point[0])      # 3 - indexing works exactly like a list
5 print(colors[1:])    # ('green', 'blue') - slicing works too
6
7 point[0] = 5         # TypeError! tuples are immutable, just like strings
```

Everything you can do to read a tuple, indexing, slicing, looping over it, checking membership with `in`, works identically to a list. What you cannot do is change it after creation: no `.append()`, no `.remove()`, no assigning to an index. This isn't a limitation so much as a promise: when you hand a tuple to another part of your program, you're guaranteeing that data won't be altered underneath you, which is exactly why tuples are the natural choice for things that are conceptually fixed, like a coordinate pair, an RGB color, or a calendar date's (year, month, day).

A single-item tuple needs a trailing comma, which trips up nearly everyone the first time:

```
1 not_a_tuple = (5)
2 print(type(not_a_tuple)) # <class 'int'> - just a plain int in parentheses!
3
4 actual_tuple = (5,)
5 print(type(actual_tuple)) # <class 'tuple'> - the comma is what matters, not the
                             parentheses
```

Tuple Unpacking

Tuples support a genuinely convenient pattern called unpacking: assigning each of a tuple's values to its own variable, in one line:

```
1 point = (3, 7)
2 x, y = point
3 print(x) # 3
4 print(y) # 7
5
6 name, age, city = "Ada", 28, "London" # this works with any sequence, not just tuples
7 print(name, age, city)
```

The number of variables on the left must match the number of values on the right, or Python raises a `ValueError`. This pattern comes up constantly once you start working with functions that return more than one value, starting in Lesson 1.11.

`enumerate()`: Looping With a Position

A plain `for` loop, from Lesson 1.6, gives you each item but not its position. Very often you need both, for instance to print a numbered list, and reaching for a manually managed counter is exactly the kind of thing Python gives you a cleaner tool for:

```
1 fruits = ["apple", "banana", "cherry"]
2
3 for index, fruit in enumerate(fruits):
4     print(f"{index}: {fruit}")
5 # 0: apple
6 # 1: banana
```

```
7 # 2: cherry
```

`enumerate(fruits)` produces pairs, an index and the corresponding item, one pair per position: conceptually, (0, "apple"), then (1, "banana"), then (2, "cherry"), each one a tuple exactly like the ones from earlier in this lesson. `for index, fruit in enumerate(fruits):` unpacks each of those pairs directly into two loop variables, in one step, using the exact same tuple unpacking pattern from earlier in this lesson; `index` takes the first element of each pair, `fruit` takes the second, and this happens fresh on every single pass through the loop. This is a dramatic improvement over the manual alternative:

```
1 # The old, more error-prone way - don't do this
2 index = 0
3 for fruit in fruits:
4     print(f"{index}: {fruit}")
5     index += 1
```

Both versions produce identical output, but the `enumerate()` version has no separate counter variable to remember to update, and no way to accidentally forget the increment. If you want numbering to start at 1 instead of 0, for something meant for a human to read, pass a second argument: `enumerate(fruits, start=1)`.

zip(): Looping Over Several Collections Together

`zip()` takes two or more collections and walks through them together, pairing up the items that share the same position:

```
1 names = ["Ada", "Alan", "Grace"]
2 scores = [92, 85, 88]
3
4 for name, score in zip(names, scores):
5     print(f"{name}: {score}")
6 # Ada: 92
7 # Alan: 85
8 # Grace: 88
```

Each pass through the loop gets one item from `names` and the item at that exact same position from `scores`, unpacked into two loop variables, again using tuple unpacking. This is the standard, idiomatic way to process two related lists side by side, instead of looping over one by index and manually indexing into the other. `zip()` works with more than two collections at once, and it stops as soon as the shortest one runs out, silently ignoring any leftover items in longer collections:

```
1 names = ["Ada", "Alan", "Grace"]
2 scores = [92, 85] # shorter than names!
3 cities = ["London", "Manchester", "Bristol"]
4
5 for name, score, city in zip(names, scores, cities):
6     print(f"{name}: {score}, {city}")
7 # Ada: 92, London
8 # Alan: 85, Manchester
9 # (Grace is silently skipped - scores ran out)
```

If mismatched lengths would be a bug in your program rather than something expected, check `len()` on each collection before zipping them, since `zip()` itself won't warn you.

Try It Yourself 1.8c: Given `days = ["Mon", "Tue", "Wed"]` and `high_temps = [72, 75, 68]`, use `zip()` to print a line for each day in the form "Mon: 72°F". Then use `enumerate()` on `days` alone to print each day preceded by its 1-based position, using `start=1`.

Choosing Between a List and a Tuple

Use a list when the collection might grow, shrink, or get reordered while your program runs, which describes the majority of real collections: a shopping cart, a to-do list, search results. Use a tuple when the collection represents a fixed, related group of values that shouldn't change once created, like a single record's coordinates, or when you want to signal to anyone reading your code, including future you, that this data is meant to stay exactly as it is.

Common Mistakes to Avoid

- **Writing `list2 = list1` expecting an independent copy.** It creates a second name for the exact same list. Use `.copy()` or `[:]` for a real, independent copy.
- **Writing `scores = scores.sort()`.** `.sort()` modifies the list in place and returns `None`; this line throws away your list and replaces it with `None`. Just call `scores.sort()` on its own line, with no assignment.
- **Confusing `.append()` with `.extend()`.** `.append(other_list)` nests the whole list as one item; `.extend(other_list)` adds each item individually. Pick based on which one you actually want.
- **Forgetting the comma in a single-item tuple.** `(5)` is just the integer 5 in parentheses; `(5,)`, with the trailing comma, is an actual one-item tuple.
- **Calling `.index()` on a value that isn't in the list.** Unlike a string's `.find()`, this raises `ValueError` rather than returning `-1`. Check with `in` first if you're not certain the value exists.

Lesson 1.8 Project: Grade Tracker

Write a program called `grade_tracker.py` that:

1. Starts with a list of five test scores of your choosing.
2. Adds a sixth score using `.append()`, entered via `input()` and converted to `int`.
3. Removes the lowest score using `min()` (look it up, or use `sorted()` to help) combined with `.remove()`.
4. Creates a genuine, independent copy of the remaining list using `.copy()`, sorts the copy in descending order, and prints both the original (unsorted) list and the sorted copy, proving the original was never touched.
5. Prints the average of the final scores, and uses tuple unpacking to pull out the highest and lowest remaining scores in one line: `lowest, highest = sorted(scores)[0], sorted(scores)[-1]`.

Lesson Summary: Quick Reference

- Lists `[]` are ordered and mutable; tuples `()` are ordered and immutable
- `.append()` `.insert()` `.extend()` add; `.remove()` `.pop()` del remove
- `list2 = list1` aliases, it does not copy; use `.copy()` or `[:]` for a real copy
- `.sort()` sorts in place and returns `None`; `sorted()` returns a new, sorted list
- `enumerate(x)` loops with a position; `zip(x, y)` loops over two collections together
- A one-item tuple needs a trailing comma: `(5,)`, not `(5)`

Lesson 1.9: Dictionaries & Sets

Why This Matters

A list is perfect when position matters: the first item, the third item. But plenty of real data isn't naturally ordered by position at all; it's naturally organized by name. A person's age isn't "the third thing about them," it's the value that belongs to the label "age." Dictionaries are how Python models that, and they are everywhere in real code, from configuration settings to API responses.

Learning Objectives

By the end of this lesson, you will be able to:

- Create a dictionary and safely access its values, without crashing on a missing key
- Use the essential dictionary methods, merge dictionaries with `.update()`, and iterate over keys, values, and pairs together using the `for` loop from Lesson 1.6
- Use a dictionary to count and group data, a genuinely common real-world pattern
- Create and use a set, and combine sets with union, intersection, difference, and symmetric difference

What Is a Dictionary?

A dictionary stores data as key-value pairs: each value is looked up by a unique, meaningful key, instead of by position. It's written with curly braces, with each pair as `key: value`:

```
1 person = {  
2     "name": "Ada",  
3     "age": 28,  
4     "city": "London",  
5 }  
6  
7 print(person["name"]) # "Ada"  
8 print(person["age"])  # 28
```

Compare this directly to a list: `fruits[0]` means "give me whatever happens to be first," which only makes sense if you already know the order. `person["name"]` means "give me the value stored under the key "name"", regardless of where it physically sits inside the dictionary. For data that's naturally described by named fields, a person's record, a product's details, a configuration file, this is a dramatically better fit than a list.

Keys are almost always strings in practice, though technically any immutable type works as a key, including numbers and tuples; values can be absolutely anything, including another list or dictionary. Keys must be unique within a dictionary; assigning to a key that already exists overwrites the old value rather than creating a duplicate entry, exactly the way `person["age"] = 29` would silently replace 28 with 29 rather than somehow storing both. This matters for a subtle reason worth flagging early: writing out a dictionary literal with the same key twice, like `{"name": "Ada", "name": "Grace"}`, isn't an error, since Python doesn't check for this in advance; it simply keeps the last value written for that key, silently discarding the first, which is a genuinely easy typo to make and not notice.

Other Ways to Create a Dictionary

The curly-brace literal you just saw is the most common way to write a dictionary, but it isn't the only one, and the alternatives come up often enough to be worth knowing:

```
1 empty = {} # an empty dictionary  
2 also_empty = dict() # the same thing, using the dict() constructor
```

```

3
4 person = dict(name="Ada", age=28) # keyword arguments become keys
5 print(person) # {'name': 'Ada', 'age': 28}
6
7 pairs = [("a", 1), ("b", 2), ("c", 3)]
8 from_pairs = dict(pairs) # a list of (key, value) tuples becomes a dict
9 print(from_pairs) # {'a': 1, 'b': 2, 'c': 3}

```

`dict()` with keyword arguments is a convenient shorthand when your keys are also valid variable names, though it only works for string keys written without quotes in the call itself. Building a dictionary from a list of (key, value) tuples is genuinely common in practice, especially once data is coming from somewhere else, like the paired output of `zip()` from Lesson 1.8: `dict(zip(names, scores))` is a frequently used, idiomatic one-liner for combining two parallel lists into a single dictionary.

Safely Accessing Values: `keyError` and `.get()`

Accessing a key that doesn't exist, using square brackets, crashes your program:

```

1 person = {"name": "Ada", "age": 28}
2 print(person["email"]) # KeyError: 'email'

```

This is the dictionary equivalent of a list's `IndexError` for an out-of-range position: asking for something that genuinely isn't there is treated as an error, not a quiet `None`. Often, though, you'd rather handle a missing key gracefully than crash outright, and `.get()` is built exactly for that:

```

1 person = {"name": "Ada", "age": 28}
2
3 print(person.get("email")) # None - no error, just None
4 print(person.get("email", "unknown")) # "unknown" - a custom default instead
5 print(person.get("name", "unknown")) # "Ada" - the key exists, so the default is
   ignored

```

`.get(key)` returns `None` if the key is missing, instead of raising an exception; `.get(key, default)` lets you supply exactly what should come back instead, which is almost always more useful than a bare `None`. This is the same “fail quietly instead of crashing” philosophy you first saw with a string's `.find()` in Lesson 1.7, applied here to lookups instead of searches. As a rule of thumb: use square brackets when a missing key genuinely represents a bug you want to know about immediately; use `.get()` whenever a missing key is a normal, expected possibility.

Checking whether a key exists before touching it at all uses the same `in` keyword you already know:

```

1 person = {"name": "Ada", "age": 28}
2
3 if "email" in person:
4     print(person["email"])
5 else:
6     print("No email on file.")

```

`in` on a dictionary checks its keys, not its values; `"Ada" in person` would be `False` here, even though `"Ada"` is a value in the dictionary, because `in` is answering “is this a key,” not “does this value appear anywhere.”

Try It Yourself 1.9a: Create a dictionary representing a book, with keys "title", "author", and "year". Use `.get()` to safely print a "publisher" key that doesn't exist, supplying "Unknown" as the default. Then use `in` to check for "title" before printing it.

Adding, Updating, and Removing

```

1 person = {"name": "Ada", "age": 28}
2
3 person["email"] = "ada@example.com" # adds a new key, since it didn't exist
4 person["age"] = 29                  # updates an existing key, since it did
5
6 print(person)
7 # {'name': 'Ada', 'age': 29, 'email': 'ada@example.com'}
8
9 del person["email"]
10 print(person)
11 # {'name': 'Ada', 'age': 29}
12
13 removed_age = person.pop("age")
14 print(removed_age) # 29
15 print(person)      # {'name': 'Ada'}
```

Assigning to a key with square brackets does double duty: it adds the key if it's new, and overwrites the value if it already exists, with the exact same syntax either way; Python figures out which one applies. `del` removes a key entirely, with no error if you first check the key exists. `.pop(key)` removes a key and returns its value in one step, mirroring exactly the list `.pop()` you met in Lesson 1.8, just working by key here instead of by position.

Adding or overwriting several keys at once, rather than one at a time, is what `.update()` is for:

```

1 person = {"name": "Ada", "age": 28}
2 new_info = {"age": 29, "city": "London"}
3
4 person.update(new_info)
5 print(person) # {'name': 'Ada', 'age': 29, 'city': 'London'}
```

`.update()` merges another dictionary into this one: any key that already exists gets its value overwritten, exactly like a single assignment would, and any new key gets added. This is the standard way to merge two dictionaries together, and it's considerably clearer than looping over the second dictionary's `.items()` and assigning each key by hand.

Try It Yourself 1.9a-ii: Create a dictionary of default settings, for example `{"volume": 50, "brightness": 70, "theme": "light"}`. Create a second dictionary with just one or two overrides, and use `.update()` to apply them to the first, printing the result.

Iterating Over a Dictionary

Using the `for` loop from Lesson 1.6, you can process an entire dictionary at once, in three different ways depending on what you actually need:

```

1 person = {"name": "Ada", "age": 28, "city": "London"}
2
3 for key in person:
4     print(key)
5 # name, age, city - looping directly gives you the KEYS
```

```
6
7 for value in person.values():
8     print(value)
9 # Ada, 28, London
10
11 for key, value in person.items():
12     print(f"{key}: {value}")
13 # name: Ada
14 # age: 28
15 # city: London
```

Looping over a dictionary directly gives you its keys, one at a time, which is a common source of confusion for people expecting values instead; if you want values, call `.values()` explicitly. `.items()` is what you'll reach for most often in practice: it gives you both the key and the value together, as a pair, which you unpack directly into two loop variables using the tuple unpacking pattern from Lesson 1.8. Since Python 3.7, dictionaries reliably preserve insertion order, so a loop like this always processes pairs in the exact order they were added, not in some arbitrary order.

One detail worth understanding precisely: `.keys()`, `.values()`, and `.items()` don't actually return lists, even though they behave like one inside a `for` loop. They return special view objects that stay connected to the dictionary they came from:

```
1 person = {"name": "Ada", "age": 28}
2 current_keys = person.keys()
3 print(current_keys) # dict_keys(['name', 'age'])
4
5 person["city"] = "London"
6 print(current_keys) # dict_keys(['name', 'age', 'city']) - it updated automatically!
```

`current_keys` wasn't frozen at the moment `.keys()` was called; it's a live window onto the dictionary's current keys, and it reflects changes made afterward. If you specifically need a real, independent list, snapshotted at this exact moment, wrap it explicitly: `list(person.keys())`.

A Complete Example: Counting Word Frequency

This is one of the single most common, genuinely useful patterns dictionaries are built for: counting how many times each distinct item appears in a collection.

```
1 text = "the quick brown fox jumps over the lazy dog the fox runs"
2 words = text.split()
3
4 word_counts = {}
5
6 for word in words:
7     if word in word_counts:
8         word_counts[word] += 1
9     else:
10        word_counts[word] = 1
11
12 print(word_counts)
13 # {'the': 3, 'quick': 1, 'brown': 1, 'fox': 2, 'jumps': 1,
14 #  'over': 1, 'lazy': 1, 'dog': 1, 'runs': 1}
```

Walk through the logic precisely: for every word, check whether it's already a key in `word_counts`. If it is, that word has been seen before, so increment its existing count by one. If it isn't, this is the first time this exact word has appeared, so create it fresh with a starting count of 1. By the time the loop finishes, every distinct word is a key, and its value is exactly how many times it appeared, built up one word at a time using the same accumulation idea from Lesson 1.6, just accumulating into dictionary values instead of a single running total.

There's a shorter way to write the same logic, using `.get()` with a default of 0 to collapse the `if/else` into one line:

```
1 word_counts = {}
2 for word in words:
3     word_counts[word] = word_counts.get(word, 0) + 1
```

`word_counts.get(word, 0)` returns the current count if the word has been seen, or 0 if it hasn't, which then has 1 added and gets stored right back, whether the key existed a moment ago or not. Once you're comfortable with the longer `if/else` version above and understand exactly why it works, this compressed version is worth recognizing, since you'll see it often in real code.

Try It Yourself 1.9b-ii: Given `grades = ["A", "B", "A", "C", "B", "A", "B"]`, use the counting pattern above to build a dictionary showing how many times each grade appears. Then find and print the most common grade, using `max()` with the dictionary's `.items()` and a `key=` function (hint: `max(grade_counts.items(), key=lambda pair: pair[1])` previews syntax you'll fully learn in Lesson 1.11, but try to reason through what it must be doing).

Nested Dictionaries

Values can be dictionaries themselves, which is how you model genuinely structured, real-world data:

```
1 students = {
2     "ada": {"age": 28, "grade": "A"},
3     "alan": {"age": 41, "grade": "B"},
4 }
5
6 print(students["ada"]["grade"]) # "A" - two lookups, chained
7
8 for name, info in students.items():
9     print(f"{name}: {info['age']} years old, grade {info['grade']}")
```

Each layer is looked up the same way, one [key] at a time, chained together; `students["ada"]` gets you the inner dictionary, and a second `["grade"]` reaches into that. This shape, a dictionary of dictionaries, is exactly what you'll see constantly once you start working with real JSON data in Lesson 1.13.

Sets: Unordered Collections of Unique Items

A set is written with curly braces like a dictionary, but holds individual values instead of key-value pairs, and enforces one strict rule: every item is unique, with duplicates automatically discarded.

```
1 numbers = {1, 2, 3, 2, 1}
2 print(numbers) # {1, 2, 3} - duplicates simply vanish
3
4 fruits = set(["apple", "banana", "apple"])
5 print(fruits) # {'apple', 'banana'}
```

Note that `{}` on its own creates an empty dictionary, not an empty set, since dictionaries came first and claimed that syntax; for an empty set, you have to use `set()` explicitly. A set has no concept of order or position at all, which means no indexing, no slicing, and printing one may show its items in a different order than you typed them.

```

1 tags = {"python", "tutorial"}
2
3 tags.add("beginner")
4 print(tags) # {'python', 'tutorial', 'beginner'}
5
6 tags.remove("tutorial")
7 print(tags) # {'python', 'beginner'}
8
9 print("python" in tags) # True - and it's extremely fast to check, even on huge sets

```

`.add()` adds one item, doing nothing at all if it's already present, silently, rather than raising an error; `.remove()` removes an item, but does raise an error if that item isn't there (`.discard()` is the safer alternative, removing an item if present and doing nothing otherwise). Checking membership with `in` on a set is dramatically faster than on a list once the collection gets large, which is one of the two main reasons to reach for a set in the first place.

Try It Yourself 1.9c: Create a set of tags for a blog post, add three more one at a time with `.add()`, then use `.discard()` to safely remove one that may or may not actually be present, without risking an error either way.

Combining Sets: Union, Intersection, and Difference

Sets support the mathematical operations you'd expect from a Venn diagram, and they're genuinely useful for every-day problems, not just academic exercises. Picture two overlapping circles, one for each set; every operation below is really just asking a specific question about that picture. Set up two sets to work through each one concretely:

```

1 python_students = {"ada", "alan", "grace"}
2 sql_students = {"alan", "grace", "linus"}

```

Union (`|`): everyone in either circle. The union combines both sets into one, keeping every item that appears in at least one of them, with duplicates (items in both) still only counted once, since a set can never hold duplicates in the first place:

```

1 print(python_students | sql_students)
2 # {'ada', 'alan', 'grace', 'linus'} - everyone enrolled in at least one course

```

Intersection (`&`): only what's in both circles at once. The intersection keeps exclusively the items that appear in every set involved, discarding anything that only shows up in one:

```

1 print(python_students & sql_students)
2 # {'alan', 'grace'} - enrolled in BOTH courses

```

Difference (`-`): in the left circle, but subtract out the right one. The difference keeps items from the first set, removing anything that also appears in the second; order matters here, unlike union and intersection, since “Python students minus SQL students” and “SQL students minus Python students” are genuinely different questions:

```

1 print(python_students - sql_students)
2 # {'ada'} - Python only, not also taking SQL
3

```

```

4 print(sql_students - python_students)
5 # {'linus'} - SQL only, not also taking Python

```

Symmetric difference (^): in exactly one circle, not both. This is the one most people haven't seen before: it keeps everything that's in either set, but specifically excludes anything in both, the mirror image of intersection:

```

1 print(python_students ^ sql_students)
2 # {'ada', 'linus'} - taking exactly ONE of the two courses, not both

```

Every one of these four operators also has an equivalent, more explicitly named method, which some people find more readable in code meant to be skimmed quickly: `.union()`, `.intersection()`, `.difference()`, and `.symmetric_difference()`, called exactly like any other method:

```

1 print(python_students.union(sql_students))           # same as |
2 print(python_students.intersection(sql_students))    # same as &
3 print(python_students.difference(sql_students))      # same as -
4 print(python_students.symmetric_difference(sql_students)) # same as ^

```

Both forms do exactly the same work; the operators are more compact, the methods sometimes read more clearly in longer expressions, especially once more than two sets are involved. “Which students are enrolled in both courses,” “which students are only taking one,” “everyone across both courses”, these are exactly the kind of question that takes one line with sets, and considerably more code, typically a loop with manual membership checks, if you tried to answer them using lists instead.

Try It Yourself 1.9d: Create two sets of your own, representing, for example, ingredients you have and ingredients a recipe needs. Use `-` to find which ingredients you're missing, `&` to find which required ingredients you already have, and `^` to find which ingredients are either extra (you have them but don't need them) or missing (needed but you don't have them).

Choosing the Right Collection

Four collections, four different jobs: a list for an ordered collection that might change; a tuple for an ordered, fixed group of related values; a dictionary for data naturally organized by named key rather than position; a set for a collection where uniqueness matters and order doesn't. Real programs use all four, often together, choosing based on what the data actually represents, not out of habit.

Common Mistakes to Avoid

- **Using square brackets on a key that might not exist.** `person["email"]` raises `KeyError` if the key is missing. Use `.get()` when a missing key is a normal, expected possibility.
- **Forgetting a loop over a dictionary gives you keys, not values.** `for x in person:` gives keys. Use `.values()` for values or `.items()` for both together.
- **Writing `{}` expecting an empty set.** It creates an empty dictionary. Use `set()` for an empty set.
- **Assuming `.update()` keeps the original value on a conflict.** It overwrites; whatever the argument to `.update()` provides wins for any key that exists in both.
- **Expecting a set to preserve order or support indexing.** `my_set[0]` raises an error; sets have no positions at all. Use a list if order or indexing matters.

Lesson 1.9 Project: Contact Book

Write a program called `contact_book.py` that:

1. Starts with a dictionary of contacts, where each key is a name and each value is a nested dictionary holding "phone" and "email".
2. Uses a for loop with `.items()` to print every contact in a neatly formatted line.
3. Asks the user for a name using `input()`, and uses `.get()` to safely look them up, printing their details if found and "Contact not found." if not, without crashing either way.
4. Builds two sets, `gmail_users` and `work_users`, of names based on whether their email contains "gmail" or a company domain of your choosing, and prints the intersection: contacts who are in both groups.

Lesson Summary: Quick Reference

- `dict(...)` builds from keyword arguments or `(key, value)` pairs, in addition to `{}`
- `d[key]` raises `KeyError` if missing; `d.get(key, default)` fails safely instead; `.update()` merges dictionaries
- Looping over a dict directly gives keys; use `.values()` or `.items()` for values or pairs; the counting-with-`.get()` pattern turns a collection into a frequency dictionary
- Sets (`{}` won't work, use `set()` for empty) hold unique, unordered items; `|` & `-` ^ give union, intersection, difference, and symmetric difference
- Choose list/tuple/dict/set based on whether order, uniqueness, or named keys actually matter for your data

Lesson 1.10: List/Dict/Set Comprehensions

Why This Matters

You now know how to build a list with a `for` loop: start empty, loop, append. That exact three-line pattern shows up so often in real Python code that the language provides a dedicated, compact shorthand for it. Comprehensions aren't a new concept; they're a faster way to write a pattern you already understand completely.

Learning Objectives

By the end of this lesson, you will be able to:

- Rewrite a simple `for` loop that builds a list as a list comprehension
- Add a filtering condition to a comprehension
- Write dictionary and set comprehensions
- Recognize when a comprehension has become too complex, and a full loop is the better choice

From Loop to List Comprehension

Here's a completely ordinary task: build a new list holding the square of every number in an existing list. Using what you already know from Lessons 1.7 and 1.8, you'd write this:

```
1 numbers = [1, 2, 3, 4, 5]
2 squares = []
3
4 for n in numbers:
5     squares.append(n ** 2)
6
7 print(squares) # [1, 4, 9, 16, 25]
```

This three-line shape, start with an empty list, loop, append a transformed value, is extremely common; you will write it, or read it, dozens of times in real code. A list comprehension expresses exactly the same idea in a single line:

```
1 numbers = [1, 2, 3, 4, 5]
2 squares = [n ** 2 for n in numbers]
3
4 print(squares) # [1, 4, 9, 16, 25]
```

Read `[n ** 2 for n in numbers]` in the same order you'd say it out loud: "n squared, for each n in numbers." The general pattern is `[expression for item in iterable]`: pick a name for each item as you go (n here, though it could be any name), write the expression you want computed from it, and Python builds a brand new list by running that expression once per item, in order, collecting every result. Line the two versions up side by side and the correspondence is exact: the loop's `for n in numbers:` becomes the comprehension's `for n in numbers`, and the loop's `squares.append(n ** 2)` becomes the comprehension's leading `n ** 2`, the expression describing what to collect. Nothing here is new syntax you haven't already used elsewhere, the `for ... in ...` is the exact loop header from Lesson 1.6; a comprehension simply repackages it as an expression that produces a list, instead of a statement that repeatedly calls `.append()`.

```
1 words = ["hello", "world", "python"]
2
```

```
3 lengths = [len(w) for w in words]
4 print(lengths) # [5, 5, 6]
5
6 shouted = [w.upper() for w in words]
7 print(shouted) # ['HELLO', 'WORLD', 'PYTHON']
```

The expression on the left can be anything: a calculation, a method call, a completely different value built from the item, exactly as it could be inside a full loop's `.append()` call.

Try It Yourself 1.10a: Given `prices = [19.99, 5.50, 12.00, 8.25]`, write a list comprehension that produces a new list with each price after a 10% discount, rounded to 2 decimal places using `round(value, 2)`.

Adding a Condition

A comprehension can also filter, keeping only items that satisfy a condition, by adding an `if` at the end:

```
1 numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
2
3 evens = [n for n in numbers if n % 2 == 0]
4 print(evens) # [2, 4, 6, 8, 10]
```

Read this as “n, for each n in numbers, but only if n is even.” The equivalent full loop makes the relationship between the two forms completely explicit:

```
1 evens = []
2 for n in numbers:
3     if n % 2 == 0:
4         evens.append(n)
```

Every comprehension with a trailing `if` is exactly this shape: a loop with a condition guarding whether `.append()` runs, compressed into one line. The condition can be anything that evaluates to a `bool`, including a call to a string method:

```
1 words = ["apple", "kiwi", "banana", "fig", "cherry"]
2
3 long_words = [w for w in words if len(w) > 4]
4 print(long_words) # ['apple', 'banana', 'cherry']
5
6 capitalized = [w.title() for w in words if w.startswith("b")]
7 print(capitalized) # ['Banana']
```

Notice the second example transforms and filters in the same comprehension: the expression `(w.title())` and the condition `(w.startswith("b"))` are independent of each other, and can use the loop variable however each one needs to.

Dictionary Comprehensions

The same idea works for building a dictionary, using curly braces and a key: value expression instead of a single value. Start with the equivalent full loop, the same way you did for lists, since seeing the loop first makes the shorthand easier to trust:

```
1 words = ["apple", "kiwi", "banana"]
```

```

2 lengths = {}
3
4 for w in words:
5     lengths[w] = len(w)
6
7 print(lengths) # {'apple': 5, 'kiwi': 4, 'banana': 6}

```

As a dictionary comprehension, this becomes:

```

1 words = ["apple", "kiwi", "banana"]
2 lengths = {w: len(w) for w in words}
3 print(lengths) # {'apple': 5, 'kiwi': 4, 'banana': 6}

```

The general pattern is `{key_expression: value_expression for item in iterable}`: two expressions this time, separated by a colon, both built from the same loop variable. Read `{w: len(w) for w in words}` exactly like the list version, just with two things being computed per pass instead of one: “w maps to len(w), for each w in words.” The part before the colon becomes the key; the part after becomes the value that key points to. Nothing requires the key to be the loop variable itself, unchanged, the way it was here; both sides can be any expression at all, including calling a completely different function on each one.

A genuinely common, illustrative use is flipping a dictionary around, building a new one where the old values become keys and the old keys become values:

```

1 name_to_id = {"ada": 101, "alan": 102, "grace": 103}
2
3 id_to_name = {student_id: name for name, student_id in name_to_id.items()}
4 print(id_to_name)
5 # {101: 'ada', 102: 'alan', 103: 'grace'}

```

Notice the loop unpacks each `.items()` pair into `name`, `student_id`, exactly as a full loop would, and the comprehension simply writes them back out in the opposite order, `student_id` before the colon this time instead of after. This only works cleanly when the original values are themselves unique enough to serve as keys; if two students shared an ID, one entry would silently overwrite the other, since dictionary keys must stay unique.

Dictionary comprehensions are especially useful for transforming an existing dictionary, iterating with `.items()` exactly as you learned in Lesson 1.9, keeping the same keys but recomputing every value:

```

1 prices = {"apple": 1.50, "banana": 0.75, "cherry": 4.00}
2
3 discounted = {item: round(price * 0.9, 2) for item, price in prices.items()}
4 print(discounted) # {'apple': 1.35, 'banana': 0.68, 'cherry': 3.6}
5
6 expensive = {item: price for item, price in prices.items() if price > 1.00}
7 print(expensive) # {'apple': 1.5, 'cherry': 4.0}

```

The filtering `if` works exactly the way it did for list comprehensions, at the very end, and it filters based on whichever items should be kept in the new dictionary; here it keeps the key-value pair entirely, unchanged, whenever the price condition passes, and drops the pair completely otherwise. You can also build a dictionary directly from two separate, parallel lists, combining the comprehension syntax with `zip()` from Lesson 1.8:

```

1 names = ["Ada", "Alan", "Grace"]

```

```

2 scores = [92, 85, 88]
3
4 grade_book = {name: score for name, score in zip(names, scores)}
5 print(grade_book) # {'Ada': 92, 'Alan': 85, 'Grace': 88}

```

This is such a common need, turning two parallel lists into one dictionary, that it's worth recognizing on sight: `zip()` pairs the two lists up position by position, and the comprehension turns each pair into one key-value entry.

Try It Yourself 1.10a-ii: Given `celsius = {"Mon": 20, "Tue": 25, "Wed": 18}`, write a dictionary comprehension that converts every value to Fahrenheit (the formula is $f = c * 9/5 + 32$), keeping the same day-name keys. Then write a second dictionary comprehension, from the original `celsius` dictionary, that keeps only days where the temperature was above 19.

Set Comprehensions

A set comprehension uses curly braces with a single expression, no colon, mirroring the difference between set and dictionary literals from Lesson 1.9:

```

1 words = ["apple", "kiwi", "banana", "kiwi", "apple"]
2
3 unique_lengths = {len(w) for w in words}
4 print(unique_lengths) # {4, 5, 6} - a set, so duplicates collapse automatically

```

This is genuinely useful whenever you want the distinct set of some computed value across a collection, and don't care about duplicates or order at all; here, several words share a length, but each length appears in the result exactly once.

Try It Yourself 1.10b: Given `students = {"ada": 92, "alan": 78, "grace": 88, "linus": 65}`, write a dict comprehension that keeps only students with a score of 80 or above. Then write a set comprehension over the same dictionary that collects the distinct first letters of every student's name.

When Not to Use a Comprehension

Comprehensions are for building a new collection from an existing one, ideally in one clear, readable line. The moment a comprehension needs more than one condition stacked awkwardly, nested loops, or logic that's genuinely hard to read on a single line, a full `for` loop is the better, more maintainable choice, not a lesser one:

```

1 # Technically legal, but a poor choice - too much happening in one line
2 result = [x * 2 if x % 2 == 0 else x * 3 for x in range(20) if x > 5 if x < 15]
3
4 # The same logic, as a full loop - genuinely easier to read and to debug
5 result = []
6 for x in range(20):
7     if x > 5 and x < 15:
8         if x % 2 == 0:
9             result.append(x * 2)
10        else:
11            result.append(x * 3)

```

There's no strict rule for exactly where this line falls, but a reasonable guideline is: if you have to read a comprehension twice to understand what it does, or if you find yourself nesting conditions inside conditions, write it as a full loop instead. Comprehensions exist to make simple, common patterns shorter and more readable, not to compress

everything into a single line as a matter of pride; readability always wins over cleverness in code you or anyone else will need to revisit later.

Common Mistakes to Avoid

- **Forgetting the expression comes first.** A comprehension reads `[expression for item in iterable]`, expression before the loop, which feels backward compared to a normal loop's line order at first.
- **Using `[]` when you meant a set or dict comprehension.** Square brackets always produce a list; use `{expr: expr}` for a dict comprehension and `{expr}` (no colon) for a set.
- **Cramming too much logic into one comprehension.** If it needs a second read to understand, or nests multiple conditions, write it as a full loop instead.
- **Forgetting the condition goes at the end, not inside the expression.** `[n for n if n % 2 == 0 in numbers]` is invalid; the correct order is `[n for n in numbers if n % 2 == 0]`.
- **Rewriting every single loop as a comprehension out of habit.** Not every loop should become one; a loop with multiple side effects (printing, updating several variables) usually shouldn't be forced into comprehension form at all.

Lesson 1.10 Project: Inventory Report

Write a program called `inventory_report.py` that:

1. Starts with a dictionary of products and their stock counts, for example `{"widget": 4, "gadget": 0, "gizmo": 15, "doohickey": 2}`.
2. Uses a list comprehension to build a list of product names that are out of stock (count equal to 0).
3. Uses a dict comprehension to build a new dictionary containing only products with fewer than 5 in stock, labeled "low_stock".
4. Uses a set comprehension to collect the distinct first letters of every product name.
5. Prints all three results with clear labels.

Lesson Summary: Quick Reference

- `[expr for item in iterable]` builds a list; add `if` condition at the end to filter
- `{key: value for item in iterable}` builds a dict; `{expr for item in iterable}` builds a set
- Every comprehension is shorthand for a loop that starts empty and appends/assigns each result
- If a comprehension needs a second read to understand, write it as a full loop instead

Lesson 1.11: Functions

Why This Matters

Every lesson so far has had you write code that runs once, top to bottom. Real programs reuse logic constantly: the same validation check in ten places, the same calculation on different numbers, the same setup repeated across a project. A function is a named, reusable block of code, and it is, without exaggeration, the single most important organizing tool in this entire language. This lesson is long, on purpose; functions touch nearly everything else you'll write from here on.

Learning Objectives

By the end of this lesson, you will be able to:

- Define a function, call it with arguments, and return a value from it
- Explain the difference between a local and a global variable, and when each applies
- Write functions with default values, and accept any number of arguments with `*args` and `**kwargs`
- Write short, throwaway functions with `lambda`, and use them with `map()` and `filter()`
- Write a recursive function, and explain why it needs a base case

Defining and Calling a Function

You've been calling functions since Lesson 1.2: `print()`, `len()`, `input()` are all functions, just ones Python already built for you. `def` lets you write your own:

```
1 def greet():
2     print("Hello!")
3     print("Welcome to the program.")
4
5 greet()      # this is what actually runs the function
6 greet()      # calling it again runs it again, from the top
```

`def greet():` defines a function named `greet`; everything indented underneath is its body, the code that runs each time the function is called, following the exact same indentation rule from Lesson 1.5 that marks an `if` block. Defining a function does not run it; the two `print()` lines only execute when `greet()` is actually called, with the parentheses, exactly like every built-in function you've already used. If your program only ever contained `def greet(): print("Hello!")`, with no call to `greet()` anywhere below it, running that program would print nothing at all; Python would read the definition, store it under the name `greet`, and move on, without ever executing what's inside. This is worth sitting with: a function definition is a recipe, written once; calling it is actually cooking the meal, which you can do as many times as you like, and writing the recipe down doesn't put any food on the table by itself.

Parameters and Arguments

A function that always does exactly the same thing is rarely useful. Parameters let you pass information in:

```
1 def greet(name):
2     print(f"Hello, {name}!")
3
4 greet("Ada")    # Hello, Ada!
5 greet("Alan")  # Hello, Alan!
```

name is a parameter: a placeholder variable, listed in the function's definition, that receives whatever value is passed in when the function is called. "Ada" and "Alan" are arguments: the actual values supplied at the call site. The terms are related but distinct, and worth using precisely: name is the parameter (part of the definition); "Ada" is the argument (part of the call). A function can take several parameters, separated by commas, and arguments are matched to them by position, left to right, unless you name them explicitly:

```
1 def describe_person(name, age, city):
2     print(f"{name} is {age} years old and lives in {city}.")
3
4 describe_person("Ada", 28, "London")           # matched by position
5 describe_person(age=28, city="London", name="Ada") # matched by name, any order
```

Calling with name=, age=, and so on is called using keyword arguments; it's slower to type but self-documenting, and lets you supply arguments in any order, which is genuinely useful once a function has several parameters that are easy to mix up by position alone.

The return Statement

Everything so far has functions that `print()` their result directly. Usually you want a function to hand a value back, so the calling code can use it, store it, or pass it somewhere else. That's what `return` does:

```
1 def add(a, b):
2     return a + b
3
4 result = add(3, 4)
5 print(result)      # 7
6 print(add(10, 5)) # 15 - can be used directly, without a variable
```

The moment Python hits a `return` statement, the function stops immediately and hands that value back to wherever it was called from; any code after `return`, still inside the function, never runs at all. A function with no `return` statement, or a bare `return` with no value, actually returns `None`, Python's way of saying "nothing," which is exactly what `greet()` was doing implicitly in the earlier examples: it printed something, but it never handed anything back.

```
1 def greet(name):
2     print(f"Hello, {name}!")
3
4 result = greet("Ada") # Hello, Ada! (printed)
5 print(result)        # None - greet() never used return
```

This is a genuinely important distinction, worth internalizing early: `print()` displays something to the screen, for a human to read; `return` hands a value back into the program, for code to keep using. A function can do either, both, or neither, and mixing them up, printing when you meant to return, is one of the most common early mistakes with functions, covered fully in Common Mistakes at the end of this lesson.

Try It Yourself 1.11a: Write a function `is_even(number)` that returns `True` if the number is even and `False` otherwise, using the `%` operator from Lesson 1.4. Call it with several different numbers and print each result.

Multiple Return Values

A function can return more than one value at once, separated by commas; Python packages them into a tuple automatically, which you can unpack immediately using the pattern from Lesson 1.8:

```
1 def min_max(numbers):
2     return min(numbers), max(numbers)
3
4 lowest, highest = min_max([4, 9, 1, 7, 3])
5 print(lowest)    # 1
6 print(highest)   # 9
```

`return min(numbers), max(numbers)` is really returning one tuple, `(1, 9)`; the two-variable assignment on the left unpacks it in the same call, in one line. If you only wanted the whole tuple, `result = min_max([4, 9, 1, 7, 3])` would work too, giving you `result = (1, 9)` to index or unpack later, whenever you actually need to.

Scope: Local vs Global Variables

A variable created inside a function only exists inside that function; this is called local scope, and it's one of the most important ideas in this entire lesson:

```
1 def calculate_total():
2     total = 100 # total is LOCAL to this function
3     print(total)
4
5 calculate_total() # 100
6 print(total)      # NameError! total doesn't exist out here
```

`total` is created fresh, inside `calculate_total()`, every time the function runs, and it's destroyed the instant the function finishes. Trying to use it outside the function, where it was never defined, raises a `NameError`. This isn't a limitation so much as a deliberate safety feature: it means two completely different functions can both use a variable named `total` internally, without any risk of them interfering with each other, since each function's local variables are entirely private to that one call.

A variable created outside any function, at the top level of your file, is a global variable, and functions can read it, without any special syntax:

```
1 tax_rate = 0.08 # global
2
3 def calculate_total(price):
4     return price * (1 + tax_rate) # reading the global works fine
5
6 print(calculate_total(100)) # 108.0
```

Reading a global variable from inside a function works automatically; Python looks for `tax_rate` inside the function first (it's not there), then checks the surrounding, global scope (it's there), and uses that. Assigning to a variable inside a function is different, though, and this is where beginners consistently get surprised:

```
1 counter = 0
2
3 def increment():
4     counter = counter + 1 # UnboundLocalError!
5
6 increment()
```

The moment Python sees any assignment to `counter` anywhere inside the function, it decides `counter` is a local variable for that entire function, from the very first line, even before that assignment line runs. So `counter + 1` tries to read a local `counter` that doesn't have a value yet, and Python raises an error rather than silently reading the global one. If you genuinely need to modify a global variable from inside a function, you have to say so explicitly, with the `global` keyword:

```

1 counter = 0
2
3 def increment():
4     global counter
5     counter = counter + 1
6
7 increment()
8 increment()
9 print(counter)  # 2

```

`global counter` tells Python, up front, “don’t create a local variable named `counter`; use the global one, and let me modify it.” In practice, relying on `global` is usually a sign a function could be redesigned to take a parameter and return a value instead, which is almost always clearer and safer; it’s worth knowing `global` exists and understanding exactly why `UnboundLocalError` happens without it, more than it’s worth reaching for it often in code you write yourself.

Try It Yourself 1.11b: Write a function `make_uppercase(text)` that takes a string parameter and returns it upcased, without touching any global variable at all. Then, separately, predict what happens if you try to print that function’s parameter `text` from outside the function, and verify your prediction by actually trying it.

Default Parameter Values

A parameter can have a default value, used automatically whenever the caller doesn’t supply one:

```

1 def greet(name, greeting="Hello"):
2     print(f"{greeting}, {name}!")
3
4 greet("Ada")                # Hello, Ada! - greeting uses its default
5 greet("Ada", "Hi")          # Hi, Ada! - greeting is overridden
6 greet("Ada", greeting="Welcome") # Welcome, Ada! - same, as a keyword argument

```

`greeting="Hello"` means “if the caller doesn’t provide a value for `greeting`, use `"Hello"` instead.” This makes a function flexible without forcing every caller to specify every single detail, which is exactly why so many real functions, including many you’ve already used, have parameters you’ve never had to touch yourself. Python requires all parameters with defaults to come after every parameter without one; `def greet(greeting="Hello", name):` is a `SyntaxError`, because Python would have no way to know, from `greet("Ada")` alone, whether `"Ada"` was meant for `greeting` or `name`.

One default-value trap is worth knowing about before it surprises you: never use a mutable value, like a list, as a default directly.

```

1 def add_item(item, cart=[]): # DANGEROUS - don't do this
2     cart.append(item)
3     return cart
4
5 print(add_item("apple"))    # ['apple']
6 print(add_item("banana"))  # ['apple', 'banana'] - NOT a fresh list!

```

A default value is created exactly once, when the function is defined, not fresh on every call; every call that relies on the default is silently sharing that exact same list, an aliasing problem directly related to the one you saw with lists in Lesson 1.8. The standard fix is to default to `None`, and create the real, fresh list inside the function body instead:

```
1 def add_item(item, cart=None):
2     if cart is None:
3         cart = []
4     cart.append(item)
5     return cart
```

Now every call that doesn't supply its own `cart` gets a genuinely new, empty list, created fresh inside the function body each time, rather than one shared list reused silently across every call.

***args: Any Number of Positional Arguments**

Every function so far has taken a fixed, known number of arguments. `*args` removes that limit:

```
1 def total(*args):
2     print(args)
3     return sum(args)
4
5 print(total(1, 2))           # (1, 2) -> 3
6 print(total(1, 2, 3, 4))    # (1, 2, 3, 4) -> 10
7 print(total())              # () -> 0
```

The single `*` before `args` tells Python “gather up every positional argument passed in, however many there are, and package them into a tuple called `args`.” The name `args` is just convention, not a keyword, but every Python developer recognizes `*args` on sight, so it's strongly preferred over any other name. Inside the function, `args` behaves exactly like any tuple from Lesson 1.8: you can loop over it, index it, or pass it straight to a function like `sum()` that accepts an iterable.

****kwargs: Any Number of Keyword Arguments**

`**kwargs` does the same job for keyword arguments, ones passed by name:

```
1 def describe(**kwargs):
2     print(kwargs)
3     for key, value in kwargs.items():
4         print(f"{key}: {value}")
5
6 describe(name="Ada", age=28, city="London")
7 # {'name': 'Ada', 'age': 28, 'city': 'London'}
8 # name: Ada
9 # age: 28
10 # city: London
```

The double `**` tells Python “gather up every keyword argument passed in, however many there are, and package them into a dictionary called `kwargs`.” Once inside the function, `kwargs` is an ordinary dictionary, exactly as covered in Lesson 1.9: `.items()`, `.get()`, and everything else works precisely as you'd expect.

Combining Regular Parameters, *args, and **kwargs

All three can appear in the same function, but Python requires a specific order: regular parameters first, then *args, then **kwargs.

```

1 def create_profile(username, *interests, **details):
2     print(f"Username: {username}")
3     print(f"Interests: {interests}")
4     print(f"Details: {details}")
5
6 create_profile("ada_l", "coding", "math", age=28, city="London")
7 # Username: ada_l
8 # Interests: ('coding', 'math')
9 # Details: {'age': 28, 'city': 'London'}
```

Trace through how Python sorts the arguments: "ada_l" fills the regular parameter username. Everything after it that's passed positionally ("coding", "math") gets swept into interests (the *args). Everything passed by name (age=28, city="London") gets swept into details (the **kwargs). Python figures out where each argument belongs automatically, based purely on whether it was passed positionally or by name; you never have to sort them yourself.

The reverse direction works too: * and ** can unpack an existing list or dictionary directly into a function call:

```

1 def add(a, b, c):
2     return a + b + c
3
4 numbers = [1, 2, 3]
5 print(add(*numbers)) # same as add(1, 2, 3)
6
7 values = {"a": 1, "b": 2, "c": 3}
8 print(add(**values)) # same as add(a=1, b=2, c=3)
```

This is genuinely useful when your data is already sitting in a list or dictionary, perhaps read from the JSON files you'll meet in Lesson 1.13, and you want to feed it directly into a function that expects separate arguments, without manually pulling each piece out first.

Try It Yourself 1.11c: Write a function `log_event(event_name, *details, **metadata)` that prints the event name, then each item in `details` on its own line, then each key-value pair in `metadata` on its own line. Call it at least twice with a different number of details and metadata each time, proving it genuinely adapts to whatever is passed in.

Functions as Values

Before `lambda` makes sense, one idea needs to click: in Python, a function is a value, just like a number or a string, and it can be assigned to a variable, stored in a list, or passed to another function:

```

1 def shout(text):
2     return text.upper() + "!"
3
4 my_function = shout # NO parentheses - this assigns the function ITSELF
5 print(my_function("hello")) # HELLO! - calling it through the new name works fine
```

shout with no parentheses refers to the function itself, the way "hello" refers to a string; shout(), with parentheses, actually calls it. This distinction, a function versus the result of calling it, is exactly what makes the next few tools possible: passing a function to another function, the way you'd pass any other value.

Lambda: Small, Anonymous Functions

A lambda is a function written in a single expression, with no name and no def, meant for short, throwaway logic:

```
1 square = lambda x: x ** 2
2 print(square(5)) # 25
3
4 # The equivalent full function, for comparison:
5 def square(x):
6     return x ** 2
```

Read lambda x: x ** 2 as “a function that takes x and returns x ** 2.” Everything after the colon is an expression, automatically returned; there's no def, no name, and no explicit return keyword at all. A lambda can take multiple parameters, exactly like a normal function, separated by commas before the colon:

```
1 add = lambda a, b: a + b
2 print(add(3, 4)) # 7
```

Lambdas are deliberately limited: the body must be a single expression, not a sequence of statements, no loops, no if/elif/else blocks, no multiple lines. Assigning a lambda to a name, the way square and add do above, works, but if you find yourself doing that regularly, a normal def is almost always the better, more readable choice; lambdas earn their keep specifically as short, inline, one-time functions passed directly into something else, which is exactly the pattern the next two sections are built around.

map(): Applying a Function to Every Item

map() applies a function to every item in a collection, producing a new result for each one, without you writing the loop yourself:

```
1 numbers = [1, 2, 3, 4, 5]
2
3 squared = map(lambda x: x ** 2, numbers)
4 print(list(squared)) # [1, 4, 9, 16, 25]
```

map(function, iterable) calls function once per item in iterable, collecting the results; map() itself returns a special, memory-efficient map object, not a list directly, which is why list(...) wraps it here to actually see the results. This is worth comparing directly to the list comprehension you already know from Lesson 1.10, since they solve the exact same problem:

```
1 squared = [x ** 2 for x in numbers] # the comprehension version
2 squared = list(map(lambda x: x ** 2, numbers)) # the map() version - equivalent
```

Both produce identical results. In modern, everyday Python, the comprehension is generally considered more readable and is used more often; map() is still worth recognizing, especially since you'll encounter it in existing code, and it becomes genuinely convenient when you're mapping an already-named function, not a lambda, since map(str.upper, words) needs no lambda wrapper at all.

filter(): Keeping Only What Matches

`filter()` keeps only the items from a collection for which a function returns `True`:

```
1 numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
2
3 evens = filter(lambda x: x % 2 == 0, numbers)
4 print(list(evens)) # [2, 4, 6, 8, 10]
```

`filter(function, iterable)` calls `function` once per item, and keeps only the items where it returned something truthy, discarding the rest; like `map()`, it returns a filter object that needs `list(...)` to actually see as a list. Again, compare directly to the equivalent comprehension:

```
1 evens = [x for x in numbers if x % 2 == 0] # the comprehension
   version
2 evens = list(filter(lambda x: x % 2 == 0, numbers)) # the filter() version
```

The same guidance applies: comprehensions are generally preferred in new code for their readability, but `filter()` and `map()` are common enough in real, existing Python that recognizing them on sight is genuinely worth the small amount of extra syntax they ask for.

Try It Yourself 1.11d: Given `words = ["apple", "kiwi", "banana", "fig", "cherry"]`, use `filter()` with a `lambda` to keep only words longer than 4 characters, then use `map()` on that result to convert them all to uppercase. Print the final list. Then write the same two-step transformation as a single list comprehension, and confirm both approaches give the same result.

Recursion: A Function That Calls Itself

A recursive function is one that calls itself, directly, as part of solving its own problem. It sounds circular, but it's a genuinely powerful way to express problems that are naturally defined in terms of smaller versions of themselves:

```
1 def countdown(n):
2     if n <= 0:
3         print("Liftoff!")
4     else:
5         print(n)
6         countdown(n - 1) # the function calls ITSELF, with a smaller n
7
8 countdown(3)
9 # 3
10 # 2
11 # 1
12 # Liftoff!
```

Every recursive function needs exactly two parts, and both are non-negotiable. The base case is the condition where the function stops calling itself and just returns or does something directly; here, `n <= 0` is the base case, printing "Liftoff!" with no further recursive call. The recursive case is where the function calls itself again, but with an input that's genuinely closer to the base case than before; here, `countdown(n - 1)` calls itself with a smaller number, one step closer to `n <= 0` each time. Without a base case, or with a recursive case that never actually approaches it, the function calls itself forever, which eventually crashes with a `RecursionError` once Python's internal call limit is reached, the recursive equivalent of the infinite `while` loop from Lesson 1.6.

A more typical use of recursion returns a computed value, rather than just printing:

```
1 def factorial(n):
2     if n <= 1:                # base case
3         return 1
4     return n * factorial(n - 1) # recursive case
5
6 print(factorial(5)) # 120
```

Trace through `factorial(5)` carefully, since watching exactly how recursive calls stack up is the key to understanding recursion at all. `factorial(5)` needs `factorial(4)`'s answer before it can compute `5 * factorial(4)`, `factorial(4)` needs `factorial(3)`'s answer first, and so on, all the way down to `factorial(1)`, which is the base case and returns 1 immediately, with no further calls. Once that innermost call finally returns, every waiting call above it resumes, in reverse order, each one completing its own multiplication: `factorial(2)` computes `2 * 1 = 2`, `factorial(3)` computes `3 * 2 = 6`, `factorial(4)` computes `4 * 6 = 24`, and finally `factorial(5)` computes `5 * 24 = 120`. Every call has to wait for the one it triggered before it can finish, which is exactly why the base case is so critical: it's the only thing that ever actually stops the chain and starts it unwinding back to an answer.

Almost anything recursion can do, a loop can also do, often more efficiently, since Python has to keep track of every waiting call in memory until it resumes:

```
1 def factorial_loop(n):
2     result = 1
3     for i in range(1, n + 1):
4         result *= i
5     return result
```

Recursion tends to be genuinely worth reaching for when a problem is naturally defined recursively, in terms of smaller versions of itself, the way navigating nested folders, or nested lists inside lists, or tree-like data naturally is; for straightforward counting or accumulation, exactly like factorial, a loop is usually simpler, easier to follow, and, in Python specifically, faster.

Try It Yourself 1.11e: Write a recursive function `sum_to(n)` that returns the sum of every whole number from 1 to `n` (so `sum_to(4)` is `1 + 2 + 3 + 4 = 10`). Identify, in a comment above your function, exactly what the base case is and exactly what makes the recursive case get closer to it on every call.

Common Mistakes to Avoid

- **Printing inside a function instead of returning.** `def add(a, b): print(a + b)` looks like it works when called directly, but `result = add(3, 4)` leaves `result` holding `None`, since nothing was ever returned.
- **Forgetting code after return never runs.** Once a function hits `return`, it exits immediately; any lines below it, still inside that function, are simply skipped, every time.
- **Assigning to a global variable without the `global` keyword.** This raises `UnboundLocalError`, since any assignment anywhere in a function makes Python treat that name as local for the entire function.
- **Using a mutable value, like a list, as a default parameter.** It's created once, at definition time, and silently shared across every call that relies on it. Default to `None` and create the real value inside the function body instead.
- **Getting the parameter order wrong.** `def f(**kwargs, *args):` is a `SyntaxError`; the required order is always regular parameters, then `*args`, then `**kwargs`.

- **Trying to cram multiple statements into a lambda.** A lambda's body must be a single expression; anything needing an `if`, a loop, or several lines belongs in a full `def` instead.
- **Writing a recursive function with no base case, or one that never actually gets closer to it.** Both produce infinite recursion, ending in a `RecursionError` once Python's call limit is hit.

Lesson 1.11 Project: Flexible Grading Toolkit

Write a program called `grading_toolkit.py` that:

1. Writes a function `average(*scores)` that accepts any number of scores using `*args` and returns their average. Calling it with zero scores should return 0 instead of raising an error.
2. Writes a function `letter_grade(score, curve=0)` with a default parameter, returning "A"/"B"/"C"/"F" based on `score + curve`, using the thresholds from Lesson 1.5's grade example.
3. Writes a function `summarize(**student_info)` that accepts a student's details as `**kwargs` (name, scores, and anything else you choose) and prints a formatted summary line using `.items()`.
4. Uses `filter()` with a lambda to find every score in a class list that's below 60 (failing), and `map()` with a lambda to add 5 curve points to every score in a separate list.
5. Writes a small recursive function `count_passing(scores, threshold=60)` that recursively counts how many scores in a list meet or exceed the threshold (hint: check the first score, then recurse on the rest of the list using slicing from Lesson 1.7).
6. Calls all five functions with sample data and prints clearly labeled results for each.

This project deliberately touches every major idea from this lesson: default parameters, `*args`, `**kwargs`, `lambda`, `map()`, `filter()`, and recursion, in one connected program.

Lesson Summary: Quick Reference

- `def name(params): ... return value` defines a function; calling it with `()` actually runs it
- A local variable only exists inside its function; `global` is required to modify a global one from inside a function
- `param=default` supplies a default; never use a mutable value as one
- `*args` collects extra positional arguments into a tuple; `**kwargs` collects extra keyword arguments into a dict
- `lambda args: expression` writes a short, single-expression function, often passed to `map()` or `filter()`
- Every recursive function needs a base case that stops it, and a recursive case that moves toward that base case

Lesson 1.12: Docstrings & Type Hints

Why This Matters

Python never forces you to declare a variable's type, or to document what a function expects, which is part of what made Phase 1 approachable from Lesson 1.1 onward. But as programs grow past a single file, “what does this function expect, and what does it give back?” becomes a real question other people, and future you, need answered quickly, without reading the whole function body. Docstrings and type hints are how you write that information directly into your code.

Learning Objectives

By the end of this lesson, you will be able to:

- Write a clear docstring for a function, following a consistent, professional convention
- Add type hints to variables, function parameters, and return values, including tuples, `*args/**kwargs`, and nested collections
- Use `Optional` and `Union` for values that could be more than one type
- Explain why type hints are documentation, not something Python enforces at runtime

Writing a Docstring

A docstring is a string, written as the very first line inside a function, describing what it does. Unlike a regular `#` comment, it's an actual, retrievable piece of the function itself:

```
1 def add(a, b):  
2     """Return the sum of a and b."""  
3     return a + b
```

Triple quotes are the convention, even for a one-line docstring like this, since it keeps every docstring in your codebase visually consistent and makes it trivial to expand into multiple lines later without changing the quoting style. A short, single-sentence docstring like this one is enough for a small, obvious function; describe what it does, not how it does it, since the “how” is right there in the code for anyone who needs it.

For a function whose parameters, return value, or behavior genuinely need more explanation, a multi-line docstring following a consistent structure is worth the extra few lines:

```
1 def calculate_discount(price, percentage):  
2     """Calculate the discounted price of an item.  
3  
4     Args:  
5         price: The original price, before any discount.  
6         percentage: The discount percentage, as a whole number (e.g. 20 for 20%).  
7  
8     Returns:  
9         The final price after the discount is applied.  
10    """  
11    return price * (1 - percentage / 100)
```

This shape, a one-line summary, a blank line, then `Args:` and `Returns:` sections, is one of several widely used docstring conventions (this particular one is often called Google style); what matters more than which exact conven-

tion you pick is picking one and staying consistent throughout a project, so every function's documentation reads the same way.

Reading Docstrings Back

A docstring isn't just a comment for someone reading the source file; Python actually stores it, and you can retrieve it programmatically:

```
1 def add(a, b):  
2     """Return the sum of a and b."""  
3     return a + b  
4  
5 print(add.__doc__) # Return the sum of a and b.  
6 help(add) # prints a nicely formatted version, with the function signature
```

`help()` is exactly what displays when you look up documentation for a built-in function or a library you've imported; every properly documented function, including ones you write yourself, works the same way. This is a big part of why docstrings matter more than an equivalent `#` comment placed above the function: comments are only visible to someone reading the source file directly, while a docstring travels with the function itself, visible to anyone using it, including through tools and editors that surface it automatically as you type.

Try It Yourself 1.12a: Write a function `is_prime(n)` with a proper multi-line docstring (summary, Args, Returns) describing what it checks and what it returns, even before you write the actual logic inside it. Then implement the logic, and confirm `help()` shows your docstring correctly.

Basic Type Hints

A type hint is written after a colon, naming the expected type:

```
1 name: str = "Ada"  
2 age: int = 28  
3 price: float = 19.99  
4 is_member: bool = True
```

This looks like it should restrict what you can store in `name`, but it doesn't; Python still lets you write `name = 42` right after this line, with no error at all, even though the hint says `str`. Try it for yourself and it's genuinely a little surprising the first time: the hint is recorded somewhere Python could theoretically check, but the language deliberately chooses not to look at it while your program actually runs. Type hints are documentation with a precise, checkable format, not a rule the language enforces while your program runs. Their real value comes from external tools: your editor uses them to warn you about likely mistakes as you type, and a separate tool called a type checker (such as `mypy`, outside the scope of this course) can scan your whole project and flag places where the hints and the actual code disagree, all before you ever run the program.

Type Hints on Functions

This is where type hints earn their keep the most: a function signature with hints tells you everything you need to call it correctly, without reading its body at all:

```
1 def greet(name: str, age: int) -> str:  
2     return f"{name} is {age} years old"  
3  
4 print(greet("Ada", 28))
```

Read `def greet(name: str, age: int) -> str`: as: “greet takes a `str` called `name` and an `int` called `age`, and returns a `str`.” The `-> str` after the parentheses is the return type hint; it documents what return will hand back, exactly the way the parameter hints document what goes in. Combined with a docstring, a fully documented function tells you almost everything you need without opening its body:

```
1 def calculate_average(scores: list) -> float:
2     """Return the average of a list of numeric scores."""
3     return sum(scores) / len(scores)
```

Any type you’ve already met can be used as a hint: `str`, `int`, `float`, `bool`, `list`, `dict`, `tuple`, `set`, the exact same names from earlier in this phase.

optional and union: When a Value Could Be More Than One Type

Sometimes a value is genuinely allowed to be one of several types, most commonly, “this normal type, or `None`.” Python’s typing module provides names for exactly this:

```
1 from typing import Optional
2
3 def find_user(user_id: int) -> Optional[dict]:
4     if user_id == 1:
5         return {"id": 1, "name": "Ada"}
6     return None
```

`Optional[dict]` means “a `dict`, or `None`”, a direct, precise way to document a pattern you’ve already used constantly since Lesson 1.9’s `.get()` method, which also returns “the real value, or `None`.” Seeing `Optional[dict]` in a function signature tells you, before calling it, that you need to handle a possible `None` result, perhaps with `if result is None`: from Lesson 1.5.

```
1 from typing import Union
2
3 def format_id(value: Union[int, str]) -> str:
4     return f"ID-{value}"
```

`Union[int, str]` means “either an `int` or a `str`, both are acceptable”, useful when a function is deliberately written to handle more than one type. Modern Python (3.10 and newer) also allows a shorter `|` syntax for the same idea: `int | str` instead of `Union[int, str]`, and `dict | None` instead of `Optional[dict]`; you’ll see both styles in real code, and it’s worth being able to read either.

Type Hints on Collections

Beyond just “this is a list,” you can specify what’s inside it:

```
1 def get_names(users: list[dict]) -> list[str]:
2     return [user["name"] for user in users]
3
4 scores: dict[str, int] = {"Ada": 92, "Alan": 88}
```

`list[dict]` means “a list where every item is a `dict`”; `dict[str, int]` means “a `dict` where every key is a `str` and every value is an `int`.” This lowercase form works directly in Python 3.9 and later; older code often uses

`List[dict]` and `Dict[str, int]` instead, capitalized and imported from `typing`, which still works everywhere but is no longer necessary in modern Python.

Tuples get their own, slightly different syntax, because a tuple’s whole point, from Lesson 1.8, is often that each position holds a specific, different type:

```
1 point: tuple[int, int] = (3, 7) # exactly two ints
2 record: tuple[str, int, float] = ("Ada", 28, 5.6) # three specific types, in order
3
4 def get_bounds(numbers: list[int]) -> tuple[int, int]:
5     return min(numbers), max(numbers)
```

Each position inside `tuple[...]` names that exact position’s type, in order, which is precisely why this differs from `list[int]`: a list’s hint describes every item, uniformly, because a list can be any length; a tuple’s hint describes each slot individually, because a tuple’s length and shape are usually fixed and meaningful, exactly as `get_bounds` returns a two-item tuple every time. For a tuple that genuinely holds a variable number of same-typed items, a rarer case, `tuple[int, ...]` (with a literal ellipsis) means “any number of ints.”

Hinting `*args` and `**kwargs`

Recall `*args` and `**kwargs` from Lesson 1.11: they collect extra arguments into a tuple and a dictionary, respectively, which means they can be hinted too, but the hint describes each individual item they’ll collect, not the tuple or dictionary itself:

```
1 def total(*args: int) -> int:
2     return sum(args)
3
4 def build_profile(**kwargs: str) -> dict:
5     return dict(kwargs)
6
7 print(total(1, 2, 3)) # every arg must be an int
8 print(build_profile(name="Ada", city="London")) # every value must be a str
```

`*args: int` means “every individual positional argument collected must be an `int`”, not “`args` is a list of `int`” written strangely; Python already knows `args` itself will be a tuple, so the hint only needs to describe what’s inside it. `**kwargs: str` works the same way for keyword arguments: every value passed in must be a `str` (the keys are always strings already, by definition, so only the value type needs hinting).

Nested and Complex Type Hints

Real data is often layered several levels deep, and type hints nest exactly the way the data does:

```
1 def group_by_city(people: list[dict[str, str]]) -> dict[str, list[str]]:
2     groups: dict[str, list[str]] = {}
3     for person in people:
4         city = person["city"]
5         if city not in groups:
6             groups[city] = []
7         groups[city].append(person["name"])
8     return groups
```

Read `list[dict[str, str]]` from the outside in, exactly the way you’d read the actual data structure: “a list, of dictionaries, where each dictionary maps `str` keys to `str` values.” The return hint, `dict[str, list[str]]`,

reads the same way: “a dictionary mapping `str` keys to lists of `str`.” This is precisely the kind of shape you’ll hint constantly once you’re working with real JSON data in the very next lesson, where a list of records, each one a dictionary, is one of the most common shapes there is.

When a value’s type is genuinely too open-ended to name precisely, or you haven’t decided yet, `Any` (also from `typing`) is an explicit escape hatch:

```
1 from typing import Any
2
3 def process(data: Any) -> None:
4     print(data)
```

`Any` tells both the reader and any type-checking tool “this could genuinely be anything; no promise is being made here.” It’s honest, but it also opts that value out of the safety type hints are meant to provide, so it’s worth using sparingly and deliberately, not as a default way to avoid figuring out the real type.

For a type that’s long or used repeatedly, a type alias gives it a short, readable name:

```
1 UserRecord = dict[str, str | int]
2
3 def save_user(user: UserRecord) -> None:
4     print(user)
```

`UserRecord` is now just a name for that exact type; using it in a hint means exactly the same thing as writing out `dict[str, str | int]` directly, but it reads far more clearly once a shape like this appears in several function signatures across a file.

Try It Yourself 1.12a-ii: Write a function `summarize_scores(*scores: int) -> tuple[int, int, float]` that returns the minimum, maximum, and average of any number of scores passed in. Call it with several different numbers of arguments and print the result each time.

What an Editor Actually Catches

It’s worth seeing concretely what “your editor warns you” looks like in practice, since it’s the single biggest everyday payoff of adding hints at all. Given this hinted function:

```
1 def calculate_total(price: float, quantity: int) -> float:
2     return price * quantity
```

If you then write `calculate_total("19.99", 3)` elsewhere in the same file, an editor like VS Code will typically underline `"19.99"` before you ever run the program, because it can see the hint says `float` and the argument you actually passed is a `str`. This is the entire point: the mistake gets caught while you’re still typing, rather than surfacing as a confusing traceback later, or worse, silently producing wrong output if the mismatched types happen not to crash outright.

Try It Yourself 1.12b: Take the `grading_toolkit.py` functions you wrote for the Lesson 1.11 project, and add complete type hints and proper docstrings to each one. Nothing about their behavior should change; you’re purely documenting what already works.

Common Mistakes to Avoid

- **Believing type hints are enforced.** `def greet(name: str):` does not stop someone from calling `greet(42)`. Python runs it anyway. Hints are documentation, checked only by external tools, not by Python itself at runtime.
- **Forgetting to import from typing.** `Optional` and `Union` aren't built in like `str` or `int`; you need `from typing import Optional, Union` first.
- **Using `Optional[X]` for a parameter that isn't actually allowed to be `None`.** Reserve it for genuinely optional values; overusing it makes the hints less trustworthy.
- **Hinting the wrong return type.** If a function sometimes returns early with `None`, the return hint must say so (`Optional[dict]`), not just `dict`.
- **Writing a docstring that restates the code instead of explaining intent.** `"""Adds a and b."""` above `return a + b` adds nothing a reader couldn't already see; a good docstring earns its place by explaining something the code alone doesn't make obvious.

Lesson 1.12 Project: Type-Hinted Toolkit

Rewrite the following three functions, giving each a complete, accurate type hint on every parameter and the return value, plus a proper multi-line docstring:

1. `is_prime(n)`: hint the parameter and the `bool` return.
2. `find_student(name, students)`: takes a `str` and a `list[dict]`, returns `Optional[dict]`, since a name might not be found.
3. `calculate_average(scores)`: takes a `list[int]`, returns a `float`.

Then write a short `main()` function that calls all three with valid sample data, printing the results, proving the functions still work exactly as before; type hints and docstrings should never change what a function actually does.

Lesson Summary: Quick Reference

- A docstring is a triple-quoted string as the first line of a function, retrievable with `.__doc__` or `help()`
- `def f(x: int) -> str:` hints a parameter and a return value; `name: str = "Ada"` hints a variable
- Type hints are documentation only; Python does not enforce them at runtime
- `Optional[X]` means “X, or None”; `Union[X, Y]` (or `X | Y`) means “X or Y”
- `list[dict]` hints every item uniformly; `tuple[int, str]` hints each position individually
- `*args: int` and `**kwargs: str` hint the individual items collected, not the tuple/dict itself
- Any opts a value out of type checking; use sparingly. A type alias gives a long, repeated type a short name

Lesson 1.13: File Handling & JSON

Why This Matters

Every program you've written so far forgets everything the moment it stops running. Real programs need to persist data: save a user's settings, write out a report, read in data someone else produced, and very often talk to the outside world in a format other programs understand. This lesson is where your programs start remembering things between runs, start reliably finding the right file to remember them in, no matter which computer they're running on, and start speaking JSON, the format nearly every web API, configuration file, and app-to-app data exchange uses.

Learning Objectives

By the end of this lesson, you will be able to:

- Open, read, and write a text file safely, using the pattern that automatically closes it
- Convert between Python objects and JSON text, in both directions, and read and write JSON files directly
- Navigate nested JSON, and handle malformed JSON without crashing
- Build and inspect file paths with `pathlib`, correctly on any operating system
- Check whether a file or folder exists, create folders, and list a directory's contents

Opening a File: the `with` Statement

Python opens a file with the built-in `open()` function, and the standard, correct way to do it is inside a `with` block:

```
1 with open("notes.txt", "r") as f:
2     content = f.read()
3     print(content)
```

`open("notes.txt", "r")` opens `notes.txt` in read mode ("r"); as `f` names the open file `f` for the rest of the block. Everything indented under the `with` has the file open and ready to use; the moment that block ends, whether normally or because of an error partway through, Python automatically closes the file for you. This matters more than it might seem: a file left open can lose unsaved data, lock other programs out of it, or quietly leak resources in a long-running program. Writing `open()` without `with` means remembering to call `f.close()` yourself, every single time, including on every path where an error might occur, which is exactly the kind of easy-to-forget detail `with` removes entirely.

The second argument to `open()` is the mode, and picking the right one matters:

Mode	Meaning
"r"	Read (default); error if the file doesn't exist
"w"	Write; creates the file if missing, erases existing content
"a"	Append; creates the file if missing, adds to the end

"w" deserves a specific warning: opening a file in write mode immediately erases whatever was already in it, even before you write anything new, simply by opening it that way. If you want to add to a file without destroying what's already there, "a" (append) is almost always what you actually want.

Reading a File

Three methods cover the ways you'll actually want to read a file's contents:

```
1 with open("notes.txt", "r") as f:
2     whole_thing = f.read() # the ENTIRE file, as one string
3
4 with open("notes.txt", "r") as f:
5     first_line = f.readline() # just ONE line, including its newline
6
7 with open("notes.txt", "r") as f:
8     all_lines = f.readlines() # a LIST of every line, each including its newline
```

`.read()` is right for small files you want to treat as one block of text. `.readline()` reads exactly one line, advancing to the next line each time you call it again, useful when you genuinely need to process a file one line at a time with fine control. `.readlines()` reads everything at once, but returns a list of individual lines rather than one combined string, which is convenient when you specifically want that list shape. In practice, though, the most common and most idiomatic way to process a file line by line is simpler than any of these three: loop over the file object directly, exactly the way you looped over a list back in Lesson 1.6:

```
1 with open("notes.txt", "r") as f:
2     for line in f:
3         print(line.strip()) # .strip() removes the trailing newline
```

This reads one line at a time, automatically, without loading the entire file into memory at once, which becomes genuinely important once a file is large; `.read()` on a multi-gigabyte log file tries to load the whole thing into memory in one go, while looping over the file directly never holds more than one line at a time. Every line read this way includes its trailing newline character, which is why `.strip()` (from Lesson 1.7) is almost always applied immediately, to get just the text content without it.

Try It Yourself 1.13a: Create a text file called `shopping.txt` by hand (or with code), with one item per line. Write a program that opens it, loops over it directly, and prints each item with a number in front of it, using `enumerate()` from Lesson 1.8.

Writing to a File

Writing follows the same with pattern, using "w" or "a" mode and `.write()`:

```
1 with open("output.txt", "w") as f:
2     f.write("First line\n")
3     f.write("Second line\n")
```

Unlike `print()`, `.write()` does not add a newline automatically; you have to include `\n` yourself, exactly as shown, or every line you write will run together into one. Writing several lines at once from a list uses `.writelines()`, which, despite the name, also does not add newlines for you:

```
1 lines = ["First line\n", "Second line\n", "Third line\n"]
2
3 with open("output.txt", "w") as f:
4     f.writelines(lines)
```

Each string in `lines` already ends with `\n` here, which is why they don't run together; `.writelines()` is really just a shortcut for calling `.write()` once per item, nothing more. To add to a file without erasing what's already there, open it in append mode instead:

```

1 with open("log.txt", "a") as f:
2     f.write("New entry added.\n")

```

Running this once creates `log.txt` with one line; running the exact same code again adds a second line underneath, rather than replacing the file, which is precisely the behavior a running log needs.

What JSON Actually Looks Like

JSON (JavaScript Object Notation) is the format the internet runs on; nearly every web API, configuration file, and app-to-app data exchange uses it. It looks almost exactly like a Python dictionary, which is not a coincidence: JSON's structure was deliberately designed to map cleanly onto the same key-value idea from Lesson 1.9.

```

1 {
2     "name": "Ada",
3     "age": 28,
4     "is_member": true,
5     "skills": ["Python", "SQL"],
6     "manager": null
7 }

```

If you squint, this is almost identical to a Python dictionary; `true/false/null` are JSON's spellings of Python's `True/False/None`, and everything else looks the same. That similarity is exactly why converting between the two is so simple in Python: every JSON type has one, and only one, obvious Python equivalent, and vice versa.

object	array	string	int-like	float-like	bool/null
<code>{}</code>	<code>[]</code>	<code>"..."</code>	<code>123</code>	<code>1.5</code>	<code>true/null</code>
<code>dict</code>	<code>list</code>	<code>str</code>	<code>int</code>	<code>float</code>	<code>bool/None</code>

This table is worth knowing by heart, because it also tells you what JSON can't represent: there's no JSON equivalent for a Python tuple, set, or custom object, which matters the moment you try to convert one. JSON earned its place as the internet's default data format for a simple reason: it's plain text, so it travels over any network connection or saves into any file, same as anything from earlier in this lesson; it's human-readable, unlike some binary formats; and, as this table shows, it maps directly onto data structures every major programming language already has, not just Python's. That's what makes it the common language a Python program, a JavaScript web page, and a mobile app can all read and write without any special translation step between them.

From Python to JSON Text: `json.dumps()`

The `json` module is part of the standard library; no installation needed, just `import json`. `json.dumps()` ("dump string") converts a Python object into a JSON-formatted string:

```

1 import json
2
3 person = {
4     "name": "Ada",
5     "age": 28,
6     "is_member": True,
7     "skills": ["Python", "SQL"],
8 }
9
10 json_text = json.dumps(person)

```

```

11 print(json_text)
12 print(type(json_text))
13 # {"name": "Ada", "age": 28, "is_member": true, ...}
14 # <class 'str'>

```

Notice `True` became `true` and the quotes changed style; `json.dumps()` handles every one of these translation rules for you. The result is a plain string, not a dictionary anymore; you could email it, print it, or send it over a network exactly as is. For anything you intend a human to actually read, pass `indent=`, which pretty-prints the result across multiple lines:

```

1 print(json.dumps(person, indent=2))
2 # {
3 #   "name": "Ada",
4 #   "age": 28,
5 #   "is_member": true,
6 #   "skills": [
7 #     "Python",
8 #     "SQL"
9 #   ]
10 # }

```

From JSON Text to Python: `json.loads()`

`loads()` (“load string”) does the reverse: it parses a JSON-formatted string back into real Python objects:

```

1 json_text = '{"name": "Ada", "age": 28, "skills": ["Python", "SQL"]}'
2
3 person = json.loads(json_text)
4 print(person)           # {'name': 'Ada', 'age': 28, ...}
5 print(type(person))    # <class 'dict'>
6 print(person["name"])  # Ada

```

Once `loads()` hands you back a dictionary, everything from Lesson 1.9 applies directly: indexing with `["name"]`, `.get()`, looping with `.items()`. This is the entire point of JSON support: it turns text from the outside world into ordinary Python data you already know how to work with.

Reading and Writing JSON Files

Real programs usually read JSON from a file rather than a hardcoded string. Because JSON is just text, this combines directly with the `with open(...)` pattern from earlier in this lesson. `json.load()` (no “s”) reads directly from an already-open file object; `json.dump()` writes directly to one:

```

1 data = {"name": "Ada", "age": 28}
2
3 with open("person.json", "w") as f:
4     json.dump(data, f, indent=2) # writes to the file
5
6 with open("person.json", "r") as f:
7     loaded = json.load(f) # reads from the file
8
9 print(loaded) # {'name': 'Ada', 'age': 28}

```

The naming is consistent once you notice the pattern: the versions ending in `s` (`dumps`, `loads`) work with strings, and the trailing `s` is worth reading as short for “string” to make the pattern stick; the versions without the `s` (`dump`, `load`) work directly with an open file, the exact file object with `open(...)` as `f` already hands you. Mixing them up is a common early mistake: `json.dump(data, "file.json")` fails, because the second argument needs to be an open file object from `open(...)`, not a filename; Python doesn’t open the file for you, since `dump()` expects something it can call `.write()` on directly, which a plain filename string can’t do.

Working with Nested JSON

Real JSON is rarely flat; APIs commonly return dictionaries containing lists of dictionaries, several levels deep. Once parsed, it’s just nested Python data, navigable exactly like the nested dictionaries and lists from Lesson 1.9:

```

1 data_text = '''
2 {
3     "class": "Software Engineering",
4     "students": [
5         {"name": "Ada", "grade": 92},
6         {"name": "Alan", "grade": 88}
7     ]
8 }
9 '''
10
11 data = json.loads(data_text)
12 print(data["class"])           # Software Engineering
13 print(data["students"][0]["name"]) # Ada
14 print(len(data["students"]))   # 2
15
16 for student in data["students"]:
17     print(f"{student['name']}: {student['grade']}")

```

`data["students"]` is a list; `data["students"][0]` is the first dictionary inside that list; `["name"]` pulls a value out of that dictionary; each step is just one more layer of indexing, read left to right.

Try It Yourself 1.13b: Build a Python dictionary representing a small music playlist: a name, and a list of songs, where each song is its own dictionary with `title`, `artist`, and `duration_seconds`. Convert it to a JSON string with `indent=2` and print it, then write it to a file called `playlist.json`, then read that file back in and print the title of the second song.

Handling Malformed JSON

Any time JSON text comes from outside your own program, a user, a file that might be corrupted, a network response, you have to assume it might not actually be valid JSON. `json.loads()` doesn’t fail silently or guess; it raises `json.JSONDecodeError`, a specific exception you’ll learn to catch properly with `try/except` in Lesson 1.14:

```

1 bad_text = '{"name": "Ada", "age": }' # missing a value - invalid JSON
2
3 try:
4     data = json.loads(bad_text)
5 except json.JSONDecodeError as e:
6     print(f"Could not parse JSON: {e}")
7     data = {} # fall back to a safe default

```

This is the same “guard the risky operation, decide what happens on failure” idea you’ve seen already with a dictionary’s `.get()` in Lesson 1.9, applied here to parsing instead of lookup. Wrapping every `json.loads()` call that

touches outside data in a try/except is considered standard, defensive practice; a single malformed file shouldn't be able to crash your entire program.

Absolute vs. Relative Paths

Every example so far has opened a file by typing a plain filename like "notes.txt", which quietly relies on the file living in your current folder. Real programs need to be more deliberate. A relative path ("notes.txt", "data/scores.json") is interpreted starting from wherever your program is currently running, the same "current folder" concept from the terminal work in Lesson 1.2. An absolute path (/Users/ada/projects/notes.txt on macOS/Linux, C:\Users\ada\projects\notes.txt on Windows) points to one exact, unambiguous location on the computer, regardless of where the program was started from.

Relative paths are convenient and portable; they work regardless of where your project folder happens to sit on someone else's computer. Absolute paths are unambiguous but tied to one specific machine's folder layout. Most of the time, especially for files that live alongside your code, relative paths are what you want. The next section, pathlib, is exactly the tool Python gives you for building and working with both kinds of paths correctly, on any operating system.

Building Paths with pathlib.Path

Older code builds paths by gluing strings together (folder + "/" + filename), which breaks the moment someone runs it on Windows, where paths use backslashes instead of forward slashes. Python's pathlib module solves this once and for all:

```
1 from pathlib import Path
2
3 p = Path("data") / "scores.json"
4 print(p)           # data/scores.json (or data\scores.json on Windows)
5 print(type(p))     # <class 'pathlib.PosixPath'> (or WindowsPath)
```

The / here is not division; pathlib deliberately overloads it to mean "join this piece onto the path," and it automatically uses whichever slash style the current operating system expects. This is the single biggest reason to prefer pathlib over manually building path strings: the same code produces a correct path on every operating system, without an if statement checking which one you're on.

```
1 p = Path("data") / "reports" / "2026" / "summary.json"
2 print(p) # data/reports/2026/summary.json
```

Each / adds one more level, exactly like building up a folder structure by hand.

Inspecting and Resolving a Path

A Path object carries useful pieces of itself as properties, without you needing to parse the string yourself:

```
1 p = Path("data/reports/summary.json")
2
3 print(p.name)      # summary.json - the final component
4 print(p.stem)      # summary - name without extension
5 print(p.suffix)    # .json - just the extension
6 print(p.parent)    # data/reports - containing folder
```

These come up constantly in practice: `.suffix` to check a file's type before processing it, `.stem` to build a new filename based on an existing one, like turning `summary.json` into `summary_backup.json`. Two methods turn any path into an unambiguous, absolute one, or answer “where am I right now?” directly:

```
1 print(Path.cwd())      # the current working folder, as an absolute path
2 print(Path.home())     # the current user's home folder
3 print(Path("data/scores.json").resolve()) # expanded to a full absolute path
```

`Path.cwd()` (“current working directory”) answers exactly the question the terminal’s `pwd` command answers, but from inside your running program. `.resolve()` takes any relative path and expands it into the full, absolute version, based on the current folder, useful when you want to print or log exactly which file your program touched.

Checking Existence and Creating Folders

```
1 p = Path("notes.txt")
2 print(p.exists())      # True or False
3 print(p.is_file())     # True if it exists AND is a file
4 print(p.is_dir())      # True if it exists AND is a folder
5
6 folder = Path("output")
7 folder.mkdir(exist_ok=True) # creates the folder; ok even if it already exists
```

`exist_ok=True` is worth using by habit; without it, `.mkdir()` raises `FileExistsError` if the folder is already there, which is rarely what you want when you’re just making sure a folder exists before writing into it. If the parent folder doesn’t exist yet either, pass `parents=True` to create the whole chain at once: `Path("a/b/c").mkdir(parents=True, exist_ok=True)`.

Listing a Folder’s Contents

```
1 folder = Path(".") # "." means "the current folder"
2
3 for item in folder.iterdir():
4     print(item)
5
6 json_files = list(folder.glob("*.json"))
7 print(json_files) # every .json file in this folder
8
9 all_json = list(folder.rglob("*.json"))
10 print(all_json) # every .json file anywhere UNDER this folder, including subfolders
```

`.iterdir()` lists everything directly inside a folder, files and subfolders alike, with no filtering. `.glob(pattern)` is more targeted: it uses wildcard patterns to filter as it goes, where `*` means “any sequence of characters”; `folder.glob("*.json")` reads as “everything ending in `.json`.” `.glob()` only looks one level deep by default; `.rglob()` (“recursive glob”) searches every subfolder too, as deep as they go. Use `.glob()` when you specifically want just the current folder; reach for `.rglob()` the moment “anywhere underneath here” is what you actually mean.

Combining `pathlib` With File Handling

Everything from earlier in this lesson works directly with a `Path` object; you can pass one straight to `open()` instead of a plain string:

```
1 log_file = Path("logs") / "app.log"
```

```
2 log_file.parent.mkdir(exist_ok=True) # make sure "logs" exists first
3
4 with open(log_file, "a") as f:
5     f.write("Started successfully.\n")
```

`log_file.parent` gives you the containing folder (`logs`) as its own `Path`, which is exactly what `.mkdir()` needs, a clean example of how these small pieces combine. For simple, whole-file cases, `pathlib` also offers a shortcut that skips `open()`/`with` entirely:

```
1 Path("notes.txt").write_text("Hello, file!\n")
2 content = Path("notes.txt").read_text()
3 print(content)
```

`.write_text()` and `.read_text()` each open the file, do the one operation, and close it again automatically, all in a single call, genuinely convenient for small files where you don't need line-by-line control. For anything requiring append mode, careful line-by-line reading of a large file, or JSON, the full `open()`/`with` pattern from earlier in this lesson is still the right tool.

Try It Yourself 1.13c: Write a program that creates a folder called `archive` if it doesn't exist, then uses `.glob("*.json")` to find every `.json` file in your current folder and print each one's `.name` and `.stem`. Finally, use `.write_text()` to create a small summary file inside `archive` listing how many `.json` files were found.

Common Mistakes to Avoid

- **Opening a file in "w" mode when you meant to keep existing content.** "w" erases the file immediately upon opening, before you write anything new. Use "a" to append instead.
- **Forgetting `.write()` doesn't add a newline.** Every line you want on its own line needs an explicit `\n` at the end.
- **Confusing dumps/loads with dump/load.** The `s` versions work with strings; the plain versions work with an already-open file. Passing a filename where a file object is expected raises an error.
- **Not handling malformed JSON.** `json.loads()` raises `json.JSONDecodeError` on invalid input; wrap it in `try/except` whenever the JSON is coming from an untrusted source.
- **Building paths with plain string concatenation.** `folder + "/" + filename` breaks on Windows. Use `Path(folder) / filename` instead.
- **Calling `.mkdir()` without `exist_ok=True`** and crashing the second time your program runs, once the folder already exists.
- **Using `.glob()` when subfolders should be included too.** `.glob()` only searches one level deep; use `.rglob()` for a recursive search.

Lesson 1.13 Project: Inventory Archive

Write a program called `inventory_archive.py` that:

1. Using `pathlib`, creates a folder called `data` if it doesn't already exist.
2. Builds a Python list of at least six dictionaries, each representing a product with `name`, `category`, `price`, and `quantity`.
3. Writes that list to `data/inventory.json` using `json.dump()` with `indent=2`.

4. Reads `data/inventory.json` back in with `json.load()`, and computes the total value of the inventory (summing `price * quantity` across every item).
5. Opens `data/archive_log.txt` in append mode and writes one line recording how many items were archived and the total value just computed, so the log grows by one line every time the program runs.
6. Finally, uses `.glob("*.json")` on the data folder to confirm `inventory.json` is there, and prints its `.name` and `.stem`.

This project deliberately combines every idea from this lesson: file handling, JSON in both directions, and `pathlib`, in one connected program.

Lesson Summary: Quick Reference

- `with open(path, mode) as f:` opens a file and closes it automatically; modes are `"r"`, `"w"` (erases first), `"a"` (appends)
- Loop over a file object directly to read it line by line without loading it all into memory
- JSON objects `{}` map to `dict`; JSON arrays `[]` map to `list`
- `json.dumps(obj)` / `json.loads(text)` convert to/from a JSON string; `json.dump(obj, f)` / `json.load(f)` read/write directly to an open file
- Wrap `json.loads()` in `try/except json.JSONDecodeError` for untrusted input
- `Path("a") / "b"` builds a path correctly on any operating system; `.name` `.stem` `.suffix` `.parent` inspect it
- `.iterdir()` lists everything; `.glob()` filters one level; `.rglob()` filters recursively

Lesson 1.14: Error Handling (Try/Except)

Why This Matters

You've already met plenty of ways a Python program can crash: a `KeyError` from a missing dictionary key, a `ValueError` from converting bad text with `int()`, a `json.JSONDecodeError` from malformed JSON. Up to now, every one of those has simply stopped your program cold. Real programs can't afford that: a file might not exist, a user might type nonsense, a network request might fail. Error handling is how you let a program anticipate what might go wrong and recover, instead of crashing the instant something does.

Learning Objectives

By the end of this lesson, you will be able to:

- Catch an exception with `try/except` instead of letting your program crash
- Catch specific exception types separately, and explain why that's better than catching everything
- Use `else` and `finally` to run code based on whether an error happened
- Raise your own exceptions with `raise`, including custom exception classes

What an Exception Actually Is

Every crash you've seen so far, `TypeError`, `ValueError`, `KeyError`, `IndexError`, `FileNotFoundError`, is Python raising an exception: a signal that something went wrong, which, left unhandled, propagates all the way up and stops your program, printing a traceback. Exceptions aren't a flaw in Python; they're the language's standard, structured way of reporting that an operation couldn't complete as asked.

```
1 age = int("not a number")
2 print("This line never runs.")
```

Once `int("not a number")` raises `ValueError`, Python abandons the rest of the program immediately; the `print()` line never executes, and the interpreter exits, printing a traceback that shows exactly where and why. That traceback is genuinely useful for debugging, but in a program meant to keep running (a script processing a hundred files, a program waiting on user input), letting one bad value take down everything is rarely what you want.

Common Exception Types

Python organizes errors into distinct, named types, and you've already met most of the common ones across earlier lessons without necessarily pausing on them as a group. Knowing which type of exception a given mistake actually raises is what lets you catch precisely the right one, rather than guessing, which is exactly what the rest of this lesson depends on.

Exception	Raised when
ValueError	A value has the right type but an inappropriate value, like <code>int("abc")</code> (Lesson 1.3)
TypeError	An operation is used on a type it doesn't support, like <code>"5" + 5</code> (Lesson 1.3)
KeyError	A dictionary lookup uses a key that doesn't exist, like <code>person["email"]</code> (Lesson 1.9)
IndexError	A list or string index is out of range, like <code>fruits[10]</code> on a shorter list (Lesson 1.8)
ZeroDivisionError	Dividing by zero with <code>/</code> or <code>//</code>
FileNotFoundError	Opening a file in <code>"r"</code> mode that doesn't exist (Lesson 1.13)
AttributeError	Calling a method or accessing an attribute that a value's type doesn't have, like <code>"text".append()</code>
NameError	Using a variable that was never defined, or is out of scope (Lesson 1.11)
<code>json.JSONDecodeError</code>	Parsing text with <code>json.loads()</code> that isn't valid JSON (Lesson 1.13)
ModuleNotFoundError	Importing a module that isn't installed or doesn't exist (Lesson 1.15)

A few of these are worth a closer look, since the distinction between them is a genuinely common source of confusion. `ValueError` versus `TypeError` trips up nearly everyone at first: both sound like “you gave me the wrong thing,” but the difference is precise. `TypeError` means the type itself is wrong for the operation, like trying to add a string and an integer directly; no value of that type would have worked. `ValueError` means the type is perfectly fine, but this particular value isn't, like `int("abc")`, where a string is exactly what `int()` expects, just not that specific string.

```

1 "5" + 5          # TypeError - str and int can never be added directly
2 int("abc")       # ValueError - int() accepts strings, just not this one

```

`KeyError` versus `IndexError` is a similar pair, distinguished by which collection raised it: `KeyError` always comes from a dictionary lookup with a missing key; `IndexError` always comes from a list, tuple, or string position that doesn't exist. Since both are, at heart, “you asked for something that isn't there,” it's easy to reach for the wrong one when guessing which to catch, so it's worth remembering the rule by which collection is involved, not by the general shape of the mistake.

Every one of these exception types, and every exception in Python generally, ultimately inherits from a common base class called `Exception`, which is precisely why `except Exception:` catches literally anything, the point raised again shortly in this lesson.

The try/except Statement

`try` lets you attempt something that might fail, and `except` catches the failure if it happens, letting your program continue instead of crashing:

```

1 try:
2     age = int(input("Enter your age: "))
3     print(f"Next year you'll be {age + 1}.")
4 except ValueError:
5     print("That doesn't look like a valid number.")
6
7 print("The program keeps running either way.")

```

Python runs everything inside the `try` block normally. If every line succeeds, the `except` block is skipped entirely, and execution continues after it, as if the `try` had never been there. If a `ValueError` is raised anywhere inside the `try` block, though, Python immediately stops running the rest of that block, at the exact line where the error occurred, meaning any code still remaining in the `try` block after that point, like the `print(f"Next year...")` line here if the conversion itself failed, never executes at all; Python jumps straight to the matching `except`, runs it, and then continues with whatever comes after; the crash never happens, and “The program keeps running either way.” always prints, regardless of which path was taken.

The exact same indentation rules from Lesson 1.5 apply here: everything indented under `try` is protected; everything indented under `except` only runs if that specific error occurred.

Catching Specific Exceptions

`except ValueError:` only catches a `ValueError`; a different kind of error inside the same `try` block would not be caught by it, and would still crash the program. You can catch several different exception types with separate `except` clauses, each handled its own way:

```
1 data = {"scores": [85, 92, 78]}
2
3 try:
4     index = int(input("Which score number do you want? "))
5     print(data["scores"][index])
6 except ValueError:
7     print("Please enter a whole number.")
8 except IndexError:
9     print("That position doesn't exist in the list.")
10 except KeyError:
11     print("That key doesn't exist in the dictionary.")
```

Python checks each `except` clause in order, top to bottom, exactly like an `elif` chain, and runs the first one whose exception type matches what was actually raised. A single input like `"abc"` triggers `ValueError` (from `int()`); a valid number that’s too large, like 99, triggers `IndexError` (from the list lookup) instead. Being specific about which exceptions you catch, rather than catching everything indiscriminately, matters for a concrete reason: it lets each kind of failure get a response that actually makes sense for it, instead of one generic, unhelpful message covering every possible problem.

If you genuinely need to handle several exception types the exact same way, group them in one clause with a tuple:

```
1 try:
2     result = 10 / int(input("Enter a divisor: "))
3 except (ValueError, ZeroDivisionError):
4     print("Please enter a non-zero number.")
```

This catches either a `ValueError` (bad text) or a `ZeroDivisionError` (dividing by zero) with the same response, without duplicating the handling code for both.

Try It Yourself 1.14a: Write a program that asks the user for two numbers using `input()` and divides the first by the second. Wrap it in `try/except` to handle both a `ValueError` (if either input isn’t a number) and a `ZeroDivisionError` (if the second number is 0), printing a different, specific message for each.

Accessing the Exception Itself

Sometimes the generic fact that “something went wrong” isn’t enough; you want the actual error message Python generated. Adding `as` after the exception type gives you a variable holding the exception object itself:

```
1 try:
2     age = int("twenty-five")
3 except ValueError as e:
4     print(f"Conversion failed: {e}")
5     # Conversion failed: invalid literal for int() with base 10: 'twenty-five'
```

`e` here holds the actual exception Python raised, and converting it to a string (which an f-string does automatically) gives you Python's own description of exactly what went wrong. This is genuinely useful while developing and debugging, and often worth logging even in a finished program, rather than only showing the user a generic message.

Catching Any Exception (and Why You Usually Shouldn't)

Python lets you catch every possible exception with a bare `except:`, or more explicitly with `except Exception:`, but this is a genuine trap for beginners, worth understanding before you're tempted to reach for it:

```
1 try:
2     result = risky_calculation()
3 except Exception:
4     print("Something went wrong.")
```

This looks convenient, catch everything, never crash, but it silently swallows every kind of error equally: a missing file, a typo in a variable name, a genuine bug in your own logic, all get the exact same unhelpful "Something went wrong." message, and the real cause is hidden rather than fixed. A bare `except:` is even broader, catching things like the user pressing `Ctrl+C` to intentionally stop the program, which you almost never want to intercept. As a rule, catch the specific exception types you actually expect and know how to handle; let anything else propagate and crash loudly, since a loud crash during development is far more useful than a silent, generic failure that hides a real bug.

else and finally

Two optional clauses round out `try/except`, each answering a different question about when code should run. `else` runs only if the `try` block completed with no exception at all:

```
1 try:
2     number = int(input("Enter a number: "))
3 except ValueError:
4     print("That wasn't a valid number.")
5 else:
6     print(f"Great, you entered {number}.")
7     print(f"Its square is {number ** 2}.")
```

Putting the success-path code in `else`, rather than just adding it to the end of the `try` block, makes the intent explicit: this code assumes nothing in the `try` block failed, and separating it clearly marks where the "risky" part ends and the "safe, only-on-success" part begins.

`finally` runs no matter what happens, whether the `try` succeeded, an exception was caught, or even if an exception was raised and not caught at all:

```
1 try:
2     file = open("data.txt", "r")
3     content = file.read()
4 except FileNotFoundError:
5     print("File not found.")
```

```

6     content = ""
7 finally:
8     print("Cleanup: this always runs.")

```

`finally` is built for cleanup work that has to happen regardless of outcome, closing a file, releasing a resource, logging that an attempt was made. In practice, for files specifically, the `with` statement from Lesson 1.13 already handles this kind of cleanup automatically, which is exactly why `with open(...) as f:` is preferred over manually pairing `open()` with a `finally` block; `finally` is more general-purpose, useful any time you need guaranteed cleanup that isn't already handled by a `with` block.

All four pieces, `try`, `except`, `else`, and `finally`, can appear together in that exact order:

```

1 try:
2     number = int(input("Enter a number: "))
3 except ValueError:
4     print("Invalid input.")
5 else:
6     print(f"You entered {number}.")
7 finally:
8     print("Done processing this attempt.")

```

Try It Yourself 1.14b: Write a program that tries to open a file called `config.txt` for reading. If it doesn't exist (catch `FileNotFoundError`), print a message saying so. If it does open successfully, use `else` to print its contents. Either way, use `finally` to print "Attempt finished."

Raising Your Own Exceptions

So far, every exception has come from Python itself or the standard library. Your own functions can raise exceptions too, using `raise`, whenever a caller does something your function genuinely can't work with:

```

1 def set_age(age):
2     if age < 0:
3         raise ValueError("Age cannot be negative.")
4     print(f"Age set to {age}.")
5
6 set_age(25)    # Age set to 25.
7 set_age(-5)   # raises ValueError: Age cannot be negative.

```

`raise ValueError(...)` creates a new `ValueError` with your own custom message and raises it immediately, exactly as if Python itself had raised it; anyone calling `set_age()` can catch it with `except ValueError:` just like any built-in one. Raising a specific, well-named exception type with a clear message, rather than letting a function silently do the wrong thing with bad input, is how you build functions other people (including future you) can trust and debug quickly.

You aren't limited to Python's built-in exception types. Defining your own, for errors specific to your own program, is straightforward, and is a genuinely common pattern once programs grow past a single file:

```

1 class InsufficientFundsError(Exception):
2     """Raised when a withdrawal exceeds the available balance."""
3     pass
4
5 def withdraw(balance, amount):

```

```
6     if amount > balance:
7         raise InsufficientFundsError(
8             f"Cannot withdraw {amount}; balance is only {balance}."
9         )
10    return balance - amount
11
12    try:
13        withdraw(100, 150)
14    except InsufficientFundsError as e:
15        print(f"Transaction failed: {e}")
```

A custom exception is just a class (classes are covered fully in Phase 2) that inherits from Python's built-in `Exception`; for now, treat this as a template you can copy: naming your own exception clearly, like `InsufficientFundsError`, makes calling code able to catch precisely that failure, and nothing else, which is considerably clearer than raising a generic `ValueError` for every possible thing that could go wrong in a larger program.

Try It Yourself 1.14c: Write a function `validate_email(email)` that raises a `ValueError` with a clear message if the text doesn't contain an "@" symbol (use the `in` keyword from Lesson 1.7 to check), and otherwise prints that the email looks valid. Call it inside a `try/except` with both a valid and an invalid email, proving both paths work.

Common Mistakes to Avoid

- **Catching Exception (or a bare `except:`) out of convenience.** This silently hides bugs alongside genuine, expected failures. Catch specific exception types you actually anticipate and know how to handle.
- **Putting too much code inside `try`.** Only wrap the specific line or lines that can actually raise the exception you're catching; wrapping an entire function makes it unclear which line was actually risky, and can accidentally swallow errors from unrelated code.
- **Using `try/except` for normal control flow.** Checking `if key in my_dict:` before accessing it (Lesson 1.9) is usually clearer than wrapping the access in `try/except KeyError:` when you can easily check first; reserve exceptions for situations that are genuinely exceptional or awkward to check in advance.
- **Forgetting that `else` requires no exception, not just reaching the end of `try`.** If any exception was raised and caught, `else` does not run, even though the program continues normally afterward.
- **Assuming `finally` means "only runs on success."** It's the opposite: `finally` runs unconditionally, success, caught failure, or even an uncaught one on its way out.

Lesson 1.14 Project: Safe Calculator

Write a program called `safe_calculator.py` that:

1. Defines a custom exception class `InvalidOperationError` (inheriting from `Exception`).
2. Writes a function `calculate(a, b, operation)` that performs `+`, `-`, `*`, or `/` based on a string argument, raising `InvalidOperationError` if `operation` isn't one of those four symbols.
3. In a loop, repeatedly asks the user for two numbers and an operation using `input()`, calling `calculate()` inside a `try` block.
4. Catches `ValueError` (bad number input), `ZeroDivisionError` (dividing by zero), and `InvalidOperationError` (bad operation) separately, each with its own clear message.
5. Uses `else` to print the result only when the calculation genuinely succeeded, and `finally` to print a separator line after every attempt, success or failure.
6. Lets the user type "quit" at any prompt to exit the loop.

Lesson Summary: Quick Reference

- `try: / except ExceptionType:` catches a specific exception instead of crashing
- Catch specific exception types, not a bare `except:` or `except Exception:`, so real bugs stay visible
- `except ExceptionType as e:` gives you the exception object, including its message
- `else` runs only if `try` succeeded with no exception; `finally` always runs, no matter what
- `raise ExceptionType("message")` raises your own exception; custom exception classes inherit from `Exception`

Lesson 1.15: Packages & Imports

Why This Matters

Every program so far has lived in a single file. Real projects don't: they split code across multiple files so that related pieces stay together and unrelated pieces stay apart, and they reuse code other people have already written instead of rebuilding it from scratch. `import` is the single keyword that makes both of those possible, and understanding it precisely, not just well enough to copy an example, is what lets you structure a project of any real size.

Learning Objectives

By the end of this lesson, you will be able to:

- Import an entire module, or specific names from it, using every common form of import
- Explain the difference between Python's standard library and third-party packages
- Split your own code across multiple files and import between them correctly
- Organize a folder of files into a proper package using `__init__.py`

What a Module Actually Is

A module is simply a `.py` file, nothing more; the moment you save any Python file, you've created a module, whether or not you ever intended it as one. When you import a module, Python runs that file once, top to bottom, and gives you a reference to everything it defined, functions, classes, variables, so you can use them from a different file.

```
1 import math
2
3 print(math.sqrt(16))  # 4.0
4 print(math.pi)       # 3.141592653589793
```

`math` here is a module from Python's standard library, the huge collection of modules that ships with every Python installation, no extra download needed, first mentioned back in Lesson 1.1. `import math` makes every name inside it available, but only reachable through `math.`, which is deliberate: it keeps `math`'s names, like `sqrt`, clearly separated from anything named `sqrt` you might define yourself elsewhere in the same file.

The Forms of `import`

Python offers several ways to import, and the differences between them matter for how you'll actually use the imported names.

```
1 import math
2 print(math.sqrt(16))  # must prefix with math.
```

```
1 from math import sqrt
2 print(sqrt(16))  # no prefix needed, just sqrt
```

```
1 from math import sqrt, pi
2 print(sqrt(16), pi)  # import several specific names at once
```

```
1 import math as m
```

```
2 print(m.sqrt(16)) # renamed, common for long module names
```

`import math` brings in the whole module, accessed through `math.name`. `from math import sqrt` brings in just that one name directly, usable without any prefix at all, useful when you only need a handful of specific things and want to avoid typing the module name repeatedly. The trade-off is worth naming directly: `import math` means every call needs the `math.` prefix, `math.sqrt(16)`, `math.pi`, which is slightly more typing but makes it obvious, at every single call site, exactly where that name came from; `from math import sqrt` saves the prefix, `sqrt(16)`, but a reader scanning just that line has no way to tell, without scrolling up to the imports, that `sqrt` isn't a function you wrote yourself. `import math as m` gives the module a shorter local nickname, extremely common with certain well-known third-party packages later in this course, shortening a long, frequently-typed module name into a short, memorable one.

One more form exists, and it's worth knowing specifically so you can recognize why to avoid it:

```
1 from math import * # AVOID - imports every single name from math directly
2 print(sqrt(16))
3 print(pi)
```

The star import pulls in every public name from the module at once, with no prefix. It looks convenient, but it makes it genuinely difficult to tell, just by reading your code, where any given name actually came from, and it risks silently overwriting a name you already defined yourself if the module happens to use the same one. Stick to importing the whole module, or specific named pieces; avoid `*` imports in code you write for this course and beyond.

Try It Yourself 1.15a: Using the standard library's `random` module, write a short program that imports just `randint` from it (`from random import randint`) and prints five random numbers between 1 and 100, using a loop from Lesson 1.6.

The Standard Library vs. Third-Party Packages

Every module you've imported so far, `json` in Lesson 1.13, `math` just now, is part of Python's standard library: it ships with every Python installation, with nothing extra to install. The standard library is large and covers an enormous range of everyday needs (dates and times, random numbers, file paths, mathematics, and far more), which is exactly the "batteries included" philosophy Lesson 1.1 first mentioned.

A third-party package is code written and published by someone outside Python's core development team, distributed separately and installed yourself; `requests` (for making web requests, Phase 3) is exactly this kind of package, one you'll install and use later in this course. You import a third-party package exactly the same way as a standard-library one, with a plain `import`; Python doesn't distinguish syntactically between the two. What differs is that a third-party package has to actually be installed on your machine first, which is exactly what Lesson 1.16 covers, using a tool called `pip`.

```
1 import json # standard library - always available, no install needed
2 import requests # third-party - must be installed first with pip
```

If you try to import a third-party package that isn't installed, Python raises `ModuleNotFoundError`, telling you exactly which name it couldn't find; that error is your signal to go install it, not a sign anything is broken.

Splitting Your Own Code Across Files

Everything so far has imported code someone else wrote. Importing between your own files works identically, and it's how real projects stay organized once a single file would otherwise grow unmanageably long. Suppose you have two files in the same folder:

```
1 # helpers.py
2 def greet(name):
3     return f"Hello, {name}!"
4
5 def square(n):
6     return n ** 2
```

```
1 # main.py
2 import helpers
3
4 print(helpers.greet("Ada")) # Hello, Ada!
5 print(helpers.square(5))   # 25
```

`import helpers` here works exactly like `import math` did earlier, because from Python's point of view there's no real difference: `helpers.py` is just another module, and Python finds it because it's sitting in the same folder as `main.py`. Every name `helpers.py` defines at its top level, both functions here, becomes reachable through `helpers.`, exactly the same rule that applied to `math.sqrt`. You can, of course, also use `from helpers import greet, square` to import specific names directly, following the exact same rules covered earlier in this lesson.

This is the standard way to organize a growing project: group related functions into a file named for what they do (`helpers.py`, `validators.py`, `database.py`), and import between them as needed, instead of letting one file accumulate everything.

What Happens When a Module Is Imported

It's worth understanding precisely what `import` actually does, since it explains behavior that otherwise looks surprising. The very first time a module is imported anywhere in a running program, Python executes that file from top to bottom, exactly once, running every top-level statement in it, not just defining the functions and classes inside it.

```
1 # greetings.py
2 print("greetings.py is being loaded!")
3
4 def say_hello():
5     print("Hello!")
```

```
1 # main.py
2 import greetings # prints "greetings.py is being loaded!" immediately
3
4 greetings.say_hello()
5 import greetings # does NOT print again - already loaded, reused instantly
```

The `print()` at the top level of `greetings.py` runs the moment it's first imported, which is why you'll often see genuine setup code, not just function definitions, at a module's top level. Importing the same module a second time, anywhere in the same running program, does not re-run the file; Python caches the already-loaded module and simply hands back the same one, instantly, which is both a performance optimization and the reason top-level code in a module should generally be safe to run exactly once.

The `if __name__ == "__main__":` Pattern

Every module has a built-in variable called `__name__`, and Python sets it automatically depending on how the file is being used: if the file is being run directly (`python3 greetings.py`), `__name__` is set to the string `"__main__"`; if the file is instead being imported from somewhere else, `__name__` is set to the module's own name instead. This lets a single file behave differently depending on which of those two situations it's in:

```
1 # greetings.py
2 def say_hello():
3     print("Hello!")
4
5 if __name__ == "__main__":
6     say_hello() # only runs when this file is executed directly
```

Running `python3 greetings.py` directly prints `"Hello!"`, since `__name__` is `"__main__"` in that case. But if another file does `import greetings`, that `if` block is skipped entirely, since `__name__` is `"greetings"` instead, not `"__main__"`; only the function definition happens, with nothing executing automatically. This pattern is genuinely everywhere in real Python code: it lets a file work both as a standalone script you can run directly and as a module other files can import cleanly, without unwanted code firing off just because it was imported.

Try It Yourself 1.15b: Create two files in the same folder: `calculator.py`, with a function `add(a, b)` that returns their sum, guarded so nothing runs automatically on import (using the pattern above with a small test print inside the `if` block), and `main.py`, which imports `calculator` and calls `calculator.add(3, 4)`. Run `calculator.py` directly first, then run `main.py`, and confirm the test print only appears in the first case.

Organizing Files Into a Package

Once a project has several related modules, grouping them into a folder, called a package, keeps things organized further still. A package is a folder containing an `__init__.py` file, which can be completely empty; its presence alone is what tells Python “treat this folder as an importable package,” not just an ordinary folder.

```
1 my_project/
2   main.py
3   utils/
4       __init__.py
5       text_helpers.py
6       math_helpers.py
```

With this layout, `main.py` can reach into the package using dotted names that mirror the folder structure:

```
1 # main.py
2 from utils import text_helpers
3 from utils.math_helpers import square
4
5 print(text_helpers.shout("hi"))
6 print(square(5))
```

`from utils import text_helpers` imports the `text_helpers.py` module from inside the `utils` package, letting you reach its contents through `text_helpers.`; `from utils.math_helpers import square` reaches one level deeper, directly importing a single name from a specific module inside the package. This nested, folder-based organization is exactly how larger real-world Python projects, including the ones you'll build starting in Phase 3, are

laid out: related functionality grouped into packages, each containing focused modules, rather than dozens of loose files or one enormous one.

Common Mistakes to Avoid

- **Using `from module import *`.** It hides where names actually came from and can silently overwrite names you've already defined. Import the whole module, or specific named pieces, instead.
- **Forgetting a third-party package needs installing first.** `import` requests without installing it raises `ModuleNotFoundError`; this means “go install it,” covered in Lesson 1.16, not that something is broken.
- **Naming your own file the same as a standard-library module.** A file named `math.py` in your own project shadows the real `math` module, and `import math` from anywhere in that project will load your file instead, usually breaking anything that expected the real one.
- **Expecting a module's top-level code to re-run on a second import.** It doesn't; Python caches the module after the first import and reuses it, so top-level `print()` statements and similar only ever fire once per run.
- **Putting real logic outside an `if __name__ == "__main__":` guard in a file meant to be imported.** Anything at a module's top level, outside that guard, runs automatically the instant the file is imported, even if that's not what you intended.

Lesson 1.15 Project: Personal Utility Package

Build a small package called `toolkit` that:

1. Is a folder containing an `__init__.py` (empty is fine), `text_tools.py`, and `number_tools.py`.
2. In `text_tools.py`, defines at least two functions (for example, one that reverses a string using slicing from Lesson 1.7, and one that counts vowels in a string).
3. In `number_tools.py`, defines at least two functions (for example, one that checks if a number is prime, and one that returns the average of a list).
4. Each of the two files ends with an `if __name__ == "__main__":` block containing a couple of `print()` calls that demonstrate its own functions, so each file can also be run directly on its own to test it.
5. Writes a separate `main.py`, outside the `toolkit` folder, that imports specific functions from both modules using `from toolkit.text_tools import ...` and `from toolkit.number_tools import ...`, and calls each of them with sample data.

Run each module file directly first to confirm its own test block works, then run `main.py` and confirm the test blocks do not run this time, proving you understand exactly when `__name__` equals `"__main__"` and when it doesn't.

Lesson Summary: Quick Reference

- A module is just a `.py` file; `import` runs it once and gives you access to what it defined
- `import x`, `from x import y`, `import x as z` are the standard forms; avoid `from x import *`
- The standard library ships with Python; third-party packages must be installed first, or you'll get `ModuleNotFoundError`
- A module's top-level code runs once, on first import, and is cached on every later import
- `if __name__ == "__main__":` runs code only when a file is executed directly, not when it's imported
- A folder becomes an importable package once it contains an `__init__.py` file

Lesson 1.16: Virtual Environments & Pip

Why This Matters

Lesson 1.15 drew a line between the standard library, which is always available, and third-party packages, which have to be installed. This lesson is about actually installing them, safely, and about a problem that catches nearly every beginner at least once: two different projects on the same computer needing two different, incompatible versions of the same package. Virtual environments are how professional Python projects avoid that problem entirely, and every project from Phase 3 onward in this course assumes you're comfortable creating one.

Learning Objectives

By the end of this lesson, you will be able to:

- Explain why installing packages globally, for your whole computer, eventually causes real problems
- Create, activate, and deactivate a virtual environment
- Install, list, and remove packages with pip
- Save and restore a project's exact dependencies using a `requirements.txt` file

The Problem: One Computer, Conflicting Needs

Imagine two projects on the same machine. Project A was built two years ago and only works correctly with version 1.0 of some package. Project B, started last week, needs version 3.0 of that exact same package, since the newer version changed how some function works. If packages install globally, one single copy shared by your entire computer, these two projects cannot both work at the same time; installing what B needs breaks A, and installing what A needs breaks B.

This isn't a rare, contrived scenario; it's the normal state of affairs once you have more than one or two Python projects going, and it's precisely the problem a virtual environment solves: instead of one global set of installed packages shared by everything, each project gets its own private, isolated set, completely independent of every other project's.

What a Virtual Environment Actually Is

A virtual environment is, underneath everything, just a folder, containing its own private copy of the Python interpreter and its own private place to install packages, entirely separate from your computer's main Python installation and from every other project's virtual environment. Creating one uses Python's built-in `venv` module, part of the standard library, no installation needed:

```
1 python3 -m venv venv
```

Read this command in three parts: `python3 -m venv` runs the `venv` module as a program (the `-m` flag means "run this module directly"), and the final `venv` is the name of the folder to create. Naming the folder `venv` is an extremely common convention, though it could be named anything; running this command creates a new folder called `venv` inside your current project folder, containing a private Python interpreter and an empty space for packages.

Activating and Deactivating

Creating a virtual environment doesn't put you inside it; you have to activate it explicitly, every time you start working on that project in a new terminal session:

```
1 # macOS / Linux
2 source venv/bin/activate
```

```
3  
4 # Windows  
5 venv\Scripts\activate
```

Once activated, your terminal prompt typically changes to show the environment's name in parentheses, like `(venv)`, as a constant visual reminder that it's active. From this point on, until you deactivate it, `python3` and `pip` both refer to this environment's private copies, not your computer's global ones; anything you install goes into this project's isolated space alone, invisible to every other project. What's actually happening underneath is worth knowing, since it demystifies the whole mechanism: activating an environment temporarily changes which `python3` and `pip` your terminal finds first when you type those commands, pointing them at the copies living inside `venv` instead of your computer's system-wide ones; deactivating simply reverses that, with nothing about your system's actual Python installation ever touched either way.

Deactivating returns you to your computer's global Python, and works identically on every operating system:

```
1 deactivate
```

It's worth building the habit of checking whether your prompt shows an active environment before installing anything; installing a package without an environment activated installs it globally, quietly recreating the exact conflict virtual environments exist to prevent.

Try It Yourself 1.16a: In your `python_practice` folder from Lesson 1.2, create a virtual environment named `venv`, activate it, and confirm your terminal prompt changed to show it's active. Then run `which python3` (macOS/Linux) or `where python` (Windows) and notice the path now points inside your project's `venv` folder, not your computer's main Python installation.

Installing Packages With pip

`pip` is Python's standard package installer, and it comes bundled with Python itself; you've had it available since Lesson 1.2, whether you've used it yet or not. With a virtual environment activated, installing a third-party package is one line:

```
1 pip install requests
```

This downloads `requests` (and anything it depends on) from the Python Package Index, the standard public repository of Python packages, and installs it into the currently active virtual environment. Once installed, `import requests` works exactly like any other import from Lesson 1.15, no different syntactically from importing the standard library's `json`.

`pip` supports a specific version too, useful when a project genuinely needs one exact version rather than whatever is newest:

```
1 pip install requests==2.31.0
```

A few other `pip` commands cover most of what you'll need day to day:

Command	What it does
<code>pip install package</code>	Installs the latest version of a package
<code>pip install package==1.0</code>	Installs a specific version
<code>pip uninstall package</code>	Removes an installed package
<code>pip list</code>	Lists every package installed in the current environment
<code>pip show package</code>	Shows details about one installed package, including its version

Saving Dependencies: `requirements.txt`

A virtual environment itself is never shared or uploaded anywhere, it's often genuinely gigabytes in size once a project has several packages installed, and it's tied to your specific computer. What gets shared instead is a plain text file listing exactly which packages, and which versions, the project depends on, so anyone else (or you, on a different computer, or a server deploying your project) can recreate an identical environment. `pip` generates this file for you directly from what's currently installed:

```
1 pip freeze > requirements.txt
```

`pip freeze` prints every installed package and its exact version, one per line; the `>` redirects that output into a file called `requirements.txt` instead of printing it to the terminal. A typical result looks like this:

```
1 requests==2.31.0
2 python-dateutil==2.8.2
3 certifi==2024.2.2
```

Anyone who has this file, on any computer, can recreate the exact same set of packages, at the exact same versions, with a single command, after creating and activating their own virtual environment:

```
1 pip install -r requirements.txt
```

`-r requirements.txt` tells `pip` “install everything listed in this file,” rather than naming one package at a time. This is precisely why `requirements.txt` gets committed to version control while the `venv` folder itself never does: the file is small, portable, and completely reproducible; the folder is large, machine-specific, and trivially regenerated from the file whenever it's actually needed.

Try It Yourself 1.16b: With your virtual environment from the earlier Try It Yourself still activated, install `requests` using `pip install requests`, then run `pip freeze > requirements.txt` and open the resulting file to see what it contains. Then run `pip uninstall requests` and confirm with `pip list` that it's gone.

The Complete Workflow

Putting every piece from this lesson together, this is the standard sequence you'll repeat at the start of essentially every project from here forward:

```
1 # 1. Create a virtual environment (once, per project)
2 python3 -m venv venv
3
4 # 2. Activate it (every time you start working)
5 source venv/bin/activate
6
```

```
7 # 3. Install what you need
8 pip install requests
9
10 # 4. Save exactly what's installed
11 pip freeze > requirements.txt
12
13 # 5. When you're done for now
14 deactivate
```

And on a different computer, or after cloning a project someone else built:

```
1 # 1. Create your own fresh environment
2 python3 -m venv venv
3
4 # 2. Activate it
5 source venv/bin/activate
6
7 # 3. Recreate the exact same dependencies
8 pip install -r requirements.txt
```

This exact five-step rhythm, create, activate, install, freeze, deactivate, is worth committing to memory; it's the beginning of essentially every Python project you'll build from Phase 3 onward.

Common Mistakes to Avoid

- **Installing packages without an activated virtual environment.** This installs globally, quietly recreating the version-conflict problem this entire lesson exists to solve. Check that your prompt shows an active environment before running `pip install`.
- **Committing the venv folder itself to a shared project.** It's large, specific to one computer, and entirely regenerable from `requirements.txt`; share the requirements file, never the folder.
- **Forgetting to activate the environment in a new terminal window.** Activation only lasts for the terminal session it was run in; opening a new terminal tab or window means activating again.
- **Running `pip freeze` without an environment activated.** This captures every package installed globally on your entire computer, not just what this one project actually needs, producing a bloated, inaccurate `requirements.txt`.
- **Assuming `deactivate` deletes the environment.** It doesn't; it just returns your terminal to the global Python. The environment folder, and everything installed in it, is untouched and ready to reactivate later.

Lesson 1.16 Project: Reproducible Project Setup

Starting from a brand new, empty folder:

1. Create and activate a virtual environment called `venv`.
2. Install two third-party packages of your choosing (`requests` and one other, such as `colorama` or `python-dateutil`).
3. Write a small script, `check_env.py`, that imports both packages and prints something using each (for example, `requests.__version__`), proving they're actually installed and reachable.
4. Generate a `requirements.txt` with `pip freeze`.

5. Deactivate the environment, then delete the venv folder entirely (simulating handing this project to someone else, or moving to a new computer).
6. Recreate a fresh virtual environment, activate it, and reinstall everything using only `pip install -r requirements.txt`. Run `check_env.py` again to confirm it still works, with a completely regenerated environment.

This project proves the entire point of this lesson concretely: nothing about a project's actual dependencies was lost by deleting venv, because `requirements.txt` was always enough to rebuild it.

Lesson Summary: Quick Reference

- A virtual environment is an isolated, per-project set of installed packages, avoiding version conflicts between projects
- `python3 -m venv venv` creates one; `source venv/bin/activate` (macOS/Linux) or `venv\Scripts\activate` (Windows) activates it; `deactivate` exits it
- `pip install package` installs; `pip uninstall package` removes; `pip list` shows what's installed
- `pip freeze > requirements.txt` captures exact installed versions; `pip install -r requirements.txt` recreates them elsewhere
- Share `requirements.txt`; never share the venv folder itself

Lesson 1.17: Everyday Libraries

Why This Matters

Lesson 1.15 introduced `import` and the standard library in the abstract; Lesson 1.13 already put two standard-library modules, `json` and `pathlib`, to real, practical use. This lesson closes out Phase 1 by touring four more standard-library modules you will reach for constantly, in nearly every program you write from here forward: dates and times, randomness, everyday math, and two collection types that solve extremely common problems in a single line. None of this is new syntax; it's new, genuinely useful vocabulary built entirely on everything else in this phase.

Learning Objectives

By the end of this lesson, you will be able to:

- Get the current date and time, format it for display, and do basic arithmetic on dates with `datetime`
- Generate random numbers and make random choices with `random`
- Use math for the everyday calculations `+` `-` `*` `/` can't do alone
- Use `collections.Counter` and `collections.defaultdict` to simplify patterns you've been writing by hand since Lesson 1.9

Working With Dates and Times: `datetime`

The `datetime` module is how Python represents a specific point in time, or a specific calendar date, as a real object you can format, compare, and do arithmetic on, rather than as plain text you'd have to parse yourself.

```
1 from datetime import datetime
2
3 now = datetime.now()
4 print(now)           # 2026-03-14 15:42:07.123456
5 print(now.year)      # 2026
6 print(now.month)     # 3
7 print(now.day)       # 14
8 print(now.hour)      # 15
```

`datetime.now()` returns a `datetime` object representing the exact current moment; its `.year`, `.month`, `.day`, `.hour`, `.minute`, and `.second` attributes give you each individual piece, already converted to the right type (`int`), with no string parsing required at all. This is worth appreciating directly: if dates were just plain text, like `"2026-03-14"`, getting the year back out would mean slicing the string apart by hand, exactly the kind of fragile, error-prone work Lesson 1.7's slicing lesson warned about; a real `datetime` object instead already knows its own year, month, and day as separate, correctly-typed values, ready to use directly.

Printing a `datetime` object directly gives you a technically-correct but not especially readable format. `.strftime()` ("string format time") formats it however you actually want, using a set of codes that stand for specific pieces:

```
1 now = datetime.now()
2 print(now.strftime("%Y-%m-%d"))    # 2026-03-14
3 print(now.strftime("%B %d, %Y"))   # March 14, 2026
4 print(now.strftime("%I:%M %p"))    # 03:42 PM
```

`%Y` is a four-digit year, `%m` a zero-padded month, `%d` a zero-padded day, `%B` the full month name, `%I` a 12-hour hour, `%M` minutes, and `%p` AM/PM; everything else in the format string, like the dashes and commas above, is passed through

exactly as typed. You won't memorize every code the first time you see this table, and you don't need to; looking up the specific code you need, when you need it, is completely normal, expected practice, exactly as Lesson 1.1 already promised about this entire course.

Two other constructors are worth knowing specifically. `date.today()` gives you just a calendar date, with no time-of-day component at all, useful when the time genuinely doesn't matter:

```
1 from datetime import date
2
3 today = date.today()
4 print(today)           # 2026-03-14
5 print(today.weekday()) # 5 - Monday is 0, Sunday is 6
```

And `timedelta` represents a span of time, a duration, rather than a specific moment, which lets you do genuinely useful date arithmetic:

```
1 from datetime import datetime, timedelta
2
3 now = datetime.now()
4 tomorrow = now + timedelta(days=1)
5 next_week = now + timedelta(weeks=1)
6 three_hours_ago = now - timedelta(hours=3)
7
8 print(tomorrow.strftime("%Y-%m-%d"))
```

Adding or subtracting a `timedelta` from a `datetime` produces a brand new `datetime`, shifted by exactly that amount; this is the standard, correct way to answer “what date is 30 days from now,” rather than trying to manually account for how many days are in each month yourself.

Try It Yourself 1.17a: Print today's date formatted as "Day, Month DD, YYYY" (for example, "Saturday, March 14, 2026", which needs the `%A` code for full weekday name, on top of the codes shown above). Then calculate and print the date exactly 90 days from today.

Generating Randomness: `random`

The `random` module covers everything from picking a single random number to shuffling an entire list, and it's genuinely everywhere: games, simulations, sampling data, generating test values.

```
1 import random
2
3 print(random.randint(1, 6))      # a random int, 1 through 6, BOTH ends included
4 print(random.random())          # a random float, 0.0 up to (not including) 1.0
5 print(random.uniform(1.5, 9.5)) # a random float within a specific range
```

`random.randint(a, b)` is the one you'll use most often for whole numbers; unlike `range()` from Lesson 1.6, both endpoints are included, so `randint(1, 6)` can genuinely return 6, simulating a single die roll exactly.

Beyond generating fresh numbers, `random` also works directly on existing sequences, lists, tuples, strings, from Lessons 1.7 and 1.8:

```
1 fruits = ["apple", "banana", "cherry", "date"]
2
3 print(random.choice(fruits))    # one random item
```

```

4 print(random.sample(fruits, 2)) # 2 different random items, no repeats
5 random.shuffle(fruits)
6 print(fruits) # the list, reordered in place, permanently

```

`random.choice(sequence)` picks a single random item. `random.sample(sequence, n)` picks `n` distinct items with no repeats, returned as a new list, leaving the original untouched. `random.shuffle(list)` reorders a list randomly in place, exactly like `.sort()` from Lesson 1.8 modifies in place and returns `None`; it only works on an actual list, since shuffling requires reassigning positions, which an immutable tuple or string can't allow.

Try It Yourself 1.17b: Simulate rolling two six-sided dice 10 times using a loop and `random.randint(1, 6)`, printing each roll's total. Then, given a list of five friends' names, use `random.choice()` to pick who goes first, and `random.shuffle()` to put the rest in a random turn order.

Everyday Math: `math`

You met `math` briefly as an example in Lesson 1.15; this section covers the specific functions from it you'll actually reach for. Recall that `**` from Lesson 1.4 already covers exponents, so `math` fills in the operations arithmetic operators genuinely can't:

```

1 import math
2
3 print(math.sqrt(16))      # 4.0 - square root
4 print(math.ceil(4.2))    # 5 - rounds UP to the nearest whole number
5 print(math.floor(4.8))   # 4 - rounds DOWN to the nearest whole number
6 print(math.pi)          # 3.141592653589793
7 print(math.e)            # 2.718281828459045

```

`math.ceil()` and `math.floor()` are worth comparing directly to `round()` from Lesson 1.3: `round()` goes to whichever whole number is mathematically closest, while `ceil` always rounds up and `floor` always rounds down, regardless of how close the decimal part is to either side; `math.ceil(4.01)` is still 5, not 4.

```

1 print(math.factorial(5))  # 120 - same as the recursive example from Lesson 1.11
2 print(math.gcd(12, 18))  # 6 - greatest common divisor
3 print(math.log(100, 10)) # 2.0 - log base 10 of 100

```

`math.factorial()` does exactly what the recursive `factorial()` function from Lesson 1.11 did by hand, provided for you, tested and optimized; recognizing when the standard library already solves a problem you were about to write a loop or recursive function for yourself is a genuinely valuable habit as you write more real code.

Two Collections Worth Knowing: `collections.Counter` and `collections.defaultdict`

Lesson 1.9 built a word-frequency counter by hand, checking whether each key already existed before incrementing it. That exact pattern, counting occurrences of things, is so common that the standard library provides a purpose-built tool for it.

```

1 from collections import Counter
2
3 text = "the quick brown fox jumps over the lazy dog the fox runs"
4 words = text.split()
5
6 word_counts = Counter(words)
7 print(word_counts)
8 # Counter({'the': 3, 'fox': 2, 'quick': 1, 'brown': 1, ...})

```

```
9
10 print(word_counts["the"])          # 3 - works just like a dictionary
11 print(word_counts.most_common(2))
12 # [('the', 3), ('fox', 2)] - the 2 most frequent items, as (item, count) tuples
```

`Counter(iterable)` does, in one line, exactly what the manual loop from Lesson 1.9 did over several: it counts every item in whatever you hand it and hands back something that behaves like a dictionary, item to count, with one genuinely useful extra: `.most_common(n)` instantly gives you the `n` most frequent items, sorted, without you writing any sorting logic yourself. A `Counter` also handles a missing key gracefully, unlike a plain dictionary's square brackets from Lesson 1.9: asking for the count of something that never appeared returns 0, not a `KeyError`.

```
1 print(word_counts["python"]) # 0 - not a KeyError, just 0
```

`defaultdict` solves a related but different problem: building up a dictionary where each value is itself a collection, like a list, which normally requires checking whether the key exists yet before you can append to it.

```
1 from collections import defaultdict
2
3 groups = defaultdict(list)
4
5 students = [("Ada", "Math"), ("Alan", "Science"), ("Grace", "Math")]
6
7 for name, subject in students:
8     groups[subject].append(name)
9
10 print(dict(groups))
11 # {'Math': ['Ada', 'Grace'], 'Science': ['Alan']}
```

Compare this directly to how Lesson 1.9 would have required writing this: checking `if subject not in groups:` and creating an empty list before every single `.append()`. `defaultdict(list)` removes that check entirely; the moment you access a key that doesn't exist yet, it automatically creates it with an empty list (whatever you passed to `defaultdict`, here `list`, is called to produce the default value) before your code even runs, so `.append()` always has something valid to append to, on the very first use of any key.

Try It Yourself 1.17c: Given `grades = ["A", "B", "A", "C", "B", "A", "B"]` from the Lesson 1.9 Try It Yourself, redo that exercise using `Counter` instead of a manual loop, and use `.most_common(1)` to find the single most common grade in one line.

Choosing the Right Tool

Four modules, four different everyday jobs, and this is worth holding onto as a habit more than as a list to memorize: before writing a loop or a manual check to solve a problem, it's worth pausing to ask whether the standard library already solves it. Dates and durations belong to `datetime`; randomness of any kind belongs to `random`; calculations beyond basic arithmetic belong to `math`; and counting or grouping things, patterns you've now written by hand in Lesson 1.9, are very often better served by `collections`. Every one of these, exactly like `json` and `pathlib` in Lesson 1.13, ships with Python already; the only new skill is knowing they exist and reaching for `import` instead of reinventing them.

Common Mistakes to Avoid

- **Forgetting `randint()`'s upper bound is inclusive.** Unlike `range()`, `random.randint(1, 6)` can return 6. Mixing up this rule with `range()`'s exclusive stop is an easy, common mistake.
- **Using `round()` when you actually meant `math.ceil()` or `math.floor()`.** `round()` goes to the nearest value; if you specifically need “always up” or “always down” regardless of how close the decimal is, `round()` is the wrong tool.
- **Calling `.shuffle()` on a tuple or expecting it to return a new list.** `.shuffle()` only works on a mutable sequence (a list) and modifies it in place, returning `None`, exactly like `.sort()` from Lesson 1.8.
- **Reaching for a manual counting loop out of habit.** If you're building a dictionary purely to count occurrences, `Counter` almost always says the same thing in less code, with `.most_common()` as a genuine bonus.
- **Forgetting `datetime.now()` includes the time, when you only wanted the date.** Comparing two `datetime.now()` calls from different moments of the same day will differ down to the microsecond; use `date.today()` when only the calendar date matters.

Lesson 1.17 Project: Daily Journal Generator

Write a program called `journal_generator.py` that, bringing together tools from this entire phase:

1. Uses `datetime` to record the current date and time, formatted with `.strftime()` as a readable header (for example, "Saturday, March 14, 2026 - 3:42 PM").
2. Uses `random.choice()` to select one writing prompt from a list of at least five you write yourself (for example, "What's one thing that went well today?").
3. Asks the user, with `input()`, to write a short journal entry responding to that prompt.
4. Using `pathlib` (Lesson 1.13), creates a `journal` folder if it doesn't exist, and appends the date, prompt, and entry to a file called `journal/entries.json`, structured as a JSON list of objects (each with `date`, `prompt`, and `entry` keys), reading the existing file first if it's already there so previous entries aren't lost.
5. Wraps the file reading and writing in `try/except` (Lesson 1.14) to handle the case where `entries.json` doesn't exist yet or contains invalid JSON, starting fresh with an empty list in either case.
6. Finally, uses `collections.Counter` on every prompt that's ever been used across all saved entries, and prints the single most common prompt with `.most_common(1)`, along with the total number of entries saved.

This project is a deliberate capstone for all of Phase 1: types and input, file handling and JSON, error handling, and this lesson's everyday libraries, all working together in one small, genuinely useful program.

Lesson Summary: Quick Reference

- `datetime.now()` / `date.today()` get the current moment or date; `.strftime()` formats it; `timedelta` adds or subtracts a duration
- `random.randint(a, b)` (inclusive both ends), `random.choice()`, `random.sample()`, `random.shuffle()` cover most everyday randomness
- `math.sqrt()`, `math.ceil()`/`math.floor()`, `math.factorial()` cover calculations beyond basic arithmetic operators
- `Counter(iterable)` counts occurrences in one line, with `.most_common(n)` built in; missing keys return 0, not `KeyError`

- `defaultdict(list)` auto-creates a default value for a missing key, removing manual existence checks before appending
- Before writing a manual loop for a common problem, check whether the standard library already solves it

Phase 1 Capstone Project: Personal Library Manager

Every lesson project in this phase focused on the one or two skills that lesson had just introduced. This capstone is different: it's a single, larger program that deliberately pulls together tools from across the entire phase, the way a real, small utility program actually would. Nothing here is a new concept; every piece was covered in a lesson you've already completed. Building this is the best evidence you'll have, going into Phase 2, that these seventeen lessons genuinely fit together into one coherent skill, not seventeen separate, disconnected ones.

What You're Building

A command-line program, `library_manager.py`, that manages a small personal book collection: adding books, marking them as read, searching, and keeping the whole collection saved to disk as JSON between runs, so nothing is lost when the program closes.

Capstone Requirements

1. **Data structure (Lessons 1.8, 1.9).** Represent the library as a list of dictionaries, each with `title`, `author`, `genre`, `year`, and `is_read` (a bool) keys.
2. **Persistence (Lesson 1.13).** On startup, use `pathlib` to check whether `library.json` exists; if it does, load it with `json.load()`. If it doesn't (or contains invalid JSON), start from an empty list, handling both cases with `try/except` (Lesson 1.14). Every time the collection changes, save it back with `json.dump(..., indent=2)`.
3. **Adding a book.** Write a function `add_book(library, title, author, genre, year)` (Lesson 1.11) with full type hints and a docstring (Lesson 1.12), returning the updated list. Validate that `year` is a reasonable four-digit number, raising a `ValueError` (Lesson 1.14) with a clear message if not.
4. **Marking a book read.** Write a function that finds a book by title (using a loop or a comprehension, Lessons 1.6/1.10) and flips its `is_read` to `True`, printing a clear message if no book with that title exists.
5. **Searching and filtering.** Write functions to find every book by a given author, every book in a given genre, and every unread book, each using a list comprehension (Lesson 1.10).
6. **Statistics.** Using `collections.Counter` (Lesson 1.17), report how many books exist per genre, and print the most common genre in the collection with `.most_common(1)`.
7. **A menu loop.** Wrap everything in a `while` loop (Lesson 1.6) that repeatedly asks the user, with `input()`, what they'd like to do (add a book, mark one read, search, view statistics, or quit), calling the appropriate function for each choice, and saving to `library.json` after every change.
8. **Code organization (Lesson 1.15).** Split the program across at least two files: `library.py` holding every function that operates on the library data, and `main.py` holding only the menu loop, importing what it needs from `library`. Guard any test code in `library.py` with `if __name__ == "__main__":`.

Stretch Goals

If the core requirements feel comfortable, extend the project further, entirely with tools you already have: add a `date_added` field using `datetime.date.today()` (Lesson 1.17) whenever a book is added; add a function that recommends a random unread book using `random.choice()` (Lesson 1.17); or export the unread list to its own separate file using `pathlib`.